Junicode

the font for medievalists

specimens and user manual

for version 2.224

Contents

1. About Junicode

Junicode is modeled on the Pica Roman type purchased by Oxford University in 1692 and used to set the bulk of the Latin text of George Hickes, *Linguarum vett. septentrionalium thesaurus grammatico-criticus et archaeologicus* (Oxford, 1703–5). This massive two-volume folio is not only a major work of scholarship on the languages and literatures of northern Europe in the Middle Ages, but also a fine example of the work of the Oxford Press at this period: printed in multiple types (for every language had to have its proper type) and lavishly illustrated with engravings of manuscript pages, coins and artifacts. This book is the model for Junicode.

This font also includes two other typefaces from the *Thesaurus*: Pica Saxon, used to set passages in the Old English language, and a typeface reproducing the Gothic alphabet ("Gothic" here being not the late medieval style, but rather the earliest extensively attested Germanic language). These were commissioned by the literary scholar Franciscus Junius (1591–1677) and bequeathed by him to the University. Examples of all these typefaces can be found in *A Specimen of the Several Sorts of Letter Given to the University by Dr. John Fell, Sometime Lord Bishop of Oxford. To Which Is Added the*

Letter Given by Mr. F. Junius (Oxford, 1693).[1]

Junicode has two distinct Greek faces. The first, newly designed to harmonize with the roman face, is upright and modern. The other, accompanying the italic face, is based on type designed by Alexander Wilson (1714–86) of Glasgow and used in numerous books published by the Foulis Press, most notably the great Glasgow Homer of 1756–58.

The Junicode project began around 1998, when the developer began to revise his older (early 1990s) "Junius" fonts for medievalists to take account of the Unicode standard, then relatively new. The font's name, a contraction of "Junius Unicode," was supposed to be a stopgap, serving until a more suitable name could be found, but "Junicode" quickly stuck, and it is now so well known that it can't be changed.[2] The project has been active for its entire history, responding to frequent requests from users and changes in font technology; a particular focus of recent versions of Junicode (numbered 2.000 and higher) is the promotion of best practices in the presentation of medieval texts, especially in the area of accessibility. This aspect of the font is explored in the Introduction to the Feature Reference.

[1] There is a facsimile of this work at the Digital Bodleian.

[2] An effort to change the name to "JuniusX" produced only confusion. If you find a font by the name JuniusX on a free font site, that is nothing more than an early version of Junicode 2.

2. Specimens

Old and Middle English

Wē æthrynon mid ūrum ārum þā ȳðan þæs dēopan wǣles; wē ġesāwon ēac þā muntas ymbe þǣre sealtan sǣ strande, and wē mid āðēnedum hræġle and ġesundfullum windum þǣr ġewīcodon on þām ġemǣrum þǣre fæġerestan þēode. Þā ȳðan ġetācniað þisne dēopan cræft, and þā muntas ġetācniað ēac þā miċelnyssa þisses cræftes. (Regular)

Siþen þe sege and þe assaut watz sesed at Troye,
Þe borȝ brittened and brent to brondez and askez,
Þe tulk þat þe trammes of tresoun þer wroȝt
Watz tried for his tricherie, þe trewest on erthe:
Hit watz Ennias þe athel, and his highe kynde,
Þat siþen depreced prouinces, and patrounes bicome
Welneȝe of al þe wele in þe west iles. (SemiExpanded)

Apply the OpenType feature ss02 (Stylistic Set 2) for insular letter-forms.

Heꞃ cẏneƿulꝼ benam ꞅıᵹebꞃẏhꞇ hıꞅ ꞃıceꞅ ⁊ ƿeſtꞅeaxna ƿıoꞇan ꝼoꞃ unꞃẏhꞇum ꝺęꝺū buꞇon hamꞇúnꞅcıꞃe ⁊ he hæꝼꝺe þa oþ he oꝼꞅloᵹ þone alꝺoꞃmon þe hī lenᵹeſt ƿunoꝺe ⁊ hıene þa cẏneƿulꝼ on anꝺꞃeꝺ aꝺꞃæꝼꝺe ⁊ ꝥ þæꞃ ƿunaꝺe oþ þæꞇ hıne án ꞅƿán oꝼſtanᵹ æꞇ ƿꞃẏꝼeꞇeꞅꝼloꝺan ⁊ he ƿꞃęc þone alꝺoꞃmon cumbꞃan ⁊ ꞅe cẏneƿulꝼ oꝼꞇ mıclum ᵹeꝼeohꞇum ꝼeahꞇ uuıþ bꞃeꞇƿalū. (SemiCondensed)

Old Irish

Fect n-oen do Ailill ⁊ do Meidb íar n-dergud a rígleptha dóib i Cruachanráith Chonnacht, arrecaim

comrad chind-cherchailli eturru. Fírbriathar, a ingen, bar Ailill, is maith ben ben dagfir. Maith omm, bar ind ingen. Cid diatá latsu ón. Is de atá lim, bar Ailill, ar it ferr-su indiu indá in lá thucus-sa thu. (Condensed Medium)

For insular letter-forms, apply the OpenType feature ss02 (Stylistic Set 2), making sure the language is set to Irish.

bamaith-fe fhemut, an Medb. If maith nach cualammar ⁊ nach fetammar, an Ailill, acht do bithfiu an bantincur mnaa ⁊ bidba na crich ba nerrom duit oc breith do flait ⁊ do chrech i fúatach úait. Ni framlaid bára, an Medb, acht m'athair i n-andrigi hErenn .i. Eocho Feidlech mac Find meic Findomain meic Findeoin meic Findguni meic Rogein Rúaid meic Rigéoin meic blathachta meic beothechta meic Enna Agnig meic Oengusa Turbig. bátar aice fe ingena d'ingenaib: Derbriu, Ethi ⁊ Éle, Clothru, Mugain, Medb, merri ba uarliu ⁊ ba urraitiu díb. (Regular)

For a (somewhat) uncial look, try combining ss02 with smcp (Small Caps), adding other variants as you see fit.

bamaith-fe fhemut, an Meob. If maith nach cualammar ⁊ nach fetammar, an Ailill, acht do bithfiu an bantincur mnaa ⁊ bidba na crich ba nerrom duit oc breith do flait ⁊ do chrech i fúatach úait. Ni framlaid bára, an Meob, acht m'athair i n-andrigi hErenn .i. Eocho Feidlech mac Find meic Findomain meic Findeoin meic Findguni meic Rogein Rúaid meic Rigéoin meic blathachta meic beothechta meic Enna Agnig meic Oengusa Turbig. bátar aice fe ingena d'ingenaib: Derbriu, Ethi ⁊ Éle, Clothru, Mugain, Meob, merri ba uarliu ⁊ ba urraitiu díb. (Regular)

Old Icelandic

For Nordic shapes of þ and ð in an English context, specify the appropriate language (e.g. Icelandic or Norwegian); or apply the OpenType ss01 (Stylistic Set 1) feature.

Um haustit sendi Mǫrðr Valgarðsson orð at Gunnarr myndi vera einn heimi, en lið alt myndi vera niðri í eyjum at lúka heyverkum. Riðu þeir Gizurr Hvíti ok Geirr Goði austr yfir ár, þegar þeir spurðu þat, ok austr yfir sanda til Hofs. Þá sendu þeir orð Starkaði undir Þríhyrningi; ok fundusk þeir þar allir er at Gunnari skyldu fara, ok

réðu hversu at skyldi fara. (SemiExpanded Medium)

Runic

Junicode has features for automated transliteration of Latin letters into various runic systems.

ᚠᛁᛋᚳ ᚠᛚᚩᛞᚢ ᚪᚻᚩᚠ ᚩᚾ ᚠᛖᚱᚷᛖᚾᛒᛖᚱᛁᚷ ᚹᚪᚱᚦ ᚷᚪ:ᛊᚱᛁᚳ ᚷᚱᚩᚱᚾ ᚦᚫᚱ ᚻᛖ ᚩᚾ ᚷᚱᛖᚢᛏ
ᚷᛁᛊᚹᚩᛗ ᚻᚱᚩᚾᚫᛊ ᛒᚪᚾ
ᚱᚩᛗᚹᚪᛚᚢᛊ ᚪᚾᛞ ᚱᛖᚢᛗᚹᚪᛚᚢᛊ ᛏᚹᛟᚷᛖᚾ ᚷᛁᛒᚱᚩᚦᚫᚱ ᚪᚠᛟᛞᛞᚫ ᛞᛁᚫ ᚹᚣᛚᛁᚠ ᚩᚾ ᚱᚩᛗᚫ
ᚳᚪᛊᛏᛁ: ᚩᚦᛚᚫ ᚢᚾᚾᛖᚷ (Expanded)

German

Ich ſag üch aber / minen fründen / Foͤꝛchtēd üch nit voꝛ denen die den lyb toͤdend / vnd darnach nichts habennd das ſy mer thůgind. Ich wil üch aber zeigē voꝛ welchem ir üch foͤꝛchten ſollend. Foͤꝛchtend üch voꝛ dem / der / nach dem er toͤdet hat / ouch macht hat zewerffen inn die hell: ja ich ſag üch / voꝛ dem ſelben foͤꝛchtēd üch. Koufft man nit fünff Sparen vm̄ zween pfennig? (Condensed)

Latin

Junicode contains the most common Latin abbreviations, making it suitable for diplomatic editions of Latin texts.

Adiuuanos dſ̄ ſalutariſ noſter & ꝓpt̄ głam nominiſ tui dnē liƀanoſ· & ꝓpitiuſ eſto peccatiſ noſtriſ ꝓpter nomen tuum· Ne foꝛte dicant ingentib: ubi eſt dſ̄ eoꝛum & innoteſcat innationib: coꝛā oculiſ nr̄iſ· Poſuerunt moſticina ſeruoꝛū ruoꝛū eſcaſ uolatilib: cęli carneſ ſcōꝝ tuoꝝ beſtiiſ tenice· Facti ſumꝰ obꝓbꝛium uiciniſ nr̄iſ· (Light)

Gothic

jabai auk ƕas gasaiƕiþ þuk þana habandan kunþi in galiuge stada anakumbjandan, niu miþwissei is siukis wisandins timrjada du galiugagudam gasaliþ matjan? fraqistniþ auk sa unmahteiga ana þeinamma witubnja broþar in þize Xristus gaswalt. swaþ þan frawaurkjandans wiþra broþruns, slahandans ize gahugd siuka, du Xristau frawaurkeiþ. (SemiCondensed Light)

Use ss19 to produce Gothic letters automatically from transliterated text.

𐌾𐌰𐌱𐌰𐌹 𐌰𐌿𐌺 𐍈𐌰𐍃 𐌲𐌰𐍃𐌰𐌹𐍈𐌹𐌸 𐌸𐌿𐌺 𐌸𐌰𐌽𐌰 𐌷𐌰𐌱𐌰𐌽𐌳𐌰𐌽 𐌺𐌿𐌽𐌸𐌹 𐌹𐌽 𐌲𐌰𐌻𐌹𐌿𐌲𐌴 𐍃𐍄𐌰𐌳𐌰 𐌰𐌽𐌰𐌺𐌿𐌼𐌱𐌾𐌰𐌽𐌳𐌰𐌽, 𐌽𐌹𐌿 𐌼𐌹𐌸𐍅𐌹𐍃𐍃𐌴𐌹 𐌹𐍃 𐍃𐌹𐌿𐌺𐌹𐍃 𐍅𐌹𐍃𐌰𐌽𐌳𐌹𐌽𐍃 𐍄𐌹𐌼𐍂𐌾𐌰𐌳𐌰 𐌳𐌿 𐌲𐌰𐌻𐌹𐌿𐌲𐌰𐌲𐌿𐌳𐌰𐌼 𐌲𐌰𐍃𐌰𐌻𐌹𐌸 𐌼𐌰𐍄𐌾𐌰𐌽? 𐌾𐌰𐌱𐌰𐌹 𐌰𐌿𐌺 𐍈𐌰𐍃 𐌲𐌰𐍃𐌰𐌹𐍈𐌹𐌸 𐌸𐌿𐌺 𐌸𐌰𐌽𐌰 𐌷𐌰𐌱𐌰𐌽𐌳𐌰𐌽 𐌺𐌿𐌽𐌸𐌹 𐌹𐌽 𐌲𐌰𐌻𐌹𐌿𐌲𐌴 𐍃𐍄𐌰𐌳𐌰 𐌰𐌽𐌰𐌺𐌿𐌼𐌱𐌾𐌰𐌽𐌳𐌰𐌽, 𐌽𐌹𐌿 𐌼𐌹𐌸𐍅𐌹𐍃𐍃𐌴𐌹 𐌹𐍃 𐍃𐌹𐌿𐌺𐌹𐍃 𐍅𐌹𐍃𐌰𐌽𐌳𐌹𐌽𐍃 𐍄𐌹𐌼𐍂𐌾𐌰𐌳𐌰 𐌳𐌿 𐌲𐌰𐌻𐌹𐌿𐌲𐌰𐌲𐌿𐌳𐌰𐌼 𐌲𐌰𐍃𐌰𐌻𐌹𐌸 𐌼𐌰𐍄𐌾𐌰𐌽? (SemiExpanded Bold)

Sanskrit Transliteration

mānaṁ dvividhaṁ viṣayadvai vidyātśaktyaśaktitaḥ
arthakriyāyāṁ keśadirnārtho ’narthādhimokṣataḥ

sadr̥śāsadr̥śatvācca viṣayāviṣayatvataḥ
śabdasyānyanimittānāṁ bhāve dhīsadasattvataḥ (SemiCondensed Medium)

International Phonetic Alphabet

hwɑn θɑt ɑːprɪl wiθ is ʃuːrəs soːtə θə drʊxt ɔf marʧ hɑθ peːrsəd toː θə roːte and bɑːðəd ɛvrɪ væɪn ɪn swɪʧ lɪkuːr ɔf hwɪʧ vɛrtɪu ɛnʤɛndrəd ɪs θə fluːr hwɑn zɛfirʊs eːk wɪθ hɪs sweːtə bræːθ (Regular)

Greek

βίβλος γενέσεως ἰησοῦ χριστοῦ υἱοῦ δαυὶδ υἱοῦ ἀβραάμ. ἀβραὰμ ἐγέννησεν τὸν ἰσαάκ, ἰσαὰκ δὲ ἐγέννησεν τὸν ἰακώβ, ἰακὼβ δὲ ἐγέννησεν τὸν ἰούδαν καὶ τοὺς ἀδελφοὺς αὐτοῦ, ἰούδας δὲ ἐγέννησεν τὸν φάρες καὶ τὸν ζάρα ἐκ τῆς θαμάρ, φάρες δὲ ἐγέννησεν τὸν ἑσρώμ, ἑσρὼμ δὲ ἐγέννησεν τὸν ἀράμ, ἀρὰμ δὲ ἐγέννησεν τὸν ἀμιναδάβ, ἀμιναδὰβ δὲ ἐγέννησεν τὸν ναασσών, ναασσὼν δὲ ἐγέννησεν τὸν σαλμών, σαλμὼν δὲ ἐγέννησεν τὸν βόες ἐκ τῆς ῥαχά (Regular)

βίβλος γενέσεως ἰησοῦ χριστοῦ υἱοῦ δαυὶδ υἱοῦ ἀβραάμ. ἀβραὰμ ἐγέννησεν τὸν ἰσαάκ, ἰσαὰκ δὲ ἐγέννησεν τὸν ἰακώβ, ἰακὼβ δὲ ἐγέννησεν τὸν ἰούδαν καὶ τοὺς ἀδελφοὺς αὐτοῦ, ἰούδας δὲ ἐγέννησεν τὸν φάρες καὶ τὸν ζάρα ἐκ τῆς θαμάρ, φάρες δὲ ἐγέννησεν τὸν ἑσρώμ, ἑσρὼμ δὲ ἐγέννησεν τὸν ἀράμ, ἀρὰμ δὲ ἐγέννησεν τὸν ἀμιναδάβ, ἀμιναδὰβ δὲ ἐγέννησεν τὸν ναασσών, ναασσὼν δὲ ἐγέννησεν τὸν σαλμών, σαλμὼν δὲ ἐγέννησεν τὸν βόες ἐκ τῆς ῥαχά (Italic)

Lithuanian

*Lithuanian poses several typographical challenges. Make sure Contextual Alternates (calt) is turned on; for ǐ, use i followed by combining dot accent (*U+0307*) and acute (*U+0301*).*

Visa žemė turėjo vieną kalbą ir tuos pačius žodžius. Kai žmonės kėlėsi iš rytų, jie rado slėnį Šinaro krašte ir ten įsikūrė. Vieni kitiems sakė: Eime, pasidirbkime plytų ir jas išdekime. – Vietoj akmens jie naudojo plytas, o vietoj kalkių – bitumą. Eime, – jie sakė, – pasistatykime miestą ir bokštą su dangų siekiančia viršūne ir pasidarykime sau vardą, kad nebūtume išblaškyti po visą žemės veidą. (Expanded)

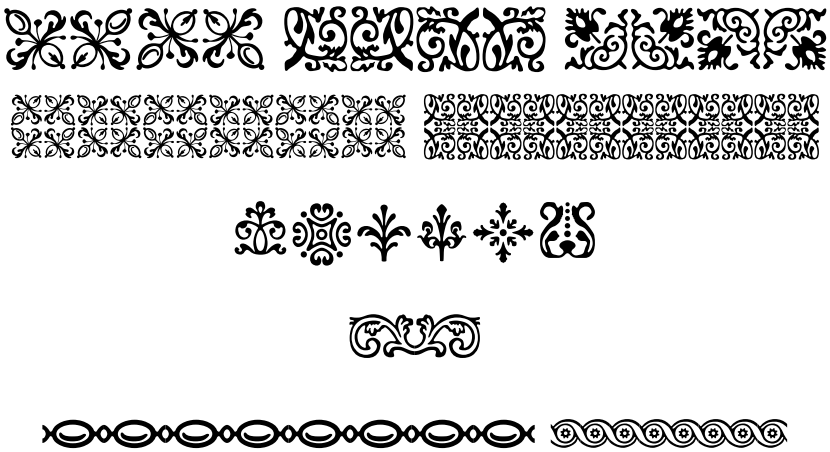
Polish

The default shape and position of ogonek in Junicode are suitable for modern Polish. For the medieval Latin e-caudata, consider using cv62.

Mieszkańcy całej ziemi mieli jedną mowę, czyli jednakowe słowa. A gdy wędrowali ze wschodu, napotkali równinę w kraju Szinear i tam zamieszkali. I mówili jeden do drugiego: Chodźcie, wyrabiajmy cegłę i wypalmy ją w ogniu. A gdy już mieli cegłę zamiast kamieni i smołę zamiast zaprawy murarskiej, rzekli: Chodźcie, zbudujemy sobie miasto i wieżę, której wierzchołek będzie sięgał nieba, i w ten sposób uczynimy sobie znak, abyśmy się nie rozproszyli po całej ziemi. (Condensed Medium)

Fleurons

Junicode contains a number of fleurons (floral ornaments) copied from a 1785 Caslon specimen book. Access these via the OpenType feature ornm. Fleurons have only one weight and width, and they are the same in roman and italic.



3. Getting Started with Junicode

Junicode comes in two flavors—static and variable. Static fonts are the ones users are most familiar with, each font file supplying a single set of outlines that do not change except in size. By contrast, a single variable font file stores a set of outlines that can morph in various ways—for example, becoming bolder or lighter, narrower or wider, and sometimes undergoing more complex transformations. The static version of Junicode consists of thirty-eight font files, each providing a distinct variation of the font's style; the variable version consists of only two (one each for roman and italic), but those two font files are capable of much more than the static version's thirty-eight.

Because the static and variable versions of Junicode are differently named ("Junicode" and "Junicode VF"), both can be installed on the same system. However, you should choose one or the other for any particular project. Choose the static version if the application you are using does not yet support variable fonts. Such applications include Microsoft Word, Apple Pages, Quark Xpress, Google Docs, Affinity Publisher, and most flavors of TEX (except for LuaTEX—see below). Another reason to choose the static version is its familiarity: if you don't need the advanced capabilities of the variable version, it is perfectly all right to stick with what you know.

All major web browsers (including browsers for mobile devices) support variable fonts, and there are good reasons to choose the variable version of Junicode for any web project. The greatest reason to go with the variable version is to speed the loading of web pages: users will never have to download more than two font files (the size of which can be radically reduced via subsetting, explained in Section 8 of this Manual). Additionally, however, variable fonts can make a page of text more dynamic and visually interesting. See Mozilla's Variable

Fonts Guide for more information about using variable fonts on the web.

A growing number of desktop applications support variable fonts. Use the variable version of Junicode in Adobe InDesign (always with the World-Ready Paragraph Composer).[1] LibreOffice has supported variable fonts since version 7.5 (2023). LuaTEX has excellent support for variable fonts: make sure your TEX installation is up to date (since in recent years support for variable fonts has improved with every release), and always choose "harf" mode in your font-selection code. For an example of font-selection code for a variable font, see the file JunicodeManual.sty (part of the source for this manual) in the "docs" directory of the GitHub Junicode site. You are welcome to copy and modify this code. A number of graphical design apps also support variable fonts, including Adobe Illustrator, PhotoShop, Figma, Sketch, and CorelDRAW.

The static version of Junicode has five weights and five widths, which are combined in many ways for a total of nineteen styles in both roman and italic.[2] It is not necessary to install all of these; in fact, your life will be simplified (font menus easier to navigate) if you make a selection. You will probably want the traditional Regular, Bold, Italic, and Bold Italic fonts, but you should survey the styles displayed in the Specimen section of this booklet, choose the ones that look best to you, and install only those. A reasonable selection for many users will include the traditional four styles for main text, several SemiExpanded styles for footnotes, and SemiCondensed for titles.

Junicode's static fonts come in two types, TrueType (files with a .ttf extension) and CFF (files with an .otf extension). These are functionally identical, but they may look subtly different on your computer's screen because of the different technologies used to render glyphs. Choose the one you find most appealing.

With around 5,000 characters, Junicode is a large font. Finding the things you

[1] The choice of a Composer is well hidden in the "Justification" section of the "Paragraph Style Options" dialog. Use of the default "Adobe Paragraph Composer" with Junicode VF may cause InDesign to crash or otherwise misbehave. To prevent crashes when using the variable version of Junicode, it is also advisable to delete InDesign's preferences when launching the program after a font upgrade. To do this, press Shift-Alt-Control (on Windows) or Shift-Control-Option-Command (on the Mac) all together, *immediately* after clicking to launch InDesign.

[2] Several of the twenty-five possible combinations (e.g. light expanded) have been omitted as unlikely to be useful; however, these can be accessed via the variable font.

want in a collection that size can be a challenge, and then entering them in your documents is another challenge. This document will help, but it presupposes a certain amount of knowledge—for example, how to install a font in Windows, Mac OS or Linux and how to install and use different kinds of software.

Medievalists will find the *MUFI Character Recommendation*, version 4.0 (2015) an essential supplement to this document. The *Recommendation* lists thousands of characters identified by the Medieval Unicode Font Initiative as being of interest to medievalists. Junicode contains all of these characters. There are two versions of the *Recommendation*: you will probably find the "Alphabetical Order" version most helpful.

From the *MUFI Character Recommendation* and Chapter 10 ("Encoded Glyphs in Junicode") of this manual, you can find out the **code points**[3] of the characters you need. These code points can be used to enter characters in your documents when they cannot be typed on the keyboard.

To enter any Unicode character in a Windows application, type its four-digit code, followed by Alt-X. To do the same in the Mac OS, first install and switch to the "Unicode Hex Input" keyboard, then type the code while holding down the Option key. In most Linux distributions you can enter a code by typing Shift-Control-U, then the code followed by Return or Enter.

Combining marks (diacritics and certain abbreviation signs) can pose special problems for medievalists. Unicode contains a great many **precomposed characters** consisting of a base letter plus one or more marks. If these are all you need you're fortunate—especially if they can be typed on an international keyboard (not all can).[4] But medieval manuscripts frequently contain combinations of base + mark that are not used in modern written languages. For these, you'll have to enter bases and marks separately.

To position a mark correctly over a base character, first enter the base, followed by the mark or marks. The sequence **m** + ◌̊ (U+1DD9) will make **m̊**; **y** + ◌̄

[3] A Unicode code point is a numerical identifier for a character. It is generally expressed as a four-digit hexadecimal (or base-16) number with a prefix of "U+". The letter capital "A," for example, is U+0041 (65 in decimal notation), and lowercase "ȝ" (Middle English yogh) is U+021D.

[4] Both Windows and the Mac OS come with international keyboards that make it easy to type special letters and diacritics. To find out how to enable these, search online for "Mac OS International Keyboard" or "Windows International Keyboard."

(U+0304) + ◌̆ (U+0306) will make y̆̄; **e** + ◌̣ (U+0323) + ◌̈ (U+1DE0) will make ë̤.

More than sixteen hundred characters in Junicode can only be accessed via OpenType features—that is, by way of the programming built into the font—and many others *should* be accessed that way for reasons explained in the Introduction to the Feature Reference section of this document.

For example, some programs (including Microsoft Word) produce small caps by scaling capitals down to approximately the height of lowercase letters. These always look too thin and light. But Junicode contains hundreds of TRUE SMALL CAPS designed to harmonize with the surrounding text. These can only be accessed via the OpenType `smcp` feature, which you can apply to a run of text much as you apply italic or bold styles: select some text and then apply the feature.

Unfortunately, not every program supports OpenType features, and some that do either support only a few or make them difficult to access. Programs that support Junicode's features fully include the free word processor LibreOffice Writer, all major browsers (Firefox, Chrome, Safari and Edge), and the typesetting programs LuaLaTeX and XeLaTeX. Adobe InDesign supports OpenType features only partially, but all features can be accessed via Roland Dreger's opentype-features script, and InDesign provides access to all of Junicode's characters via its "Glyphs" dialog.

Microsoft Word, unfortunately, provides only limited support for OpenType features. It supports the Required Features discussed below, and also variant number forms and Stylistic Sets (though only one at a time). Many characters (for example, TRUE SMALL CAPS and those accessible only via Character Variant features) cannot be accessed at all. To activate Word's OpenType support, you must open the "Font" dialog, click over to the "Advanced" tab, and check the "Kerning" box. (Oddly, the "Kerning" box enables all other OpenType features.) Then, in the same tab, select Standard Ligatures, Contextual Alternates, and any other features you want. OpenType features are best applied to character styles rather than directly to text: this will save you from having to perform this operation repeatedly.

It is also good to set the language properly for the text you're working on. Programs like Word will automatically set the language to the default for your system. If you change to a language other than your own for a passage (or even a

single word), you should set the language for that passage appropriately. This will unlock a number of capabilities. For example, in Old and Middle English, Word and other programs will use the English form of thorn and eth (þð) instead of the modern Icelandic (þð), and in ancient Greek you will be able to type accents after vowels instead of looking up the codes for hundreds of polytonic vowel + accent combinations. But these and other capabilities are only available when you set the language correctly.

In LibreOffice and InDesign you can set the language with a drop-down menu in the "Character" dialog. In Word there is a separate "Language" dialog, accessible from the "Tools" menu.

If you have questions about any aspect of Junicode, post a query in the Junicode discussion forum. If you notice a bug, or wish to request an enhancement or other change, please open an issue at the font's development site.

4. Feature Reference

4.1. Introduction

The OpenType features of Junicode version 2 and its variable counterpart (hereafter referred to together as "Junicode") have two purposes. One is to provide convenient access to the rich character set of the Medieval Unicode Font Initiative (MUFI) recommendation. The other is to enable best practices in the presentation of medieval text, promoting accessibility in electronic texts from PDFs to e-books to web pages.

Each character in the MUFI recommendation has a code point associated with it: either the one assigned by Unicode or, where the character is not recognized by Unicode, in the Private Use Area (PUA) of the Basic Multilingual Plane, a block of codes, running from U+E000 to U+F8FF, that are assigned no value by Unicode but instead are available for font designers to use in any way they please.

The problem with PUA code points is precisely their lack of any value. Consider, as a point of comparison, the letter **a** (U+0061). Your computer, your phone, and probably a good many other devices around the house store a good bit of information about this **a**: that it's a letter in the Latin script, that it's lowercase, and that the uppercase equivalent is **A** (U+0041). All this information is available to word processors, browsers, and other applications running on your computer.

Now suppose you're preparing an electronic text containing what MUFI calls LATIN SMALL LETTER NECKLESS A (**ꭥ**). It is assigned to code point U+F215, which belongs to the PUA. Beyond that, your computer knows nothing about it: not that it is a variant of **a**, or that it is lowercase, or a letter in the Latin

alphabet, or even a character in a language system. A screen reader cannot read, or even spell out, a word with U+F215 in it; a search engine will not recognize the word as containing the letter **a**.

Junicode offers the full range of MUFI characters—you can enter the PUA code points—but also a solution to the problems posed by those code points. Think of an electronic text (a web page, perhaps, or a PDF) as having two layers: an underlying text, stable and unchanging, and the displayed text, generated by software at the instant it is needed and discarded when it is no longer on the screen. For greatest accessibility the underlying text should contain the plain letter **a** (U+0061) along with markup indicating how it should be displayed. To generate the displayed text, a program called a "layout engine" will (simplifying a bit here) read the markup and apply the OpenType feature `cv02[5]`[1] to the underlying **a**, bypassing the PUA code point, with the result that readers see **a**—the "neckless a." And yet the letter will still register as **a** with search engines, screen readers, and so on.

This is the Junicode model for text display, but it is not peculiar to Junicode: it is widely considered to be the best practice for displaying text using current font technology.

The full range of OpenType features listed in this document is supported by all major web browsers, LibreOffice, XeTeX, LuaTeX, and (presumably) other document processing applications. All characters listed here are available in Adobe InDesign, though that program supports only a selection of OpenType features. Microsoft Word, unfortunately, supports only Stylistic Sets, ligatures (all but the standard ones in peculiar and probably useless combinations), number variants, and the Required Features. In terms of OpenType support, Word is the most primitive of the major text processing applications.

Many MUFI characters cannot be produced by using the OpenType variants of Junicode. These characters fall into three categories:

[1] Many OpenType features produce different outcomes depending on an index passed to an application's layout engine along with the feature tag. Different applications have different ways of entering this index: consult your application's documentation. Here, the index is recorded in brackets after the feature tag. Users of fontspec (with XeLaTeX or LuaTeX) should also be aware that fontspec indexes start at zero while OpenType indexes start at one. Therefore all index numbers listed in this document must be reduced by one for use with fontspec.

- Those with Unicode (non-PUA) code points. MUFI has done valuable work obtaining Unicode code points for medieval characters. All such characters (those with hexadecimal codes that *do not* begin with **E** or **F**) are presumed safe to use in accessible and searchable text. However, some of these are covered by Junicode OpenType features for particular reasons.

- Precomposed characters—those consisting of base character + one or more diacritics. For greatest accessibility, these should be entered not as PUA code points, but rather as sequences consisting of base character + diacritics. For example, instead of MUFI U+E498 LATIN SMALL LETTER E WITH DOT BELOW AND ACUTE, use **e** + U+0323 COMBINING DOT BELOW + U+0301 COMBINING ACUTE ACCENT: **ẹ́** (when applying combining marks, start with any marks below the character and work downwards, then continue with any marks above the character and work upwards. For example, to make **ǭ̣́**, place characters in this order: **o**, COMBINING OGONEK U+0328, COMBINING DOT BELOW U+0323, COMBINING MACRON U+0304, COMBINING ACUTE U+0301). Some MUFI characters have marks in unconventional positions, e.g. **ȯ́** LATIN SMALL LETTER O WITH DOT ABOVE AND ACUTE, where the acute appears beside the dot instead of above. This and other characters like it should still be entered as a sequence of base character + marks (here **o**, COMBINING DOT ABOVE U+0307, COMBINING ACUTE U+0301). Junicode will position the marks in the manner prescribed by MUFI.

- Characters for which a base character (a Unicode character to which it can be linked) cannot be identified, or for which there may be an inconsistency in the MUFI recommendation. These include:

  - **ſƚ** U+E8AF. This is a ligature of long **s** and **l** with stroke, but there are no base characters with this style of stroke.
  - **ꝡ́Ꝡ́** U+EFD8 and U+EFD9. MUFI lists these as ligatures (corresponding to the historic ligatures **ꝡꝠ**, but they cannot be treated as ligatures in the font because a single diacritic is positioned over the glyphs as if they were digraphs like **ꜳꜲ**.
  - **ꝑ̈Ꝑ̈** U+EBE7 and U+EBE6, for the same reason.

- ꝺ U+F159 LATIN ABBREVIATION SIGN SMALL DE. Neither a variant of d nor an eth (ð), this character may be a candidate for Unicode encoding.

- Characters for which OpenType programming is not yet available. These will be added as they are located and studied.

These characters should be avoided, even if you are otherwise using MUFI's PUA characters:

- U+F1C5 COMBINING CURL HIGH POSITION. Use U+1DCE COMBINING OGONEK ABOVE. The positioning problem mentioned in the MUFI recommendation is solved in Junicode (and, to be fair, many other fonts with OpenType programming).

- U+F1CA COMBINING DOT ABOVE HIGH POSITION. Use U+0307 COMBINING DOT ABOVE. It will be positioned correctly on any character.

4.2. Case-Related Features

4.2.1. smcp – Small Capitals

Converts lowercase letters to small caps; also several symbols and combining marks. All lower- and uppercase pairs (with exceptions noted below) have a small cap equivalent. Lowercase letters without matching caps may lack matching small caps. fghij → FGHIJ.

Note: Precomposed characters defined by MUFI in the Private Use Area have no small cap equivalents. Instead, compose characters using combining diacritics, as outlined in the introduction. For example, smcp applied to the sequence t + COMBINING OGONEK (U+0328) + COMBINING ACUTE (U+0301) will change t̨ to ṭ̨.

4.2.2. c2sc – Small Capitals from Capitals

Use with smcp for all-small-cap text. ABCDE → ABCDE.

Note: The variants of Ŋ (U+014A—see Other Latin Letters) have no lowercase equivalents. Their small capital forms can be accessed only through this feature.

4.2.3. pcap – Petite Capitals

Produces small caps in a smaller size than smcp. Use these when small caps have to be mixed with lowercase letters. The whole of the basic Latin alphabet is covered, plus a number of other letters, but fewer than half of Junicode's small caps have petite cap equivalents. klmnoþ → KLMNOÞ.

4.2.4. c2pc – Petite Capitals from Capitals

Produces petite capitals from capitals. Use with pcap to convert mixed-case texts to petite capitals. PQRST → PQRST.

Note: The variants of Ŋ (U+014A—see Other Latin Letters) have no lowercase equivalents. Their petite capital forms can be accessed only through this feature.

4.2.5. case – Case-Sensitive Forms

Produces combining marks that harmonize with capital letters: Ř, X̊, etc. Use of this feature reduces the likelihood that a combining mark will collide with a glyph in the line above. Some applications turn this feature on automatically for runs of capitals, and precomposed characters (e.g. É U+00C9, Ū U+016A) already use case-appropriate combining marks. This feature also changes oldstyle to lining figures, since these harmonize better with uppercase letters. (Some fonts use this feature to raise hyphens and dashes between capitals and lining numbers, but Junicode uses contextual kerning instead: a–b A–B.)

4.3. Alphabetic Variants

4.3.1. cv01-cv52 – Basic Latin and Greek Variants

These features also affect small cap (smcp) and underdotted (ss07) forms, where available. Variants in magenta are also available via ss06 "Enlarged Minuscules." Use the cvNN features instead of ss06 when you want to substitute an enlarged minuscule for a capital (or, less likely, a lowercase) letter everywhere in a text. See the Enlarge Axis for a more flexible method of producing enlarged minuscules.

Variant of	cvNN	Variants
A	cv01	1=ɑ, 2=Ⱥ, 3=Ⱥ, 4=a
a	cv02	1=ɑ, 2=ⱥ, 3=ɑ, 4=a, 5=ⱥ, 6=a, 7=ɒ, 8=ⱥ, 9=ɑ, 10=ⱥ, 11=ɑ
B	cv03	1=b, 2=ɓ
b β	cv04	Latin: 1=b
C	cv05	1=Ɛ, 2=c
c	cv06	1=ɕ, 2=ɕ, 3=ɕ
D	cv07	1=Ɔ, 2=d, 3=ɔ
d	cv08	1=ɔ, 2=ɔ, 3=ɗ, 4=d, 5=ɔ, 6=ɔ, 7=ɔ (also affects đ)
E	cv09	1=Ɇ, 2=Ɇ, 3=e, 4=Ɇ
e	cv10	1=ɛ, 2=e, 3=e, 4=e, 5=ɾ, 6=e, 7=e, 8=e
F	cv11	1=Ƒ, 2=f, 3=ꟻ
f	cv12	1=ꟻ, 2=ꟻ, 3=ꟻ, 4=ꟻ, 5=f, 6=fɾ, 7=f, 8=ꟻ, 9=ꟻ, 10=f
G	cv13	1=Ɠ, 2=Ɋ, 3=Ɪ, 4=g
g	cv14	1=ɠ, 2=ɢ, 3=g, 4=g, 5=g, 6=g, 7=gɾ, 8=g, 9=ɢ, 10=g, 11=ɠ, 12=ɠ, 13=ʒ, 14=g, 15=ɡ
H	cv15	1=ɧ, 2=h, 3=Ⱨ
h	cv16	1=ɦ, 2=ʜ, 3=h, 4=ɓ, 5=ɦ, 6=hɾ
I	cv17	1=İ, 2=I, 3=i, 4=l
i	cv18	1=ı, 2=ı, 3=ɨ, 4=ij, 5=j, 6=i*
J	cv19	1=J̇, 2=j

Variant of	cvNN	Variants
j	cv20	1=ȷ, 2=ɉ, 3=j, 4=j
K	cv21	1=k
k	cv22	1=k, 2=k, 3=ꝁ, 4=ꝃ, 5=k, 6=ƙ, 7=k
L	cv23	1=l
l	cv24	1=ɭ, 2=l, 3=ł, 4=l, 5=l, 6=ℓ
M	cv25	1=ꟽ, 2=ꟽ, 3=M, 4=m
m	cv26	1=m, 2=ɱ, 3=ɱ, 4=m
N	cv27	1=N, 2=n, 3=Ꞑ
n	cv28	1=ŋ, 2=ɴ, 3=ꞥ, 4=n, 5=n, 6=ŋ, 7=ɴ, 8=ꞑ, 9=ŋ
O	cv29	1=o, 2=O
o	cv30	1=o, 2=o
P	cv31	1=ꝑ, 2=p
p	cv32	1=p, 2=p**
Q	cv33	1=ꝗ, 2=q, 3=Q◌, 4=Q◌◌
q	cv34	1=ꝙ, 2=q
R	cv35	1=ꞃ, 2=ꝛ, 3=r
r	cv36	1=ꞃ, 2=ꝛ, 3=r, 4=r, 5=ꝵ, 6=ɲ, 7=ꞃ
S	cv37	1=ʃ, 2=ȣ, 3=s, 4=ȣ, 5=ß, 6=S, 7=Σ, 8=ꝭ
s	cv38	1=ꞅ, 2=s, 3=ſ, 4=ʃ, 5=ẜ, 6=ẝ, 7=s, 8=ß, 9=ꞅ, 10=S, 11=ſ, 12=ʃ, 13=st, 14=ś, 15=ſ
T	cv39	1=Ꞇ, 2=t
t	cv40	1=ꞇ, 2=tt, 3=t, 4=ʈ, 5=ƭ
U	cv41	1=u, 2=U, 3=Ų

Variant of	cvNN	Variants
u	cv42	1=u
V	cv43	1=v
v	cv44	1=v, 2=v, 3=v, 4=v, 5=v, 6=v
W	cv45	1=w, 2=w
w	cv46	1=w, 2=w
X	cv47	1=x
x	cv48	1=x, 2=x, 3=x, 4=x, 5=x
Y	cv49	1=y, 2=y
y	cv50	1=y, 2=y, 3=y, 4=y, 5=y, 6=y
Z	cv51	1=z, 2=z
z	cv52	1=z, 2=z, 3=z

* `cv18[4]` changes ii to ij at the end of a word; `cv18[5]` changes i to j at the end of a word whether another i precedes or not. These variants are chiefly useful for roman numbers, but also for Latin words ending in -ii. The j produced by this feature is searchable as i.

** `cv32[2]` should be on in any edition or extensive quotation of the *Ormulum*. The feature produces a p that differs from the default only in the way it forms a double-p ligature with `hlig`: ꝑ, not ꝓ.

When the active script is Greek (set automatically by many apps), a few of these features behave differently, substituting the "Symbol" forms of several letters (which are also accessible via their Unicode encodings):

Variant of	cvNN	Variants
β	cv04	1=ϐ (U+03D0)
ε	cv10	1=϶ (U+03F5)
φ	cv12	1=ϕ (U+03D5)

Variant of	cvNN	Variants
κ	cv22	1=ϰ (U+03F0)
π	cv32	1=ϖ (U+03D6)
ρ	cv36	1=ϱ (U+03F1)
θ	cv40	1=ϑ (U+03D1)

4.3.2. cv53-cv66, cv91 – Other Latin Letters

Some features affect both upper- and lowercase forms. cv62 also affects combining e with ogonek, accessible via either cv84 or ss10 with the entity reference &_eogo;.

Variant of	cvNN	Variants
Ą (U+0104)	cv53	1=Ⱥ, 2=Ⱥ́, 3=Ⱥ̨
ą (U+0105)	cv54	1=ⱥ, 2=ⱥ́
ꜩ (U+A733)	cv55	1=ꜭ, 2=ꜩ, 3=ꜩ, 4=æ
Æ (U+00C6)	cv56	1=ᴁ, 2=ᴂ
æ (U+00E6)	cv57	1=ɐ, 2=ᴂ, 3=ɜ, 4=ɜ, 5=ɶ, 6=ɶ, 7ɶ, 8=ɐ 9=ɐ, 10=ɜ, 11=ɐ
Ꝏ (U+A734)	cv58	1=ᴐ, 2=ꝏ, 3=ꝏ
ꝏ (U+A735)	cv59	1=ᴐ, 2=ꝏ, 3=ꝏ 4=ꝏ
ꝟ (U+A739)	cv60	1=ꝟ
đ (U+0111)	cv61	1=ɗ
Ę, ę ... (U+0118, U+0119)	cv62	1=Ɇ, ɇ ...; 2=Ɇ, ɇ ...
Ȝ, ȝ (U+021C, U+021D)	cv63	1=ȝȝ, 2=ȝȝ
◌ͥ (U+0365)	cv64	1=◌ͥ
Ŋ (U+014A)	cv91	1=ŋ, 2=ŋ

Variant of	cvNN	Variants
ꝋ (U+A7C1)	cv65	1=ɔ, 2=ø, 3=o, 4=ᴑ
ƿ, Ƿ (U+A765)	cv66	1=ƿ, Ƿ

4.3.3. swsh – Swash letters (italic only)

Produces swash versions of several capitals: *𝒜 𝒞 𝒟 ℱ*. There are three swash versions of Q, selected via index (if indexing of swsh is supported by your application): 1=*𝒬ui*, 2=*𝒬ui*, 3=*𝒬ui*. Note that swash versions of Q can also be selected with cv33. Swash Q is not permitted in word-final position; there plain italic Q is substituted. Also, if Q with extended tail might collide with a following letter, another appropriate form of Q is substituted. There are also swash versions of lowercase e and k: *𝒆 𝓀*. These are only permitted in word-final position: *elevate𝒆, skylark*.

4.3.4. ss01 – Alternate thorn and eth

Produces Nordic thorn and eth (**ÞðÞ**) when the language is English, and English thorn and eth (**þðþ**) with any other language, reversing the font's ordinary usage. This also affects small caps, crossed thorn (**ꝥ Ꝥ**—see also cv66), combining mark eth (U+1DD9, ◌ᷙ ◌ᷙ), and enlarged thorn and eth (see ss06). This feature depends on locl (Localized Forms), which in most applications will always be enabled.

4.3.5. ss02 – Insular Letter-Forms

Produces insular letter-forms, e.g. **ꝺ�something**. The result is different, depending on whether the language is English or Irish (make sure the language for your document is set properly). In English text, capitals are not affected (except W), as these do not commonly have insular shapes in early manuscripts; instead, enter the Unicode code points or use the Character Variant (cvNN) features. In English text, ss02 imitates the typography of the Old English passages of Hickes's *Thesaurus*, not the usage of Old English or Anglo-Latin manuscripts. In Irish texts, it imitates the distribution of insular characters but cannot imitate the style of particular scribal hands or typefaces.

4.3.6. ss04 – High Overline

Produces a high overline over letters used as roman numbers: cdijlmvx CDI JLMVXↃ.

4.3.7. ss05 – Medium-High Overline

Produces a medium-high overline over (or through the ascenders of) letters used as roman numbers, and some others as well: bcdhıȷklmſvxþ.

4.3.8. ss06 – Enlarged Minuscules

Letters that are lowercase in form but uppercase in function, and between upper- and lowercase in size, often used in medieval manuscripts as *litterae notabiliores* to begin sentences, paragraphs, and other textual units. This feature covers the whole of the basic Latin alphabet and a number of other letters that occur at the beginnings of sentences, plus a few punctuation marks. Uppercase letters are also covered by this feature so that enlarged minuscules can, if you like, be searched as capitals. This is Junicode's collection of enlarged minuscules:

a → a	d́ → d́	ę → ę	i → i	œ → œ	ƿ → ƿ
á → á	ꝺ̇ → ꝺ̇	f → f	ı → ı	p → p	x → x
ꬱ → ꬱ	ꝺ → ꝺ	ꝼ → ꝼ	j → j	q → q	y → y
ꜳ → ꜳ	ꝺ́ → ꝺ́	g → g	ȷ → ȷ	r → r	z → z
æ → æ	ð → ð	ᵹ → ᵹ	k → k	s → s	þ → þ
ꜵ → ꜵ	ð → ð	ꝿ → ꝿ	l → l	ſ → ſ	þ → þ
b → b	e → e	h → h	m → m	t → t	⹎ → ⹎
c → c	e → e	ꜧ → ꜧ	ꝳ → ꝳ	u → u	ˇ → ˇ
d → d	é → é	h → h	n → n	v → v	⸮ → ⸮
ḋ → ḋ	ę → ę	ħ → ħ	o → o	w → w	ˇ → ˇ

If you are using the variable version of the font (Junicode VF), consider using the Enlarge axis instead, for reasons of flexibility and accessibility.

4.3.9. `ss07` – Underdotted Text

Produces underdotted text (indicating deletion in medieval manuscripts) for most Latin and Greek letters, e.g. ạḅc̣ḍẹf̣g̣ ḤỊJ̣ḲḶṂ α̣β̣γ̣δ̣ε̣ζ̣η̣ Α̣Β̣Γ̣Δ̣Ε̣Ζ̣Η̣. This also affects small caps, e.g. ʜ̣ɪ̣ᴊ̣ᴋ̣ʟ̣ᴍ̣ɴ̣ θ̣ɪ̣ᴋ̣ʌ̣ᴍ̣ɴ̣ξ̣. If this feature fails for any letter, use U+0323, combining dot below.

4.3.10. `ss08` – Contextual Long s

In English, French, and Latin text, varies **s** and **ſ** according to rules followed by many early printers: **ſports, eſſence, ſtormy, diſheveled, transfuſions, flyneſs, cliffside**. For other languages, the English rules are applied, though these may not always be appropriate, and for some languages additional editing may be required. For this feature to work properly, `calt` "Contextual Alternates" must also be enabled (as it should be by default: see Required Features below). This feature does not work in LuaTEX, except in harf mode.

4.3.11. `ss16` – Contextual r Rotunda

Converts **r** to **ꝛ** (lowercase only) following the most common rules of medieval manuscripts: **pꝛiest, firmer, frost, oꝛnament**. For this feature to work properly, `calt` "Contextual Alternates" must also be enabled (as it should be by default: see Required Features below). This feature does not work in LuaTEX, except in harf mode.

4.3.12. `cv68` – Variant of ʔ (U+0294, glottal stop)

1=ˀ.

4.3.13. `salt` – Miscellaneous Alternates (medieval capitals, etc.)

Junicode has two series of decorative capitals in medieval scripts. These affect only the letters A-Z and a-z. `salt[1]` provides rustic capitals, a script used for text in the late ancient and early medieval periods and for display until around the eleventh century: **GAZIÉRIQUENS LIBYCOS DUXIT KARTHAGO TRIUMPHOS**. `salt[2]` provides Lombardic capitals, a style used mainly for what are now called

drop caps and not for running text, titles, or headers: 𝔄 𝔅 ℭ 𝔇 𝔈 𝔉. Rustic capital Æ is available (𝔄), but not Lombardic. `salt[3]` provides variants of rustic G and Lombardic F and T: 𝔊 𝔉 𝔗 Miscellaneous alternates (for which Character Variants are unavailable) are also gathered here:

ſ: 1=ſ, 2=ſſ, 3=ℭ.
ð: 1=𝔡, 2=ð, 3=ð.
ẏ: 1=ẏ.
ꝺ: 1=ꝺ.
- (hyphen): 1=‐.
ł: 1=ł.
ť: 1=ť.
ꝑ: 1=ꝑ.
ꝓ: 1=ꝓ, 2= 1=ꝓ.
þ: 1=þ, 2= 1=þ.

4.4. Greek

Junicode has two distinct styles of Greek. In the roman face, it is upright and modern, especially designed to harmonize with Junicode's Latin letters. In the italic, it is slanted and old-style, being based on the eighteenth-century Greek type designed by Alexander Wilson and used by the Foulis Press in Glasgow. Both Greek styles include the full polytonic and monotonic character sets: αβγδεζ *αβγδεζ*.

To set Greek properly (especially polytonic text) requires that both `locl` and `ccmp` be active, as they should be by default in most text processing applications (but in MS Word they must be explicitly enabled by checking the "kerning" box on the "Advanced" tab of the Font Dialog).

Modern monotonic Greek should be set using only characters from the Unicode "Greek and Coptic" range (U+0370–U+03FF). When monotonic text is set in all caps, Junicode suppresses accents automatically (except in single-letter words, for which you must substitute unaccented forms manually). This substitution is not performed on text containing visually identical letters from the "Greek Extended" range (U+0F00–U+1FFF). Thus when setting polytonic Greek, one should use (for example) Ἀ (U+1FBB), not Ά (U+0386), though they look the same.

You can set polytonic Greek either by entering code points from the Greek Extended range or by entering sequences of base characters and diacritics. When using the latter method, you must first make sure the language for the text in

question (whether a single word, a short passage, or a complete document) is set to Greek, and then enter characters in canonical order (that is, the sequence defined by Unicode as equivalent to the composite character). The order is as follows: 1. base character; 2. diacritics positioned either above or in front of the base character, working from left to right or bottom to top; 3. the *ypogegrammeni* (U+0345), or for capitals, if you prefer, the *prosgegrammeni* (U+1FBE).[2]

For example, the sequence ω (U+03C9) ◌̓ (U+0313) ◌́ (U+0301) ◌ͅ (U+0345) produces ᾧ. Substitute capital Ω (U+03A9) and the result is ᾼ. Note that in a number of applications the layout engine will perform these substitutions before Junicode's own programming is invoked. If either the layout engine or Junicode fails to produce your preferred result, try placing U+034F COMBINING GRAPHEME JOINER somewhere in the sequence of combining marks—for example, before the *ypogegrammeni* to make ᾯ.

For alternate Greek letter-shapes, see Basic Latin and Greek Variants above and the next section.

4.4.1. ss03 – Alternate Greek

Provides alternate shapes of β γ θ π φ χ ω: **β ɣ θ π φ χ ω**. These are chiefly useful in linguistics, as they harmonize with IPA characters. This feature will work whether the Greek or Latin script is active.

4.5. Gothic

4.5.1. ss19 – Latin to Gothic Transliteration

Produces Gothic letters from Latin: Warþ þan in dagans jainans → 𐍅𐌰𐍂𐍆 𐍈𐌰𐌽 𐌹𐌽 𐌳𐌰𐌲𐌰𐌽𐍃 𐌾𐌰𐌹𐌽𐌰𐌽𐍃. In web pages and PDFs, the letters will be searchable as their Latin equivalents.

[2] Some applications will automatically reorder sequences of letters and accents, sparing you the trouble of remembering the canonical order.

4.6. Runic

4.6.1. ss12 – Early English Futhorc

Changes Latin letters to their equivalents in the early English futhorc. Because of the variability of the runic alphabet, this method of transliteration may not produce the result you want. In that case, it may be necessary to manually edit the result. fisc flodu ahof → ᚠᛁᛋᚳ ᚠᛚᚩᛞᚢ ᚪᚻᚩᚠ.

4.6.2. ss13 – Elder Futhark

Changes Latin letters to their equivalents in the Elder Futhark. Because of the variability of the runic alphabet, this method of transliteration may not produce the result you want. In that case, it may be necessary to manually edit the result. ABCDEFG → ᚨᛒᚲᛞᛖᚠᚷ.

4.6.3. ss14 – Younger Futhark

Changes Latin letters to their equivalents in the Younger Futhark. Because of the variability of the runic alphabet, this method of transliteration may not produce the result you want. In that case, it may be necessary to manually edit the result. ABCDEFG → ᚼᛒᚴᛏᛁᚠᚴ.

4.6.4. ss15 – Long Branch to Short Twig

In combination with ss14, converts long branch (the default for the Younger Futhark) to short twig runes: ᚼᛒᚴᛏᛁᚠᚴ → ᛁᚠᚴᛏᛁᚠᚴ.

4.6.5. rtlm – Right to Left Mirrored Forms

Produces mirrored runes, e.g. ᚠᛒᚳᛞᛖᚠᚷ → ᚷᚤᛖᛞᚿᛒᚴ. This feature cannot change the direction of text or reverse its order.

4.7. Ligatures and Digraphs

Old-style fonts typically contain a standard collection of ligatures (conjoined letters), including fi, fl, ff, ffi, and ffl. Most software will display these ligatures automatically (except Microsoft Word, for which they must be enabled explicitly). Junicode has a large number of ligatures, including the standard f-ligatures, a similar set for long s, e.g. ſl, ſſ, ſſi, but also more unusual forms like ſt, ſt, ſþ (the last two with `ss02` and `cv38[11]`).

Junicode also contains more specialized ligatures: for various enclosed alphanumerics, e.g. ❶❺ → ⓯, ①⑧ → ⑱; for the five tone modifiers (U+02E5, U+02E9, U+02E6, U+02E8, U+02E7), a large number of ligatures, e.g. ˥˥ → ˥, ˥˥˧ → ˥˧; for combinations of vowel + rhotic hook (U+02DE), several more ligatures, e.g. a˞ → a˞, œ˞ → œ˞. These, like the more common ligatures, are automatic.

Many of Junicode's ligatures, however, are not automatic, but belong to the set of either Historic Ligatures or Discretionary Ligatures, both of which must be invoked explicitly. These are listed in the following sections.

4.7.1. `hlig` – Historic Ligatures

Produces ligatures for combinations that should not ordinarily be rendered as ligatures in modern text.[3] Most of these are from the MUFI recommendation, ranges B.1.1(b) and B.1.4. This feature does not produce digraphs (which have a phonetic value), for which see `ss17`. The ligatures:

af → ɾf	aN → ɐɴ	av → ɐv	æn → ꭣn	æx → ꭣx
aɾ → ɾɾ	ap → ɾp	æc → ꭣc	æp → ꭣp	ɐv → ɐv
ag → ɐg	ar → ɐr	æɾ → ꭣɾ	æɲ → ꭣɲ	bb → ƀb
al → ɾl	aR → ɐʁ	æʒ → ꭣʒ	æɾ → ꭣɾ	bg → ƀg
an → ɐn	aþ → ɾþ	æm → ꭣm	æꞇ → ꭣc	bo → ƀo

[3] Some fonts define `hlig` differently, as including all ligatures in which at least one element is an archaic character, e.g. those involving long s (ſ). In Junicode, however, a historic ligature is defined not by the characters it is composed of, but rather by the join between them. If two characters (though modern) should not be joined except in certain historic contexts, they form a historic ligature. If they should be joined in all contexts (even if archaic), the ligature is not historic and should be defined in `liga`.

ch → ch	eɼ → eɼ	hſ → hſ	oꝝ → oꝝ	ſtr → ſtr
ck → ck	eꞇ → eꞇ	kr → kr	oꝝ → oꝝ	ſƿ → ſƿ
ꝺꝺ → ꝺꝺ	ex → ex	kſ → kſ	PP → PP	ſp → ſp
ꝺe → ꝺe	ey → ey	kf → kf	pp → pp	ꞇꞇ → ꞇꞇ
ꝺo → ꝺo	fä → fä	ll → ll	ꝓp → ꝓp	U†D→U†D
ꝺv → ꝺv	fo → fo	ma → ma	ꝑꝑ → ꝑꝑ	UE → UE
de → de	gd → gd	me → me	Ps → Ps	ue → ue
do → do	gð → gð	mi → mi	pe → pe	UU → UU
Do → Do	gꝺ → gꝺ	na → na	ps → ps	uu → uu
ea → ea	gg → gg	ne → ne	Psi → Psi	ƿƿ → ƿƿ
ec → ec	ɡɡ → ɡɡ	ni → ni	psi → psi	þr → þr
eꝼ → eꝼ	go → go	ɴſ → ɴſ**	qƿ → qƿ	þf → þf
eᵹ → eᵹ	gp → gp	nv → nv	qʒ/qꝫ → qʒ/qꝫ	ðð → ðð
em → em	gr → gr	oc → oc	qꝗ → qꝗ	ðv → ðv
en → en	gv → gv	OꝚ → OꝚ	QꝚ → QꝚ	þþ → þþ
eo → eo	Hr → Hr	oꝛ → oꝛ	qꝛ → qꝛ	ƿƿ → ƿƿ
ep → ep	hr → hr	ᴏꝛ → ᴏꝛ	ſä → ſä	ꝥꝥ → ꝥꝥ
eꞃ → eꞃ	hſ → hſ	Oꝝ → Oꝝ	ſch → ſch	

* The ꜳ ligature looks like U+A733 ꜳ, but behaves differently. As in most other ligatures, each element has a set of anchors so that diacritics can be placed over or under the element, e.g. ꜳ́, ꜳ́. ** For the ligature ɴſ to work properly, U+017F ſ must be entered directly, not by applying an OpenType feature to s.

An alternative way to generate any of the ligatures listed above is to place U+200D ZERO WIDTH JOINER between the ligature's elements (most ligatures require one joiner—a few three-part ligatures require two). This method does not require that `hlig` be on, and so can reduce the amount of tagging needed in a text. U+200D is not visible and is normally ignored in searching. Using it is as safe as using `hlig`.

4.7.2. `dlig` – Discretionary Ligatures

Produces lesser-used ligatures: ct, ſp, ſtr, ſt, tr, tt, ty. The collection of discretionary ligatures in the italic face also includes *as, is, us*.

4.7.3. U+200D ZERO WIDTH JOINER

The Unicode character U+200D ZERO WIDTH JOINER (ZWJ) provides an alternate method of entering many ligatures and diagraphs, including all ligatures available via hlig and dlig and the digraphs[4] ꜳ æ ꜵ ꜷ ꜹ ꜽ œ ꝏ ꝡ, with uppercase equivalents. For the ligatures, the advantage of the ZWJ is that a text with many historic ligatures will require less tagging to turn hlig on and off (the feature is disruptive when it is on where it is not needed). The ZWJ causes two letters to be combined as a ligature. Use it where you want a digraph to be searchable as its elements: for example, U+A733 LATIN SMALL LETTER AA cannot ordinarily be searched as a or aa; if entered as a U+200D a, it will be searchable as aa (but not as U+A733). Also, combining marks can be placed over the individual elements of ligatures but not over the elements of digraphs: the sequences a U+034F U+0301 U+200D a and a U+200D a U+034F U+0301[5] yield ꜳ́ and ꜳ́, while an accent on U+A733 is centered on the digraph: ꜳ́.

4.7.4. ss17 – Rare Digraphs

Note: ss17 is deprecated as of version 2.220, and it will be removed in version 2.225. Instead of ss17, use U+200D ZERO WIDTH JOINER (see previous section).

By “digraph” we mean conjoined letters that represent a phonetic value: the most common examples for western languages are æ and œ (though these, because they are so common, are not included in this feature). Use of this feature in web pages enables easier searches: for example, producing þau from þau allows the word to be searched as “þau.” The digraphs covered by this feature are ꜳ, ꜵ, ꜷ, ꜹ, ꜽ, ðv, ð̌ɔ, gp, ꝏ, ꝡ, plus capital and small cap equivalents and digraph + diacritic combinations anticipated in the MUFI recommendation. To produce such a digraph + diacritic combination, either type a letter + diacritic combination as the second element of the digraph or type the diacritic after the second element.

[4] A digraph is a letter composed of two conjoined glyphs and representing a phonetic value. In English, æ and œ are digraphs. A ligature, on the other hand, is composed of two or more conjoined glyphs that should be recognized as separate letters.

[5] U+034F COMBINING GRAPHEME JOINER is needed to prevent the sequence a U+0301 being resolved as á before the font’s OpenType programming can be run, preventing the formation of a ligature.

For example, **a** + **ú** yields **áu**, and so does **a** + **u** + U+0301 (combining acute accent). To produce a digraph + diacritic combination not covered by MUFI (e.g. **ȣ̀**), you may have to place U+034F COMBINING GRAPHEME JOINER between the second element of the digraph and the combining mark.

4.8. Numbers and Sequencing

4.8.1. `frac` – Fractions

Applied to a slash and surrounding numbers, produces fractions with diagonal slashes. 6/9 becomes 6⁄9, 16/91 becomes 16⁄91.

4.8.2. `numr` – Numerators

Changes numbers to those suitable for use on the left/upper side of fractions with diagonal stroke (U+2044). This can be used, with `dnom`, to manually construct fractions, but for most users `frac` will be a better solution.

4.8.3. `dnom` – Denominators

Changes numbers to those suitable for use on the right/lower side of fractions with diagonal stroke (U+2044). This can be used, with `numr`, to manually construct fractions, but for most users `frac` will be a better solution.

4.8.4. `nalt` – Alternate Annotation Forms

Produces letters and numbers circled, in parenthesis, or followed by periods, as follows:

`nalt[1]`, circled letters or numbers: ⓐ ⓑ . . . ⓩ; ⓪ ① ② . . . ⑳.
`nalt[2]`, letter or numbers in parentheses: (a) . . . (z); (0) (1) . . . (20).
`nalt[3]`, double-circled numbers: ⓪ ⓵ . . . ⓾.
`nalt[4]`, white numbers in black circles: ⓿ ❶ ❷ ❸ . . . ⓴.
`nalt[5]`, numbers followed by period: 0. 1. . . . 20.

For enclosed figures 10 and higher, `rlig` (Required Ligatures) must also be enabled (as it should be by default: see Required Features below).

4.8.5. `tnum` – Tabular Figures

Fixed-width figures: 0123456789 (with `lnum`), 0123456789 (default or with `onum`).

4.8.6. `onum` – Oldstyle Figures

Junicode's default figures are oldstyle and proportional, harmonizing with lowercase characters: 0123456789. Use this feature to switch temporarily to oldstyle figures in a context where `lnum` is active.

4.8.7. `pnum` – Proportional Figures

Junicode's default figures are proportionally spaced: unlike tabular figures, they are not all the same width: 0123456789. Use this feature to switch temporarily to proportional figures in a context where `tnum` is active.

4.8.8. `lnum` – Lining Figures

Figures in a uniform height, harmonizing with uppercase letters: 0123456789 (with `tnum`), 0123456789 (default or with `pnum`).

4.8.9. `zero` – Slashed Zero

Produces slashed zero in all number styles, including superscripts, subscripts, and fractions made with `frac`: 0 o 0 o 10⁄30.

4.8.10. `ss09` – Alternate Figures

In the manner of old typefaces, Junicode's default number one is shaped like a small capital I and its zero is a plain ring. This feature provides more modern-looking figures: 01. Only oldstyle figures are affected by this feature.

4.9. Superscripts and Subscripts

4.9.1. sups – Superscripts

Produces superscript numbers and letters. Superscript numbers are in one of two styles: oldstyle proportional (from oldstyle numbers) and lining tabular (from lining numbers). All lowercase letters of the basic Latin alphabet are covered, and most uppercase letters (those encoded in Unicode). The lowercase Greek modifier letters are also covered: $^{0123\ 4567}$ $^{abcde\ ABDEG}$ $^{\beta\gamma\delta\varepsilon}$. Wherever superscripts are needed (e.g. for footnote numbers), use sups instead of the raised and scaled characters generated by some programs. With sups: ^{4567}. Scaled: ^{4567}. Latin superscripts have anchors so that diacritics can be added.

4.9.2. subs – Subscripts

Produces subscripts,including oldstyle proportional and lining tabular figures ($_{2345\ 8901}$) and the Latin and Greek letters included in the Unicode collection of subscripts ($_{a\ e\ h\ i\ j\ k\ l\ m\ n\ o\ p\ r\ s\ t\ u\ v\ x\ \beta\ \gamma\ \rho\ \varphi\ \chi}$)

4.10. Punctuation and Symbols

MUFI encodes nearly twenty marks of punctuation in the PUA. In Junicode these can be accessed in either of two ways: all are indexed variants of . (period), and all are associated with the Unicode marks of punctuation they most resemble (but it should not be inferred that the medieval marks are semantically identical with the Unicode marks, or that there is an etymological relationship between them). The first method will be easier for most to use, but the second is more likely to yield acceptable fallbacks in environments where Junicode is not available.

Marks with Unicode encoding are not included here, as they can safely be entered directly. In MUFI 4.0 several marks have PUA encodings, but have since that release been assigned Unicode code points: *paragraphus* (⸿ U+2E4D), medieval comma (⸲ U+2E4C), *punctus elevatus* (⸱ U+2E4E), *virgula suspensiva* (⸗ U+2E4A), triple dagger (⸸ U+2E4B).

4.10.1. ss18 – Old-Style Punctuation Spacing

Colons, semicolons, parentheses, quotation marks and several other glyphs are spaced as in early printed books.

4.10.2. cv69 – Variants of ⁊⹒ (U+204A / U+2E52, Tironian nota)

1=ꝛ⁊, 2=ꝛ⁊, 3=⁊⹒, 4=⁊⹒, 5=ɐ⁊, 6=⁊⹒, 7=⁊⹒, 8=ꟼ⹒, 9=ꟼ⹒, 10=⁊⹒, 11=⁊⹒. The Tironian *nota* was used throughout the Middle Ages, and it took a wide variety of shapes. The default glyphs and those on indexes 3, 4, and 11 are suitable for early medieval text, while those on indexes 1, 2, and 5 are more appropriate for later medieval text. Those on indexes 6, 7, and 8 are intended as compromises, suitable for text of any period.

4.10.3. cv70 – Variants of . (period)

1=˙, 2=„, 3=’, 4=;, 5=.,, 6=:,, 7=⸵, 8=⁘, 9=:´, 10=ʕ, 11=.´, 12=ʔ, 13=!, 14=ʕ, 15=:, 16=∞, 17=~, 18=∵, 19=/, 20=/, 21=˘, 22=:´, 23=˘, 24=s, 25=cc, 26=–, 27=~, 28=+, 29=;.
. This feature provides access to all non-Unicode MUFI punctuation marks. Some of them are available via other features (see below).

4.10.4. cv71 – Variant of · (U+00B7, middle dot)

1=◌˙ (*distinctio*), 2=◌·.

4.10.5. cv72 – Variants of , (comma)

1=„, 2=’.

4.10.6. cv73 – Variants of ; (semicolon)

1=; (*punctus versus*), 2=.,, 3=:,, 4=⸵, 5=⁘, 6=.,, 7=⁙, 8=:˘, 9=;. Several complex punctuation marks are gathered here. This does not imply that these marks are variants of the semicolon.

4.10.7. cv74 – Variants of ⸎ (U+2E4E, *punctus elevatus*)

1=⸎, 2=⸎, 3=⸎, 4=⸎ (*punctus flexus*), 5=⸎, 6=⸎, 7=⸎. Some of these are affected by ss06, Enlarged Minuscules.

4.10.8. cv75 – Variant of ! (exclamation mark)

1=*!* (*punctus exclamativus*).

4.10.9. cv76 – Variants of ? (question mark)

1=?, 2=?, 3=?. Shapes of the *punctus interrogativus*.

4.10.10. cv77 – Variant of ~ (ASCII tilde)

1=∼ (same as MUFI U+F1F9, “wavy line”).

4.10.11. cv78 – Variant of * (asterisk)

1=⁖, 2=*, 3=*, 4=*, 5=⁜. MUFI defines U+F1EC as a *signe de renvoi*. Manuscripts employ a number of shapes (of which this is one) for this purpose. Junicode defines it as variant 1 of the asterisk—the most common modern *signe de renvoi*.

4.10.12. cv79 – Variants of / (slash)

1=⸝, 2=/. The first of these is Unicode, U+2E4E.

4.11. Spacing Abbreviations

4.11.1. cv80 – Variant of ꝝ (U+A75D, rum abbreviation)

1=ꝝ.

4.11.2. cv82 – Variants of spacing ꝰ (U+A770)

1=ꝯ, 2=ꝰ. cv82[1] produces the baseline *-us* abbreviation (same as MUFI U+F1A6). MUFI also has an uppercase baseline *-us* abbreviation (U+F1A5), but as there is no uppercase version of U+A770 to pair it with, it is indexed separately here.

4.11.3. cv83 – Variants of ꝫ (U+A76B, "et" abbreviation)

1=;, 2=ꝫ, 3=:, 4=⁊. [1] and [3] are identical in shape to a semicolon and a colon, but as they are semantically the same as U+A76B, it is preferable to use those characters with this feature. [2] produces a subscript version of the character, a common variant in early printed books. [4] has a lower extension that crosses the descender of a preceding q.

4.11.4. cv99 – Word omitted symbol (variant of U+00B0, degree)

The degree sign is often used as an editorial sign in editions of the Greek New Testament. This feature scales and positions it to match other editorial signs (U+2E00–u+2E0D).

4.12. Combining Marks

Junicode provides more than 350 combining marks, including nearly 150 Unicode-encoded marks, variants of these, and unencoded equivalents of MUFI diacritics (which, being PUA-encoded, do not function well as combining marks). All of Junicode's unencoded marks are accessible via OpenType features (listed below) or tags. Most are produced by positioning the mark directly after the base character: for example, the combination á is produced by the sequence a U+035B. Several are "double" marks, marking the relationship between two characters. These are produced by placing the double mark *between* the two base characters: U+035C (◌͜◌), U+035F (◌͟◌), U+0362 (◌͢◌), U+035D (◌͝◌), U+035E (◌͞◌), U+0360 (◌⃕◌), U+0361 (◌͡◌), U+1DCD (◌᷍◌). The height of these double marks is automatically adjusted to clear neighboring letters and combining marks, e.g. l͡o, á͡o, æ̂u, o͟͜e. If a double diacritic appears too high or too low (e.g. l̈ö), place U+200C ZERO-WIDTH

NON-JOINER somewhere between the double diacritic and the combining mark: the sequence `l U+035E U+200C o U+030E` produces l͞o̎.

4.12.1. U+034F COMBINING GRAPHEME JOINER

The character at U+034F is defined by Unicode as a combining mark. It is invisible, having no outlines or advance width, and is normally ignored by software; and yet it is extremely useful, and for some applications essential. Its name is somewhat misleading: it is more "non-joiner" than "joiner," being used to prevent normalization of sequences of base glyphs and combining marks. Most (perhaps all) rendering engines, when they encounter a sequence like `a ◌̄ (U+0304)`, normalize it by substituting a precomposed character that combines the two glyphs of the sequence—in this case, U+0101 **ā**. This substitution happens before the rendering engine begins to execute the font's OpenType code. Since no ligature in Junicode has the character **ā** as an element, the combining mark ◌̄ prevents the formation of a ligature.

But if you place U+034F between the base character and the combining mark (`a U+034F U+0304`), the sequence is not normalized. It looks the same as the precomposed character, but plain **a** remains in the text, and so ligatures can form. This can be useful when you want to place a combining mark over one element of a ligature: the sequence `a U+034F U+0301 U+200D n`[6] produces a ligature with combining mark: **án**. Without U+034F the ligature cannot form: **án**.

4.12.2. `cv84` – MUFI combining marks (variants of U+0304)

MUFI encodes a number of combining marks in the PUA (with code points between E000 and F8FF), but when these characters are entered directly, they can interfere with searching and accessibility, and some important applications fail to position them correctly over their base characters. To avoid these problems, enter U+0304 (◌̄, COMBINING MACRON) and apply `cv84`, with the appropriate index, to both mark and base character. This collection of marks does not include any Unicode-encoded marks (from the "Combining Diacritical Marks" ranges), as these can safely be entered directly. It does include three marks (`cv84[30]`, `[31]`

[6] For U+200D and its uses, see the section ZERO WIDTH JOINER.

and [32]) that lack MUFI code points but are used to form MUFI characters, and three more ([2], [33], and [34]) that have no code points in Unicode or MUFI.

This feature may often appear to have no effect. When this happens, try placing U+034F COMBINING GRAPHEME JOINER between the base character and the combining mark—for example, the sequence u U+034F U+0304 produces **ū**.

These marks can sometimes be produced by other cvNN features, which may be preferable to cv84 as providing more suitable fallbacks for applications that do not support Character Variant (cvNN) features.

For some marks with PUA code points, users may find it easier to use entities than this feature.

These marks are not affected by most other features. This is to preserve flexibility, given the rule that the feature that produces them must be applied to both the mark and the base character. For example, if you had to apply smcp "Small Caps" to U+1DE8 ◌ᵇ͛ to get cv84[11] ◌ᴮ͛, it would be impossible to produce the sequence **nb̊a** (or the reverse case NB̊A) with the diacritic properly positioned.

1 = ◌ᵒ͛	8 = ◌ᵃᴺ͛	15 = ◌ᵎ͛	22 = ◌ᵠ͛	29 = ◌ᵅ͛	36 = ◌ᵔ͛
2 = ◌ᵃ͛	9 = ◌ᵃ͛	16 = ◌ʲ͛	23 = ◌ᵑ͛	30 = ◌ᵕ͛	37 = ◌ᵠ͛
3 = ◌ᵣ͛	10 = ◌ᵃᴿ͛	17 = ◌ᴶ͛	24 = ◌ᵡ͛	31 = ◌ᵖ͛	38 = ◌ᵕ͛
4 = ◌ᵀ͛	11 = ◌ᴮ͛	18 = ◌ᴷ͛	25 = ◌ᵀ͛	32 = ◌ᵠ͛	39 = ◌ᵐ͛
5 = ◌ᵑ͛	12 = ◌ᴰ͛	19 = ◌ᵐ͛	26 = ◌ʸ͛	33 = ◌ᵐ͛	
6 = ◌ᵎ͛	13 = ◌ᵉ͛	20 = ◌ᵒ͛	27 = ◌ᵇ͛	34 = ◌ᵐ͛	
7 = ◌ᵃⁿ͛	14 = ◌ᵋ͛	21 = ◌ᵒ͛	28 = ◌ᵅ͛	35 = ◌ᶾ͛	

4.12.3. ss11 – Zero-width to spacing mark

Converts combining marks (including zero-width abbreviations) to their spacing equivalents. At present, the conversion most likely to be of interest to medievalists is of U+035B combining zigzag, ◌͛ to (◌ᶻ).

4.12.4. cv67 – Variant of combining r rotunda (U+1DE3)

1=◌ᶻ͛. The spacing zigzag abbreviation that was available via cv67 is now available via s11. The *enim* abbreviation that was formerly available here can be accessed

via **n** + `salt[4]` or N with `salt[3]`.

4.12.5. `cv81` – Variants of ◌͛ (U+035B, combining zigzag above)

1=◌͛, 2=◌͝, 3=◌͘. Positioning of the zigzag can differ from that of other combining marks, e.g. **b̛ f̛ d̛**. If `calt` "Contextual Alternates" is enabled (as it is by default in most apps), variant forms of `cv81[2]` will be used with several letters, e.g. **d͝ p͝ q͝**. Enable `case` for forms that harmonize with capitals (**A͝ B͝ C͝ D͝**), `smcp` for forms that harmonize with small caps (**ᴇ͝ ꜰ͝ ɢ͝ ʜ͝**).

4.12.6. `ss10` – Character Entities for Combining Marks

Instead of `cv84` for representing non-Unicode combining marks, some users may wish to use XML/HTML-style entities. When `ss10` is turned on (preferably for the entire text), these entities appear as combining marks and are correctly positioned over base characters. For example, the letter **e** followed by **&_eogo;** will yield **e̩**. An advantage of entities is that they are (unlike the PUA code points or the indexes of `cv84`) mnemonic and thus easy to use. A disadvantage is that searches cannot ignore combining marks entered by this method as they can using the `cv84` method. (Every method of entering non-Unicode combining marks has disadvantages: users should weigh these, choose a method, and stick with it.)

If you use any of these entities in a work intended for print publication, you should call your publisher's attention to them, since they will likely have their own method of representing them.

&_dotac;	→	◌̈	&_zzang;	→	◌͛	&_as;	→	◌ᶻ
&_cumac;	→	◌̊	&_zzcur;	→	◌͜	&_bsc;	→	◌ᴮ
&_wamac;	→	◌̍	&_zzvrt;	→	◌͘	&_dsc;	→	◌ᴰ
&_semac;	→	◌̄	&_didi;	→	◌̇	&_eogo;	→	◌̩
&_upmac;	→	◌̍	&_et;	→	◌͗	&_emac;	→	◌ᴱ
&_zzmac;	→	◌̄	&_ansc;	→	◌ᴬᴺ	&_idotl;	→	◌ᴵ
&_namac;	→	◌̄	&_an;	→	◌ᵃⁿ	&_j;	→	◌ʲ
&_ogdot;	→	◌̣	&_ar;	→	◌ᵃʳ	&_jdotl;	→	◌ᴶ
&_ovdot;	→	◌̇	&_arsc;	→	◌ᴬᴿ	&_ksc;	→	◌ᴷ

&_munc;	→	◌ͫ	&_orr;	→	◌ꝛ	&_sa;	→	◌ᷤ
&_oogo;	→	◌ǫ	&_oru;	→	◌ꝛ	&_tsc;	→	◌ᴛ
&_oslash;	→	◌ø	&_q;	→	◌q	&_y;	→	◌ͮ
&_omac;	→	◌ō	&_ru;	→	◌ꝝ	&_thorn;	→	◌ᷦ

For another function of Stylistic Set 10, see Chapter 6, Entering Characters with Tags.

4.12.7. ss20 – Low Diacritics

The MUFI recommendation includes a number of precomposed characters consisting of base letters with ascenders (e.g. **b**, **þ**) and a number of combining marks. Instead of being positioned above ascender height as usual (e.g. **hͣ**), the MUFI glyphs have the marks positioned above the x-height (e.g. **hͣ**). Using the MUFI code points for these precomposed glyphs can interfere with searching and drastically reduce accessibility. Users of Junicode should instead use a sequence of base character + combining mark, and apply ss20 to the two glyphs. Affected base letters are **b**, **d**, **ð**, **ꝺ**, **h**, **ħ**, **k**, **þ**, and **ꝥ** with variants: other letters are not affected. Variant shapes of **ꝺ** and **ð** that accommodate the combining mark will be substituted for the normal base characters (but this is not necessary for other base characters). Examples: **bͦ ðͯ hͭ kͤ þͭ ðʀ**.

4.12.8. cv85 – Variant of ◌ᷓ (U+1DD3, combining open a)

1=◌ᷓ.

4.12.9. cv86 – Variant of ◌ᷘ (U+1DD8, combining insular d)

1=◌ᷘ.

4.12.10. cv87 – Variant of ◌᷑ (U+1DD1, combining r rotunda)

1=◌᷑.

4.12.11. cv88 – Variant of combining dieresis (U+0308)

1=◌̈. This also affects precomposed characters on which this variant dieresis may occur, e.g. ä.

4.12.12. cv89 – Variant of ◌̅ (U+0305, combining overline)

1=◌̇̅.

4.12.13. cv90 – Variants of ◌͞◌ (U+035E, combining double macron)

1=◌͞◌, 2=◌͞◌.

4.12.14. cv92 – Variant of combining breve below (U+032E)

1=◌◌̮◌. Position the mark after the middle of three glyphs, and apply cv92 to both the mark and (at least) the middle glyph. This mark is not available via cv84.

4.13. Currency and Weights

4.13.1. cv93 – Variants of ¤ (U+00A4, generic currency sign)

1 = ΠΙ	6 = v̈	11 = ꝟ	16 = £	21 = Ͼ	26 = ʒ
2 = ✗	7 = ſ℞	12 = ꝉ	17 = ℔	22 = [illegible]	27 = [illegible]
3 = ¥	8 = gꝛ	13 = ꝺ	18 = [illegible]	23 = [illegible]	
4 = ꝥ	9 = ɸ	14 = ℒ	19 = [illegible]	24 = ß	
5 = ʒ	10 = ꝟ	15 = ₤	20 = [illegible]	25 = ßꝉ	

All of MUFI's currency and weight symbols (those that do not have Unicode code points) are gathered here, but some are also variants of other currency signs (see below).

4.13.2. cv94 – Variant of ℔ (U+2114)

1=ꝉ. Same as MUFI U+F2EB (French Libra sign).

4.13.3. cv95 – Variants of £ (U+00A3, British pound sign)

1=𞋿, 2=𞋾, 3=𞋽, 4=𞋼, 5=𞋻, 6=£. Same as MUFI U+F2EA, F2EB, F2EC, F2ED, F2EE, F2EF, pound signs from various locales.

4.13.4. cv96 – Variant of ₰ (U+20B0, German penny sign)

1=𞋰. Same as MUFI U+F2F5.

4.13.5. cv97 – Variant of ƒ (U+0192, florin)

1=𞋹. Same as MUFI U+F2E8.

4.13.6. cv98 – Variant of ℥ (U+2125, Ounce sign)

1=𞋺. Same as MUFI U+F2FD, Script ounce sign.

4.14. Ornaments

4.14.1. ornm – Ornaments

Produces ornaments (fleurons) in either of two ways: as an indexed variant of the bullet character (U+2022) or as variants of a-z, A-C:

a, 0	i, 8	q, 16	y, 24
b, 1	j, 9	r, 17	z, 25
c, 2	k, 10	s, 18	A, 26
d, 3	l, 11	t, 19	B, 27
e, 4	m, 12	u, 20	C, 28
f, 5	n, 13	v, 21	
g, 6	o, 14	w, 22	
h, 7	p, 15	x, 23	

The method with letters of the alphabet is easier, but the method with bullets will produce a more satisfactory result when text is displayed in an environment where Junicode is not available or ornm is not implemented.

4.14.2. Lady Junicode

Lady Junicode cannot be produced by an OpenType feature, believing that it would be vulgar to make herself so accessible. She has, indeed, commanded that the author of this document not publish her code point, located in one of the more private corners of the Private Use Area. She has, however, given permission to publish her miniature:

If you encounter her while adventuring in her domains, greet her respectfully, and she will welcome you graciously.

4.15. Required Features

Required features, which provide some of the font's most basic functionality—ligatures, support for other features, kerning, and more—include `ccmp` (Glyph Composition/Decomposition), `calt` (Contextual Alternates), `liga` (Standard Ligatures), `locl` (Localized Forms), `rlig` (Required Ligatures), `kern` (Horizontal Kerning), and `mark`/`mkmk` (Mark Positioning). In MS Word these features have to be explicitly enabled on the Advanced tab of the Font dialog (Ctrl-D or Cmd-D: enable Kerning, Standard Ligatures, and Contextual Alternates, and the others will be enabled automatically), but in most other applications they are enabled by default.

5. Entering characters with tags

Any character in Junicode that can be rendered using a Character Variant (`cvNN`) feature can also be rendered using a sequence consisting of a base character and two Unicode tags—that is, characters from the Unicode tag range. This range duplicates the ASCII character set (which consists, roughly, of things that can be typed on a U.S. English keyboard), but the characters it contains are normally invisible. They are used as modifiers for a preceding character (in other fonts, usually a flag symbol). Junicode contains a partial collection of tag characters:

U+E0061	U+E0062	U+E0063
U+E0064	U+E0065	U+E0066
U+E0067	U+E0068	U+E0069
U+E006A	U+E006B	U+E006C
U+E006D	U+E006E	U+E006F
U+E0070	U+E0071	U+E0072
U+E0073	U+E0074	U+E0075
U+E0076	U+E0077	U+E0078
U+E0079	U+E007A	U+E0030
U+E0031	U+E0032	U+E0033
U+E0034	U+E0035	U+E0036
U+E0037	U+E0038	U+E0039

When creating web pages or XML documents, you can enter these using character entities (e.g. `󠁳` for 🏴); along the same lines, you can use the

`\char` command (e.g. `\char"0E0073`) when composing TeX documents. But such entities and commands are not generally available in editors and word processors, and entering the tags themselves can be tricky because they are not only invisible, but also ignored by the application (the cursor skips over them). Instead of attempting to enter the code points for tags directly, use character entities consisting of **an ampersand, two underscores, the tag character, and a semicolon**. These will yield visible tags. For example, typing `&__6;` will yield the tag 6.

To use the tag method of entering characters, first type the base character from the table below, then the two tags. For example, to make a square C (C), enter one of these sequences:

Web page:	C󠁳󠁱
TeX:	C\char"0E0073\char"0E0071
Word processor:	C&__s;&__q; (appears as Cs q)

Then apply the OpenType feature `ss10` (Stylistic Set 10) to the passage or passages containing the tags or, if tags occur throughout, to the whole document. The tags will disappear and the preceding characters (the base characters) will be transformed.

For variants of the combining macron (U+0304) and perhaps other combining marks, it will often be necessary to place the COMBINING GRAPHEME JOINER (U+034F) between the macron and the base character that precedes it. This will prevent normalization (the shaping engine changing sequences of character + combining mark into precomposed characters), which interferes with the operation of tags.

Most of the two-tag sequences documented here are designed to be mnemonic. For example, the sequence in the example above stands for “square.” However, two-tag sequences are not capable of describing characters in any detail, and in some cases, where the number of variants is large (especially for period, combining macron, and currency), the tags are not descriptive at all, but rather an index (the same numbers used in the corresponding `cvNN` features).

Some tag sequences are applied to classes of character. p c makes petite caps, c a makes uppercase forms of combining marks, c n makes most enlarged

minuscules, and 🏷️🅽 makes most insular variants of alphabetic characters. To save space in a rather large section of this manual, these classes are omitted from this table.

These tags are compatible with Junicode's other OpenType features, including the `cvNN` features, and can be mixed with them. They will not interfere with the placement of combining marks, which can come either before or after the tag-pair. Use a `cvNN` feature when a variant should appear throughout the text, repeatedly in a particular passage (for example, a block quotation), or in a style. A tag sequence may be preferable for isolated forms or to override an OpenType feature.

In the list below, records for each character are color-coded as follows:

green	MUFI characters with PUA code points
yellow	Other characters with PUA code points
blue	Characters with Unicode code points
red	Characters without code points

With the information conveyed by these colors, and with code points listed where available, this table may be useful for other purposes than locating tags.

Tags or `cvNN` features are usually to be preferred to PUA code points, which should be used only where accessibility and searchability are not issues (mainly in printed texts). Unicode code points can safely be entered directly. Junicode makes a few of them accessible via `cvNN` features and tags because it may often be desirable to associate these characters with their bases rather than the Unicode code points. For example, the insular T (Ꞇ) is sure to be searchable as T if entered with the sequence T🏷️🅽, but if entered as U+A786 it may or may not be searchable as T, depending on the application.

Characters without code points can only be entered via tags or OpenType features. In Microsoft Word, where most OpenType features are unavailable (but `ss10` is), many characters can be accessed only via tags.

Base	Sequence	Result	Description / Code point
period	.TAG0 TAG1	·	Distinctio / F1F8
period	.TAG0 TAG2	.	Comma positura / F1E2
period	.TAG0 TAG3	’	High comma positura / F1E3
period	.TAG0 TAG4	;	Punctus versus / F1EA
period	.TAG0 TAG5	.,	Punctus with comma positura / F1E4
period	.TAG0 TAG6	:,	Colon with middle comma positura / F1E5
period	.TAG0 TAG7	·;·	Two dots over comma positura / F1F2
period	.TAG0 TAG8	⁖	Three dots over comma positura / F1E6
period	.TAG0 TAG9	⸗	Punctus elevatus diagonal stroke / F1F0
period	.TAG1 TAG0	؟	Punctus elevatus with high back / F1FA
period	.TAG1 TAG1	⸮	Punctus elevatus with onset / F1FB
period	.TAG1 TAG2	ʔ	Punctus flexus / F1F5
period	.TAG1 TAG3	!	Punctus exclamativus / F1E7
period	.TAG1 TAG4	؟	Punctus interrogativus / F160
period	.TAG1 TAG5	⸮	Punctus interrogativus horizontal tilde / F1E8
period	.TAG1 TAG6	⸮	Punctus interrogativus lemniskate / F1F1
period	.TAG1 TAG7	~	Wavy line / F1F9
period	.TAG1 TAG8	⁘	Signe de renvoi / F1EC
period	.TAG1 TAG9	⸗	Virgula suspensiva / F1F4
period	.TAG2 TAG0	/	Short virgula / F1F7
period	.TAG2 TAG1	·	Alternate centered period / 10A03B
period	.TAG2 TAG2	˘	Dotless punctus elevatus

Base	Sequence	Result	Description / Code point
period	.TAG2TAG3	˸	Punctus elevatus with s-shaped top
period	.TAG2TAG4	˸	Dotless punctus elevatus with s-shaped top
period	.TAG2TAG5	˸	Enlarged punctus elevatus
period	.TAG2TAG6	˸	Enlarged dotless punctus elevatus
period	.TAG2TAG7	˸	Enlarged punctus elevatus with s-shaped top
period	.TAG2TAG8	˸	Enlarged dotless punctus elevatus with s-shaped top
period	.TAG2TAG9	;	Semicolon with dot / F75B
U+0304	◌̄TAG0TAG1	◌̋	Combining curly bar above / F1CC
U+0304	◌̄TAG0TAG2	◌̏	Combining diagonal macron
U+0304	◌̄TAG0TAG3	◌̐	Combining bar above with dot / F1C0
U+0304	◌̄TAG0TAG4	◌̑	Combining angular zigzag / F1C7
U+0304	◌̄TAG0TAG5	◌̒	Combining curly zigzag / F1C8
U+0304	◌̄TAG0TAG6	◌̰	Combining vertical tilde / 033E
U+0304	◌̄TAG0TAG7	◌ͣ	Combining ligature an / F036
U+0304	◌̄TAG0TAG8	◌ᴺ	Combining ligature aN / F03A
U+0304	◌̄TAG0TAG9	◌ᵃ	Combining ligature ar / F038
U+0304	◌̄TAG1TAG0	◌ᴿ	Combining ligature aR / F130
U+0304	◌̄TAG1TAG1	◌ᴮ	Combining B / F013
U+0304	◌̄TAG1TAG2	◌ᴰ	Combining D / F016
U+0304	◌̄TAG1TAG3	◌ᵉ̄	Combining e with macron / F136
U+0304	◌̄TAG1TAG4	◌ᵉ̨	Combining e with ogonek / F135
U+0304	◌̄TAG1TAG5	◌ᶦ	Combining dotless i / F02F
U+0304	◌̄TAG1TAG6	◌ʲ	Combining j / F030

Base	Sequence	Result	Description / Code point
U+0304	◌̄ TAG 1 7	◌̄ᴶ	Combining dotless j / F031
U+0304	◌̄ TAG 1 8	◌̄ᴷ	Combining K / F01C
U+0304	◌̄ TAG 1 9	◌̄ᵐ	Combining uncial m / F01F
U+0304	◌̄ TAG 2 0	◌̄ᵒ̄	Combining o with macron / F13F
U+0304	◌̄ TAG 2 1	◌̄ᵒ̨	Combining o with ogonek / F13E
U+0304	◌̄ TAG 2 2	◌̄ᵒ̸	Combining o with stroke / F032
U+0304	◌̄ TAG 2 3	◌̄ᑫ	Combining q / F033
U+0304	◌̄ TAG 2 4	◌̄ꝝ	Combining rum abbreviation / F040
U+0304	◌̄ TAG 2 5	◌̄ᵀ	Combining T / F02A
U+0304	◌̄ TAG 2 6	◌̄ʸ	Combining y / F02B
U+0304	◌̄ TAG 2 7	◌̄ᵇ	Combining thorn / F03D
U+0304	◌̄ TAG 2 8	◌̄ꝛ	Combining ligature o r rotunda / F03E
U+0304	◌̄ TAG 2 9	◌̄ꝝ	Combining ligature letter o rum / F03F
U+0304	◌̄ TAG 3 0	◌̄̈	Combining diagonal dieresis
U+0304	◌̄ TAG 3 1	◌̄́	Combining dot and acute
U+0304	◌̄ TAG 3 2	◌̨̄	Combining ogonek and dot
U+0304	◌̄ TAG 3 3	◌̄	Combining narrow macron
U+0304	◌̄ TAG 3 4	◌̄̃	Combining macron with serifs
U+0304	◌̄ TAG 3 5	◌̄ᵌ	Combining et sign
U+0304	◌̄ TAG 3 6	◌̄̓	Combining spiritus asper sign
U+0304	◌̄ TAG 3 7	◌̄ₐ	Attached subscript a
U+0304	◌̄ TAG 3 8	◌̄	Curved macron with half serifs / 10A05D
U+0304	◌̄ TAG 3 9	◌̄	Macron with half serifs / 10A075

Base	Sequence	Result	Description / Code point
A	A 🏷s 🏷q	Ⱥ	Square A / F13A
A	A 🏷s 🏷d	Ⱥ	A with diagonal stroke / E8DA
a	a 🏷u 🏷n	ɐ	Uncial a / F214
a	a 🏷o 🏷p	ɑ	Open a / F202
a	a 🏷c 🏷l	a	Closed a / F203
a	a 🏷n 🏷e	ɐ	Neckless a / F215
a	a 🏷o 🏷c	𐭀	oc-shaped a / 10A001
a	a 🏷h 🏷c	𐭂	high caroline a / 10A003
a	a 🏷s 🏷q	𐭄	Square a / 10A004
a	a 🏷p 🏷o	𐭅	Pointed a / 10A005
a	a 🏷o 🏷n	𐭁	Enlarged oc-shaped a / 10A002
a	a 🏷i 🏷s	𐭃	Insular a with short headstroke / 10A07C
C	C 🏷s 🏷q	Ɔ	Square C / F106
c	c 🏷c 🏷u	ƈ	c with curl / F198
c	c 🏷h 🏷o	𐭈	c with horn / 10A076
D	D 🏷i 🏷e	𐭊	Enlarged minuscule insular D
d	d 🏷i 🏷1	𐭋	Insular d default form / A77A
d	d 🏷i 🏷2	𐭌	Insular d second form
d	d 🏷i 🏷3	𐭍	Insular d third form
d	d 🏷i 🏷e	𐭊	Enlarged minuscule insular d / EEE4
d	d 🏷c 🏷u	ɗ	d with curl / F193
d	d 🏷i 🏷s	𐭉	Insular d with straight back / EEE3
E	E 🏷u 🏷c	Ɛ	Uncial closed E / F217

Base	Sequence	Result	Description / Code point
E	E 🏷u 🏷n	Ҿ	Uncial E / F10A
e	e 🏷u 🏷n	ҿ	Uncial e / F218
e	e 🏷b 🏷a	e	e with bar / F219
e	e 🏷b 🏷h	e	High e with bar / F21A
e	e 🏷s 🏷t	ҽ	e with straight back and tongue / 10A058
e	e 🏷h 🏷t	ҽ	e with horn and tongue / 10A07B
e	e 🏷h 🏷h	ҽ	High e with horn and tongue / 10A07E
F	F 🏷i 🏷e	ꝼ	Enlarged minuscule insular F
f	f 🏷i 🏷s	ſ	Insular f with split top / 10A012
f	f 🏷i 🏷1	ſ:	Insular f with dotted hooks / F21C
f	f 🏷i 🏷2	ſ	Semi-closed insular f / EBD5
f	f 🏷i 🏷3	ſ	Closed insular f / EBD6
f	f 🏷i 🏷e	ꝼ	Enlarged minuscule insular f / EEFF
f	f 🏷n 🏷a	f	Narrow f / F000B
f	f 🏷c 🏷u	fɾ	f with curl / F194
f	f 🏷h 🏷d	f	f with half descender / 10A06F
G	G 🏷o 🏷r	Ꝙ	Ormulum G with bar / A7D0
G	G 🏷s 🏷q	Ꞡ	Square G / F10E
g	g 🏷o 🏷r	ɡ	Ormulum g with bar / A7D1
g	g 🏷s 🏷c	g	Script g / 0261
g	g 🏷c 🏷u	gɾ	g with flourish / F196
g	g 🏷n 🏷e	ȝ	Enlarged insular g / 10A024
g	g 🏷c 🏷1	g	Closed g with separate loops / F21D

Base	Sequence	Result	Description / Code point
g	g 𞁣 𞁲	ɡ	Closed g with large lower loop / F21E
g	g 𞁣 𞁳	ɡ	Closed g with small lower loop / F21F
g	g 𞁯 𞁴	ɠ	Closed g with straight-sided loop / 10A05E
g	g 𞁯 𞁥	ɢ	Orm's hard g, enlarged
g	g 𞁩 𞁳	ȝ	Insular g with short right stroke / 10A060
g	g 𞁩 𞁯	ʒ	Insular g with open bowl / 10A06D
g	g 𞁣 𞁯	ɡ	Caroline g with open bowl / 10A078
g	g 𞁩 𞁬	ȡ	Insular g with short left stroke / 10A07F
H	H 𞁵 𞁮	ƕ	Uncial H / F110
H	H 𞁨 𞁬	Ⱨ	H with high left stem / 10A026
h	h 𞁤 𞁥	ɧ	h with descender / F23A
h	h 𞁡 𞁵	ɦ	h-shaped *autem* abbreviation / E8A3
h	h 𞁥 𞁤	ɧ	Enlarged minuscule h with descender
h	h 𞁣 𞁡	ɦ	Caroline a
h	h 𞁥 𞁣	ɦ	Enlarged Caroline h / 10A025
h	h 𞁤 𞁢	ɦ	Caroline h with right descender and bar / 10A029
h	h 𞁣 𞁵	ɦ	h with curl
I	I 𞁤 𞁡	İ	I with dot above / 0130
I	I 𞁤 𞁥	I	I with descender / A7FE
I	I 𞁵 𞁳	ǀ	I without serifs / 10A074
i	i 𞁤 𞁬	ɪ	Dotless i / 0131
i	i 𞁬 𞁯	ɩ	Long I / F220
i	i 𞁢 𞁤	ɨ	i with double bar / E8A1

Base	Sequence	Result	Description / Code point
i	i f TAG i TAG	ij	i final form (when i precedes)
i	i f TAG 2 TAG	j	i final form
J	J d TAG a TAG	J̇	J with dot above / E15C
j	j d TAG TAG	ɟ	Dotless j / 0237
j	j s TAG d TAG	j̷	j with double bar / E8A2
j	j d TAG a TAG	j	j with dot accent / F000D
k	k u TAG n TAG	ᶄ	Uncial k / F208
k	k u TAG d TAG	ᶄ	Uncial k with descender / 10A02C
k	k c TAG i TAG	ᶄ	Closed k, one / F221
k	k c TAG 2 TAG	ᶄ	Closed k, two / F209
k	k c TAG u TAG	ᶄ	k with curl / F195
k	k c TAG TAG	k	k with c-like bars k / 10A077
l	l d TAG e TAG	ɭ	Descending l / F222
l	l s TAG h TAG	ɬ	l with high stroke / A749
l	l s TAG f TAG	ɬ	l with high stroke ending with flourish / F000F
U+019A	ɬ c TAG u TAG	ɬ	barred l with curl
M	M u TAG d TAG	ᶆ	Uncial M with descender / F224
M	M u TAG n TAG	ᶆ	Uncial M / F11A
M	M a TAG i TAG	ᴍ	Epigraphic M / A7FF
m	m u TAG d TAG	ᶆ	Uncial m with descender / F23D
m	m u TAG n TAG	ᶆ	Uncial m / F23C
m	m d TAG e TAG	ᶆ	m with descender / F223
m	m u TAG e TAG	ᶆ	Enlarged uncial m / F225

Base	Sequence	Result	Description / Code point
m	m 🏷u 🏷m	ᵯ	Mediascule uncial m with descender / F226
N	N 🏷d 🏷e	ᴎ	N with descender / F229
N	N 🏷l 🏷d	Ʇ	N with low bar and descender / 10A036
n	n 🏷d 🏷e	ᵰ	n with descender / F228
n	n 🏷b 🏷a	ꞑ	n with bar / E7B2
n	n 🏷c 🏷u	ᶇ	n with curl / F19A
n	n 🏷s 🏷d	ᴺ	Small cap n with descender / F22A
n	n 🏷a 🏷s	ᵨ	n with attached subscript a
n	n 🏷w 🏷p	ᴖ	Wide petite cap n with descender / 10A037
n	n 🏷e 🏷m	ꟸ	enim abbreviation
P	P 🏷r 🏷e	ꟼ	Reversed P / A7FC
p	p 🏷a 🏷i	ᵱ	Alternate p for Ormulum
Q	Q 🏷s 🏷m	ᵠ	Q with stem / F22C
Q	Q 🏷l 🏷i	Q◌	Q with long tail, one
Q	Q 🏷l 🏷2	Q◌◌	Q with long tail, two
q	q 🏷s 🏷d	ᶐ	q with diagonal stroke / E8B4
R	R 🏷r 🏷o	ʁ	R rotunda / A75A
r	r 🏷r 🏷o	ɹ	r rotunda / A75B
r	r 🏷c 🏷u	ᵲ	r with curl / F19B
r	r 🏷c 🏷2	ɾ	r with alternate curl
r	r 🏷i 🏷b	ꞅ	Insular r baseline form / 10A056
r	r 🏷i 🏷s	ꞇ	Insular r with short right leg / 10A06B
r	r 🏷l 🏷s	ɼ	r with long leg and stroke / E7E4

Base	Sequence	Result	Description / Code point
S	S 𝗌 𝗀	ꞩ	Sigmoid S / A7D8
S	S 𝖺 𝖽	Ꞅ	Ascending and descending S / 10A03D
S	S 𝗋 𝗓	Ꞃ	Reverse Z-shaped S / 10A03F
S	S 𝗌 𝖽	ꞩ᾽	S with diagonal stroke (Sanctus abbrev)
S	S 𝗆 𝗌	ſ	Middle Scots S
S	S 𝗁 𝗈	S	Hollow S
s	s 𝗂 𝗌	ꝛ	Insular s with split top / 10A03E
s	s 𝗌 𝗀	ꞩ	Sigmoid s / A7D9
s	s 𝖺 𝖽	ʃ	Ascending and descending s / 10A040
s	s 𝗅 𝗈	ſ	Long s / 017F
s	s 𝗅 𝖽	ſ	Long s with descender / F127
s	s 𝗅 𝖿	ſ	Long s with flourish / E8B7
s	s 𝗅 𝗌	ſ	Long s with diagonal stroke / E8B8
s	s 𝗅 𝗅	ſ	Long s with loop
s	s 𝗌 𝗍	ſ	Long s with short tail / 10A033
s	s 𝗆 𝗌	ſ	Middle Scots s
s	s 𝖼 𝗎	ꞃ	s with curl
s	s 𝗁 𝗄	ꞅ	s with hook
s	s 𝗅 𝗉	ſ	Long s with descender and loop / 10A067
t	t 𝖼 𝗎	ꞇ	t with curl / F199
t	t 𝖼 𝖽	⟙	Cap-like t with descender / 10A044
t	t 𝖺 𝗉	ⱦ	t with approach
U	U 𝗅 𝖼	ᑌ	Lowercase-shaped U

Base	Sequence	Result	Description / Code point
U	U[l][d]	Ꞅ	Lowercase-shaped U with descender / 10A048
v	v[s][i]	ꝟ	Enlarged minuscule v / EEF8
v	v[s][d]	ꝸ	v with diagonal stroke / E8BA
v	v[b][a]	ꝩ	v with bar / E74E
v	v[s][2]	ꝯ	v with two bars / E8BC
v	v[e][l]	ⱴ	Enlarged minuscule v with low point / 10A049
Ⱳ	Ⱳ[a][n]	Ⱳ	Ⱳ Anglicana
w	w[a][n]	ⱳ	w Anglicana
ƿ	ƿ[t][b]	ƿ	wynn with triangular bowl / 10A06E
x	x[l][o]	ꭓ	x with long left leg / AB57
x	x[s][i]	ꝰ	x with diagonal stroke (upper left) / E8BD
x	x[s][2]	ᶍ	x with diagonal stroke (lower right) / E8BE
x	x[s][3]	᷍	x with two diagonal strokes (lower right) / E8CE
Y	Y[s][d]	Ỿ	Y with diagonal stroke (Hymnus abbrev) / E8DB
y	y[r][m]	ʏ	y with main right stroke / F233
y	y[b][a]	ɏ	y with bar / E77B
y	y[c][u]	ʏ	Curved y / 10A054
y	y[s][c]	ʏ	Short curved y / 10A04F
Z	Z[v][i]	Ꙁ	Visigothic Z / A762
z	z[v][i]	ꙁ	Visigothic z / A763
z	z[m][g]	ȝ	Middle High German z / F238
U+0104	Ą[s][i]	Ⱥ	A with short diagonal stroke / F0000
U+0104	Ą[s][2]	Ⱥ	A with long diagonal stroke / F001E

Base	Sequence	Result	Description / Code point
U+0104	Ą[f][l]	Ą	A with flourish / F0002
U+0105	ą[s][1]	ą	a with short diagonal stroke / F0001
U+0105	ą[s][2]	ą	a with long diagonal stroke / F001F
U+A733	aa[u][n]	ꜳ	Uncial aa
U+A733	aa[c][l]	ꜳ	Closed aa / EFA0
U+00C6	Æ[n][e]	Æ	Neckless Æ / EFAE
U+00E6	æ[n][e]	æ	Neckless æ / EFA1
U+00E6	æ[o][p]	ɶ	Open æ / F204
U+00E6	æ[s][q]	ꜵ	æ with square a / 10A006
U+00E6	æ[r][o]	ꜵ	æ with round a / 10A007
U+00E6	æ[r][o]	ꜵ	Uncial æ
U+00E6	æ[s][h]	ꜷ	Square æ with high e
U+00E6	æ[p][o]	ꜵ	æ with pointed a
U+00E6	æ[r][h]	ꜷ	æ with round a and high e
U+00E6	æ[c][t]	ᴂ	æ with caroline a and tongue
U+00E6	æ[n][t]	ᴂ	æ with neckless a and tongue
U+A734	AO[n][e]	Ꜫ	Neckless AO / F205
U+A734	AO[e][1]	ꜳ	Enlarged minuscule AO
U+A734	AO[e][2]	ꜳ	Enlarged minuscule AO with smaller O
U+A735	ao[e][1]	ꜳ	Enlarged minuscule ao / EFDE
U+A735	ao[e][2]	ꜳ	Enlarged minuscule ao with smaller o / EAF2
U+A735	ao[n][e]	ꜫ	Neckless ao / F206
U+A735	ao[u][n]	ꜳ	Uncial ao

Base	Sequence	Result	Description / Code point
U+A739	ꝡ🏴	ꝡ	Neckless av / EFA2
U+0111	đ🏴	đ᾽	d with flourish / F0007
U+00F0	ð🏴	ꝛo	Enlarged insular eth / 10A052
U+00F0	ð🏴	ð	Insular eth with straight back / 10A061
U+00F0	ð🏴	ð	Insular eth with half crossbar / 10A07A
U+0118	Ę🏴	Ę	E with centered ogonek
U+0118	Ę🏴	Ɇ	E with diagonal stroke / F0009
U+0118	Ę́🏴	Ę́	E with dot, acute, and centered ogonek
U+0118	Ę́🏴	Ɇ́	E with dot, acute, and diagonal stroke
U+0119	ę🏴	ę	e with centered ogonek
U+0119	ę🏴	ɇ	e with diagonal stroke / F000A
U+0119	ę́🏴	ę́	e with dot, acute, and centered ogonek
U+0119	ę́🏴	ɇ́	e with dot, acute, and diagonal stroke
&_eogo;	◌&_eogo;🏴	◌̨	Combining e with centered ogonek
&_eogo;	◌&_eogo;🏴	◌̵̨	Combining e with diagonal stroke
U+021C	ȝ🏴	ȝ	Yogh with flat top
U+021C	ȝ🏴	ȣ	Yogh with insular shape
U+021D	ȝ🏴	ȝ	yogh with flat top
U+021D	ȝ🏴	ȣ	yogh with insular shape
U+014A	Ŋ🏴	ŋ	Rounded Ŋ with low hook
U+014A	Ŋ🏴	ɲ	Rounded Ŋ with baseline hook
U+A7C1	ꝁ🏴	ꝁ	Old Polish o with broken slash / F0011
U+A7C1	ꝁ🏴	ø	Old Polish o with short slash / F0012

Base	Sequence	Result	Description / Code point
U+A7C1	ɔ a 3	ꝏ	Old Polish o with lower left slash / F0013
U+A7C1	ɔ a 4	ꝏ	Old Polish o with upper right slash / F0014
U+1E8F	ẏ s d	ꭓ	Short curved y with dot / 10A050
U+A765	þ a 1	ꝥ	thorn with stroke with different slant / F149
U+A765/ENG	ꝥ a 1	þ	thorn with stroke with different slant
U+0241	ʔ a 1	ʾ	Alternate glottal stop / F001B
U+204A	⁊ a 1	ꝛ	Tironian et sign later form / F001D
U+204A	⁊ a 2	ꝛ̶	Tironian et sign later form with bar
U+204A	⁊ a 3	ɔ	Tironian et sign without descender / 10A01E
U+204A	⁊ a 4	ⅎ	Tironian et sign with bar
U+204A	⁊ a 5	ɐ	Tironian et sign round form
U+204A	⁊ a 6	⁊	Generic Tironian et sign
U+204A	⁊ a 7	ⅎ	Generic Tironian et sign with bar
U+204A	⁊ a 8	ꝯ	Generic Tironian et sign with curl
U+204A	⁊ a 9	ꝯ̶	Generic Tironian et sign with curl and bar
U+204A	⁊ a 0	⁊	Tironian et sign high and low / 10A069
U+2E52	⁊ a 1	ꝛ	Tironian Et sign later form / F001C
U+2E52	⁊ a 2	ꝛ̶	Tironian Et sign later form with bar
U+2E52	⁊ a 3	ɔ	Tironian Et sign without descender
U+2E52	⁊ a 4	ⅎ	Tironian Et sign with bar
U+2E52	⁊ a 5	ɐ	Tironian Et sign round form
U+2E52	⁊ a 6	⁊	Generic Tironian Et sign
U+2E52	⁊ a 7	ⅎ	Generic Tironian Et sign with bar

Base	Sequence	Result	Description / Code point
U+2E52	⁊a 8	⁊	Generic Tironian Et sign with curl
U+2E52	⁊a 9	⁊	Generic Tironian Et sign with curl and bar
U+2E52	⁊a 0	⁊	Capital Tironian Et sign high and low / 10A068
U+2E4D	⁋s s	⸗	s-shaped paragraphus
U+2E4D	⁋c c	⸘	cc-shaped paragraphus
U+2E4D	⁋c s	⸙	C-with-bar paragraphus
U+00AF	¯0 1	ˏ	Spacing zigzag
U+035E	◌a 1 ◌	◌͞◌	Double macron with serifs
U+035E	◌a 2 ◌	◌͞◌	Shorter double macron with serifs
U+00B7	·h i	·	Distinctio / F1F8
U+00B7	·s r	·	Slightly raised period / 10A03B
comma	,a 1	‚	Comma positura / F1E2
comma	,a 2	’	High comma positura / F1E3
semicolon	;a 1	;	Punctus versus / F1EA
semicolon	;a 2	.,	Punctus with comma positura / F1E4
semicolon	;a 3	:,	Colon with middle comma positura / F1E5
semicolon	;a 4	·̓	Two dots over comma positura / F1F2
semicolon	;a 5	⁖	Three dots over comma positura / F1E6
semicolon	;a 6	.,	Punctus with double comma positura 10A042
semicolon	;a 7	⁘	Hexagonal positura / 10A043
semicolon	;a 8	:˘	Positura with two dots and tick
semicolon	;a 9	;	Semicolon with dot / F75B
U+2E4E	⁖a 1	⁖	Punctus elevatus diagonal stroke / F1F0

Base	Sequence	Result	Description / Code point
U+2E4E	⹎ a 2	⹎	Punctus elevatus with high back / F1FA
U+2E4E	⹎ a 3	⹎	Punctus elevatus with onset / F1FB
U+2E4E	⹎ a 4	⹎	Punctus flexus / F1F5
U+2E4E	⹎ a 5	⹎	Dotless punctus elevatus
U+2E4E	⹎ a 6	⹎	Punctus elevatus with s-shaped top
U+2E4E	⹎ a 7	⹎	Dotless punctus elevatus with s-shaped top
U+2E4E	⹎ a 8	⹎	Enlarged punctus elevatus
U+2E4E	⹎ a 9	⹎	Enlarged dotless punctus elevatus / F1F5
U+2E4E	⹎ b 1	⹎	Enlarged punctus elevatus with s-shaped top
U+2E4E	⹎ b 2	⹎	Enlarged dotless punctus elevatus with s-shaped top
exclam	! a 1	!	Punctus exlamativus / F1E7
question	? a 1	?	Punctus interrogativus / F160
question	? a 2	?	Punctus interrogativus horizontal tilde / F1E8
question	? a 3	?	Punctus interrogativus lemniskate / F1F1
asciitilde	~ a 1	~	Wavy line / F1F9
asciitilde	~ s t	~	Wavy line / F1F9
asterisk	* a 1	∵	Signe de renvoi / F1EC
asterisk	* a 2	*	Aldine asterisk with long limb / F1EC
asterisk	* a 3	*	Five-spoked Aldine asterisk / F1EC
asterisk	* a 4	*	Eight-spoked Aldine asterisk / F1EC
asterisk	* a 5	※	Dotted Aldine asterisk / F1EC
slash	/ a 1	/	Virgula suspensiva / F1F4
slash	/ a 2	/	Short virgula / F1F7

Base	Sequence	Result	Description / Code point
hyphen	-[c][u]	⸗	Hyphen with curl / F1F7
U+A75D	ꝝ[a][1]	ꝛ	Alternate rum abbreviation / F0016
U+035B	◌͛[a][n]	◌͝	Combining angular zigzag / F1C7
U+035B	◌͛[c][u]	◌͡	Combining curly zigzag / F1C8
U+035B	◌͛[v][e]	◌͠	Combining vertical zigzag (tilde) / 033E
U+A770	ꝰ[a][1]	ꝯ	Baseline spacing us abbreviation / F1A6
U+A770	ꝰ[a][2]	Ꝯ	Uppercase us abbreviation / F1A5
U+A76B	ꝫ[a][1]	;	Semicolon-like et abbreviation / F1AC
U+A76B	ꝫ[a][2]	ꝫ	Subscript et abbreviation
U+A76B	ꝫ[a][3]	:	Colon-like et abbreviation
U+A76B	ꝫ[a][4]	⁏	et abbreviation crossing preceding q
U+1DD3	◌ᷓ[a][1]	◌ᷓ	Combining flattened a with macron / F1C1
U+1DD8	◌ᷘ[a][1]	◌ᷘ	Alternate combining insular d / F0005
U+1DE3	◌ᷣ[r][m]	◌ᷣ	Combining Latin small letter rum / F040
U+00E4	ä[d][i]	ȁ	a with diagonal dieresis / E8D5
U+00F6	ö[d][i]	ȍ	o with diagonal dieresis / E8D7
U+0308	◌̈[d][i]	◌̑	Combining diagonal dieresis
U+0305	◌̅[a][1]	◌̇̅	Combining horizontal stroke with dot / F1C0
U+032E	◌̮[a][1]◌	◌◌◌	Breve below three letters / F1FC
U+00A4	¤[0][1]	ΠΙ	Latin as libralis sign / F2E0
U+00A4	¤[0][2]	ꭓ	Latin small capital letter x with bar / F2E2
U+00A4	¤[0][3]	ꭚ	Latin small capital letter y with bar / F2E3
U+00A4	¤[0][4]	ꭒ	Latin small capital letter d with slash / F2E4

Base	Sequence	Result	Description / Code point
U+00A4	¤ 0 5	ʒ	Pharmaceutical dram sign / F2E6
U+00A4	¤ 0 6	v̈	Ecu sign / F2E7
U+00A4	¤ 0 7	ʃʶ	Floren sign with loop / F2E8
U+00A4	¤ 0 8	ɡɐ	Groschen sign / F2E9
U+00A4	¤ 0 9	ɸ	Helbing sign / F2FB
U+00A4	¤ 1 0	⸗	Krone sign / F2FA
U+00A4	¤ 1 1	₥	Dutch libra sign / F2EA
U+00A4	¤ 1 2	₶	French libra sign / F2EB
U+00A4	¤ 1 3	₰	Italian libra sign / F2EC
U+00A4	¤ 1 4	₤	Flemish libra sign / F2ED
U+00A4	¤ 1 5	₺	Lira nuova sign / F2EE
U+00A4	¤ 1 6	£	Lira sterlina sign / F2EF
U+00A4	¤ 1 7	₻	Old mark sign / F2F0
U+00A4	¤ 1 8	₷	Old flourish mark sign / F2F1
U+00A4	¤ 1 9	m₷	Marked small letter m sign / F2F2
U+00A4	¤ 2 0	ɱ	Flourished small letter m sign / F2F3
U+00A4	¤ 2 1	℄	Pharmaceutical obelus sign / F2F4
U+00A4	¤ 2 2	₹r	Penning sign / F2F5
U+00A4	¤ 2 3	₰	Old Reichstaler sign / F2F6
U+00A4	¤ 2 4	ß	German schilling sign / F2F7
U+00A4	¤ 2 5	ßℓ	German script schilling sign / F2F8
U+00A4	¤ 2 6	ℨ	Scudi sign / F2F9
U+00A4	¤ 2 7	℥	Script ounce sign / F2FD

Base	Sequence	Result	Description / Code point
U+2114	℔🏷a🏷1	℔	French libra sign / F2EB
U+00A3	£🏷a🏷1	₩	Dutch libra sign / F2EA
U+00A3	£🏷a🏷2	℔	French libra sign / F2EB
U+00A3	£🏷a🏷3	₰	Italian libra sign / F2EC
U+00A3	£🏷a🏷4	₤	Flemish libra sign / F2ED
U+00A3	£🏷a🏷5	₺	Lira nuova sign / F2EE
U+00A3	£🏷a🏷6	£	Lira sterlina sign / F2EF
U+20B0	₰🏷a🏷1	₱	Penning sign / F2F5
U+0192	ƒ🏷a🏷1	₣	Floren sign with loop / F2E8
U+2125	℥🏷a🏷1	℈	Script ounce sign / F2FD

6. Transcribing records

This chapter provides guidance for persons transcribing records (laws and other public documents) in the style of Charles Trice Martin's *The Record Interpreter*, *Statutes of the Realm*, and similar guides and editions. Unlike most editions of early texts, these retain (or recommend retaining) the capitalization, punctuation, and abbreviations of their manuscript sources.

6.1. A preliminary note on transcription

Here are a few observations, based on a long career as a scholarly editor of medieval and eighteenth-century texts.

Before embarking on the task of transcribing an old document, ask yourself what value you want to add to the document as it already exists, because different kinds of transcription add different kinds of value. The kind of transcription that adds the least is that which aims at the exact *visual* reproduction of a document. A transcript is not a facsimile: it needs to do something that a photograph can't do.

Converting a document from visual image to Unicode-encoded text adds a good bit of value all by itself, but only if done with due regard for the semantics of Unicode characters. Every Unicode character has a meaning, and that meaning is a help to readers. Using the wrong character is a hinderance to readers, even if it *looks* right.

For example, in transcribing a Middle English text, you may decide that the Unicode ezh (ʒ, U+0292) looks more like the yogh in your source than the Unicode yogh (ȝ, U+021D) and therefore decide to use it for yogh. But the ezh is not a yogh! It is a character in the International Phonetic Alphabet and a

letter in the alphabets of several minor languages—but not a letter in the Middle English language. If you use it where the yogh is called for, it will make your text less accessible and less searchable. Indexing, concordance and bibliographical programs may be misled by it; screen readers will misinterpret it. To solve one problem (that of visual representation), you may well have introduced a host of far more serious problems.

Fortunately, Junicode offers a solution for this particular problem. The OpenType feature `cv63` substitutes for the yogh a character that *looks* like the ezh but is semantically a yogh and therefore will be handled correctly by applications. But neither Junicode nor any other font can solve every problem of this kind. Sometimes you will have to call to mind the important principle stated above: *A transcript is not a facsimile*. It is much more important that it should have the same *meaning* as the original than that it should have the same *look*.

This chapter concerns the transcription of texts in Latin (and to some extent, other archaic languages, e.g. Old and Middle English, Old French). It is long-standing custom, when transcribing certain kinds of documents, to retain marks of abbreviation—for example, the ꝓp̊ you may find in a manuscript or printed edition representing the word *propterea*. This is okay—and Junicode can help with the task. But when dealing with the abbreviations, punctuation, and diacritics of an old text it is more important than ever that you use semantically correct characters for your transcription, as this will help readers who already face significant challenges.

For example, the abbreviation ꝓp̊ as printed here consists of an underlying sequence of Unicode characters: ꝓ (U+A753, the common abbreviation for *pro*) + p + ◌̊ (U+0363, the combining small *a*). The OpenType feature `hlig` (Historical Ligatures) has been applied to this sequence, changing its appearance but not its underlying value. That underlying value is intelligible to computer applications in the sense that they can recognize each character.

This doesn't mean, though, that computer programs can correctly interpret ꝓp̊ as *propterea*. Many (probably *most*) Latin abbreviations are ambiguous: this one, for example, can mean *propterea* or *propria*. Some abbreviations (most notoriously ◌ͤ U+035B) can mean many things, depending on context. It takes a human being with a knowledge of Latin to interpret them correctly.

So another way you can add value in your transcript is by interpreting ab-

breviations like ꝓ̊ and supplying expansions of them. Fortunately, systems for representing texts often offer ways to handle this task gracefully. For example, in a TEI (Text Encoding Initiative) text, you would use this construction:

```
<choice>
  <abbr rend="hlig">ꝓ̊</abbr>
  <expan>propterea</expan>
</choice>
```

This kind of structure can be approximated in HTML, with supporting CSS and scripting to allow readers to choose between a "diplomatic" version, with unexpanded abbreviations, and a "reading" version, with expanded abbreviations and perhaps other amenities, such as modern punctuation and capitalization.

There are other ways to add value to a transcript—for example, by correcting errors, annotating the content, or writing textual notes. Each of these operations takes your transcript farther from the facsimile and closer to the edition.

6.2. Common combining marks

A **combining mark** is a character that combines with another character (called the **base**) to form a character with accent (e.g. é) or an abbreviation (e.g. p̓ for *prae*). Unicode and the Medieval Unicode Font Initiative (MUFI) offer code points for many precomposed combinations of base + combining mark, but it is also possible to place any mark over any base character by entering first the base and then the combining mark. It is also possible to place a combining mark over another combining mark. For example, to produce q̄̃, enter this sequence: q (U+0071) + U+0363 + U+0304.

Junicode 2 contains many variants of combining marks: for example the curly zigzag ◌͛ is a variant of Unicode's angular zigzag ◌͛ (U+035B), produced by applying the OpenType feature `cv81[2]` to **both the base character and the combining mark**. Sometimes the combination of base + combining mark + OpenType feature will not produce the desired effect. When this happens, place U+034F (a special invisible combining mark, included in Unicode for exactly this purpose) between the base and the (visible) mark.

a. For a straight stroke over any letter, use the COMBINING MACRON (U+0304):

ōnis *omnis*; om̄is *omnis*; dāpna *dampna*; dam̄pa *dampna*.

The combining macron can also be applied above superscripts and combining marks. Apply the OpenType feature `cv84[33]` for a narrower macron:

antiquā *antiquam*; q̄̄ *quam*.

For the superscript *a*, use the OpenType feature `sups` (see r. below).

b. For a straight stroke through a tall letter, use the COMBINING SHORT STROKE OVERLAY (U+0335): **f̵ d̵ l̵**. But Unicode also has precomposed versions of **d**, **l** and other characters with stroke, e.g. **đ** (U+0111), **ł** (U+019A).

c. For **~** above any character, use the COMBINING TILDE (U+0303):

ã *ac*, *apud*; ã *alias*.
dn̄s *dominus*; car̃ina *carmina*; fc̃is *factis*.
põita *posita*.

d. For **~** through a vertical stroke, use the TILDE OVERLAY (U+0334): **ł̴ đ̴** (U+0303 would be positioned above the letter, e.g. **l̃**, **d̃**). For the ligatures **ﬄ**, **ﬅ**, and **ﬀ**, type the sequence for **ł**, etc. twice.

e. For the tilde positioned above two letters, use COMBINING DOUBLE TILDE (U+0360) between the letters. It is automatically repositioned to clear tall characters: c͠o t͠o d͠o o͠l. The same is true of DOUBLE BREVE (U+035D) c͝o d͝o, DOUBLE MACRON (U+035E) c͞o d͞o, DOUBLE INVERTED BREVE (U+0361) c͡o d͡o, and DOUBLE CIRCUMFLEX (U+1DCD) c᷍o d᷍o.

f. The figure used to represent *er* (and other similar combinations) is a common medieval abbreviation which takes many forms. The semantically correct Unicode character is the COMBINING ZIGZAG (◌̚, U+035B), but the best match in Junicode 2 for the figure as it appears in the *Record Interpreter* and the *Statutes* is a gothic variant of this, which MUFI encodes as U+F1C8 (the curly form zigzag). However, because for technical reasons many applications will not position the MUFI character correctly over the base, that code point should be avoided. The best way to access this variant is to apply `cv81[2]` to U+035B, as here:

de̚be *debere*; int̚ *inter*; ſ̚rū *ferrum*; gn̚o *generatio*; p̚; *prae*; serv̚e *servire*.

The curly form of the combining zigzag may be attached to any letter, and it may change shape depending on the letter it is attached to (including caps, for which use the case feature, and small caps: A͋B͋C͋D͋).

g. All letters a–z, and several others too, have combining forms. You may access these via their code points, when they are standard Unicode, via the cv84 feature, or via Junicode's special entity references. For details, see 4.10.3, Character Entities for Combining Marks.

q̊ *quo*; q̉ *qui*; quatt&_orr; *quattuor*.

6.3. Spacing characters

h. The symbol for *is*, *es* and a number of other abbreviations is the IS-SIGN (U+A76D):

forſ *foris*; om̄ſ *omnes*; ꝯtſ *competentes*; infſ *infortunium*.

This character will sometimes ligature with the preceding letter. The italic version differs from the roman stylistically (*forſ omſ ꝯtſ infſ*), but it will be intelligible to informed readers.

i. There are two characters for *-us* in Unicode: SPACING US U+A770 (do not confuse this with CON U+A76F) and COMBINING US U+1DD2. The *Record Interpreter* and *Statutes* appear to use only the spacing character:

ip̃iꝰ *ipsius*; u͛sꝰ *uersus*; pꝰtea *postea*; pꝰ *post*.

j. The three-like sign is the ET SIGN (ȝ, U+A76B, also used for *us* in the Latin ending *-ibus*). Do not use the numeral three (3) or the Middle English yogh (ȝ, U+021D):

quibȝ *quibus*; licȝ *licet*; sȝ *sed*.

k. For *-rum* the Unicode RUM ROTUNDA (U+A75D) is like the one in MUFI/Junicode. The one in the *Record Interpreter* and *Statutes* is a late stylized version of this. Use U+A75D and apply OpenType feature cv80 to obtain the correct shape:

aĩaꝛ *animarum*; coꝛpere *corrumpere*; beatoꝛ *beatorum*.

l. For *cum*, *con*, etc. use SMALL LETTER CON (U+A76F):

    9putus *computus*; 9a *contra*; 9nouit *cognouit*.

m. For *per* (or sometimes *par* and other similar sequences), use P WITH STROKE U+A751:

    ꝑsōa *persona*; 9ꝑet *comparet*.

n. For *pro,* use P WITH FLOURISH U+A753:

    ꝓceres *proceres*.

o. For *prae*, *præ*, *pre*, there is no separate character; use a variant of the ZIGZAG (f. above) with **p**:

    ᵽsẽs *praesens*.

p. For **q** with stroke through the descender, there are two Unicode points: U+A757 for a straight stroke, and U+A759 for a diagonal stroke (the *Record Interpreter* appears to use only the former, and neither is listed among the *Statutes* abbreviations):

    ꝗ *quod*; ꝗd *quid*; ꝗbꝫ *quibus*.

q. For *quae*, *que*, use **q** followed by ET (U+A76B) with or without `hlig`: **qꝫ qᵬ**. For the semicolon-like ET sign (**q⁏**), use `cv83[1]`; for the subscripted version (which can also form a ligature via `hlig`), use `cv83[2]`: **q꞉ q꞊**.

r. All of the letters a-z are available in superscript form. Access with the **sups** OpenType feature:

    qᵒs *quos*; cⁱlo *circulo*; capⁱ *capituli*.

The basic Latin letters a–z have anchors that allow you to position combining marks over them (see a. above)

s. Tironian ET sign ⁊ U+204A, cap ⁋ U+2E52. With `cv69[1]` ⁊◌̷; with `cv69[2]` ⁋◌̷.

t. For *est*, use ÷ U+223B HOMOTHETIC. Use of a mathematical sign for this purpose is not ideal, but Unicode offers no better solution.

u. For *tz* (Old French), use ꝣ U+01B6 Z WITH STROKE.

v. For an abbreviation for *Rex*, use ℟ U+211E or ℣ U+211F.

w. At least one edition uses a spacing version of the COMBINING ZIGZAG (**f.** above). Neither Unicode nor MUFI has a matching character: with Junicode, apply `cv67` to the spacing MACRON (U+00AF): ◌¯.

6.4. Other formatting

x. For underdotted text, use Stylistic Set 7, Underdotted. For letters that lack an underdotted form, use U+0323 COMBINING DOT BELOW.

6.5. On the web

Because Junicode is a very large font, web pages should use a subsetted version to speed loading (see Chapter 9, Junicode on the Web, for instructions). The variable version of the font is better for web use than the static fonts, since one variable font file can do the work of many static font files.

All major web browsers (Firefox, Chrome, Safari, Edge) are capable of accessing all of Junicode's characters via OpenType features, use of which promotes accessibility and searchability. When building a web page, study which features will be needed and write them into the appropriate element or class definition of the page's CSS style sheet. For example, if you use the curly form of the zigzag (U+035B) anywhere, you are likely to want it everywhere, and so it should be included in the CSS styling for the <body> element:

```
body {
  font-family: Junicode;
  font-feature-settings: "cv81" 2;
}
```

But the `hlig` feature, if applied to the whole text, will produce many unwanted effects, so it should be included in a class definition to be used in a applied just to the target sequence:

```
.que {
  font-feature-settings: "hlig" on;
}
filio<span class="que">q&#xA76B;</span>
```

The illustrations here use the low-level CSS font-feature-settings property. There are higher-level properties for some OpenType features, but as these are not (yet) universally supported by browsers, and some implementations are buggy, it is best to stick with font-feature-settings for now.

For the purposes addressed in this document, the font-feature-settings for the <body> element should probably be as follows:

```
font-feature-settings: 'cv69' 2, 'cv80' 1, 'cv81' 2;
```

And the following classes should be defined:

```
.super {
  font-feature-settings: 'sups' on, 'cv84' 39;
}

.que {
  font-feature-settings: 'hlig' on;
}

.deleted {
  font-feature-settings: 'ss07' on;
}
```

7. The Enlarge Axis

The character recommendation of the Medieval Unicode Font Initiative (MUFI) includes a class of characters called "Enlarged Minuscules," for representing characters that are lowercase in shape but intermediate between lowercase and uppercase in size: these are often used to begin sentences in medieval manuscripts. MUFI encodes these characters in the Private Use Area, posing accessibility and searchability problems, as explained in the introduction to the "Feature Reference" chapter of this manual.

Junicode provides a solution to these problems via the OpenType feature Stylistic Set 6 (`ss06`, "Enlarged minuscules"). This feature also works in Junicode VF, the variable version of Junicode, which in addition offers a far more flexible way of representing enlarged minuscules—the Enlarge axis.

An "axis" is an aspect of a font that can be varied along a numerical range. A family of traditional fonts like Times New Roman has a weight axis with a font file on either end: Regular and Bold. Other font families have more weights along this axis: for example, Light, Medium, ExtraBold. Most variable fonts also have a weight axis, but all weights are contained in a single file, and users are not restricted to just a few weights, but can select any weight between the extremes.

Because almost every font family has at least two weights, Weight is the most familiar axis. But several other axes are frequently found in both variable fonts and extended font families. Junicode has Weight and Width axes (Width varying from 75 Condensed to 125 Expanded, with 100 Regular in the middle), and the variable font also has an Enlarge axis, which can vary the size of many lowercase letters from that of the font's capitals to that of the lowercase letters: Just as the size of these sentence-initial letters varies widely in manuscripts, so it can vary on web pages and in print (though few applications for producing

ᴅñs ᴅñs ᴅñs ᴅñs ᴅñs

printed documents currently support variable fonts). Notice that the letters are not simply scaled: the proportions change and the weight remains consistent (a lowercase letter scaled up would look too heavy, but a letter scaled via the Enlarged axis will have its original weight at the lower end of the axis and the same weight as a capital at the top).

The Enlarge axis runs from 0 to 100. You can choose any number in that range: to match the effect of ss06 precisely, choose 32. To ensure that the xheight of all letters matches, choose 47 or less: above that value, the xheight of letters like **e** increases at a higher rate than that of letters like **b**.

To use the axis in a web page, declare a CSS class specifying the value for the axis. For example, the second of the examples in the figure above has the axis set to 75:

```
.SentenceInitial {
  font-variation-settings: "wght" 400, "wdth" 100, "ENLA" 75;
}
```

In the text, enclose the first letter of a sentence in a `<span>` with the class "SentenceInitial" (the entity is for insular d):

```
<span class="SentenceInitial">&#xA77A;</span>ñs
```

The result will be an abbreviation that begins with an "Enlarged Minuscule" insular d, precisely matching the look of the second example in the figure above.

These lowercase letters are affected by the Enlarge axis:[1]

a → ᵃ	æ → ᵆ	ɔ → ᵓ	ɣ → ᵞ	đ → ᵭ	e → ᵉ
ɐ → ᵄ	ɛ → ᵋ	ɑɪ → ᵅᵎ	b → ᵇ	ɔ → ᵓ	ę → ᶒ
ɑɑ → ᵅᵅ	ɔɔ → ᵓᵓ	ʌ → ᵚ	c → ᶜ	ð → ᶞ	ɇ → ᶓ
ɐɐ → ᵄᵄ	ɑo → ᵅᵒ	ʌʏ → ᵚʸ	d → ᵈ	ɔ → ᵓ	ę → ᶒ

[1] Note that all composite characters (e.g. **á**, **ü**) based on these are also affected, so that the actual number of affected characters is much greater than shown here.

ɕ → ɕ
f → f
f → f
ɟ → ɟ
g → g
ɡ → ɡ
ɠ → ɠ
h → h
ɦ → ɦ
ɓ → ɓ
ħ → ħ
ɧ → ɧ
i → i
ɪ → ɪ
j → j
ɟ → ɟ
k → k
l → l
ɬ → ɬ
m → m
n → n
ɲ → ɲ
o → o
ɵ → ɵ
ơ → ơ
ǫ → ǫ
ɵ → ɵ
ꝏ → ꝏ
ǫ → ǫ
ø → ø
œ → œ
p → p
ᵽ → ᵽ
ɸ → ɸ
ɋ → ɋ
q → q
ɋ → ɋ
r → r
ɲ → ɲ
ɾ → ɾ
s → s
ɾ → ɾ
t → t
ƫ → ƫ
u → u
v → v
w → w
ƿ → ƿ
x → x
ɣ → ɣ
y → y
z → z
ƀ → ƀ
ƀ → ƀ
ƀ → ƀ
ƀ → ƀ
ƀ → ƀ
ƀ → ƀ

8. Junicode on the Web

If you are using Junicode on a web page, you should prefer the variable fonts (those with "VF" in the family name and filename) to the static fonts. One variable font file can do the work of many traditional font files. For example, the Test of High-Level CSS Properties web page displays Junicode in regular, bold and semicondensed styles. It used to be that your user would have to download three font files, one for each style, but one variable font will display all three.

But you may be thinking, *That font is big*! It's true: even the compressed webfont (.woff2) is nearly a megabyte in size—enough to seriously slow down the loading of a web page.

To solve this problem, you'll need to subset the font—that is, produce a copy that contains only what you need. The subsetted font that downloads with the property test web page is approximately 275k in size—almost thirty percent of the size of the full webfont. It's still a pretty big download, but that's because the page displays a lot of the font's features. If I were displaying, say, a diplomatic transcript of a Latin text, the font would be much smaller.

So the first section of this chapter will talk about how to subset the Junicode font.

8.1. Subsetting Junicode

First the legalities. It is perfectly all right to create a modified version of Junicode via subsetting, compress it into a webfont (almost certainly in woff2 format), and host it on your web server. This is because "Junicode" is not a "Reserved Font Name" (which complicates web use of many fonts licensed under the Open Font License). If you are nevertheless nervous about the legal requirements of the

Open Font License, you can change the font name to something arbitrary with the `--obfuscate-names` option of the pyftsubset program, and you can embed the Open Font License, or a link to it, in your CSS. These steps should settle any ambiguity about whether you are in compliance with the license.

Generating a subsetted version of Junicode should be one of the last tasks you perform before deploying your web page(s). Until then, it is recommended that you work with the unmodified font. After subsetting, review your pages thoroughly to make sure everything is displayed properly. If you have forgotten to include a glyph in your subsetted font, you will see little boxes where characters should be or, perhaps, the correct characters displayed in the wrong typeface. If you have omitted features, you will see default instead of transformed characters.

There are many subsetting programs, some online and very easy to use. But for maximum control (and thus the smallest fonts), you should choose pyftsubset, a part of the fontTools library, which runs under Python 3.7 or higher. This is a command-line tool which takes a long list of arguments; you should create a shell script to run it.

Here is the script used to create the subsetted font for the property test web page mentioned above:

```
#!/bin/zsh
pyftsubset JunicodeVF-Roman.ttf \
--flavor=woff2 --output-file=JunicodeVFsubset.woff2 \
--recommended-glyphs \
--text="jq" --text-file=Junicode-2-feature-test.html \
--layout-features+=liga,ss01,ss02,ss03,ss04,ss05,ss06,ss07,ss08,\
ss10,ss12,ss13,ss14,ss15,ss16,ss17,ss18,ss19,ss20,cv01,cv02,cv05,\
cv06,cv07,cv08,cv09,cv10,dlig,hlig,onum,pnum,pcap,smcp,c2sc,subs,\
sups,zero \
--layout-features-=rlig
```

For those unfamiliar with shell scripts, the first line specifies the shell the script is to run under (in this case the default shell for Mac OS, but `bash` is another possible choice), and the backslashes mean that the command continues on the next line. The rest of the file is a list of arguments passed to pyftsubset. Let's walk through them.

First after the program name comes the name of the unsubsetted, uncompressed font file. After that, the `--flavor` argument tells the program that you want a webfont in woff2 format, and `--output` is the name of the font file you want the program to save.

Having taken care of this preliminary business, we tell pyftsubset what we want the font to contain.

`--recommended-glyphs` includes a few characters that every font should have, according to the OpenType specification—though in fact modern browsers don't care. It's best, however, to conform to the specification, since it's impossible to say with absolute certainty that no program will ever reject the font because of the absence of these few characters.

`--text-file` is the name of a file to treat as a list of characters that *must* be included in the font. In this case I have simply used the html file for the web page for this purpose. If your site contains multiple web pages, your job will be more complicated. You must make sure the text file contains all the characters used on the site—either that or supplement the text file with a `--text` argument (which here adds two lowercase letters that don't appear in the web page—just in case). The text file will contain only encoded characters—you don't have to worry here about unencoded characters produced by OpenType features.

`--layout-features+` tells the program which OpenType features you want to retain in the font. All others, except for the Required Features, are discarded. All of the characters referenced in these features will also be included in the output file, as long as those characters are variants of characters in your text file. For example, the `smcp` (Small Caps) feature has many more small caps than there are letters of the alphabet, but most of them are not included in the subsetted font. The program's parsimony with characters keeps the font file as small as possible. Note that some features are included automatically: `ccmp`, `locl`, `calt`, `liga`, `rlig`, `kern`, `mark`, and `mkmk`.

`--layout-features-` tells the program which OpenType features to omit. Normally, `rlig` (Required Ligtures) is automatically included in fonts by pyftsubset, but as it has no relevance to this web page, it can be omitted.

These are the most useful arguments, but there are many more. Type `pyftsubset --help` for a complete list. Once you have written your script, run it (in Mac OS or Linux you need to make the file executable by typing `chmod +x`

`mysubsetscript` on the command line).

Before you put your subsetted font to work, check it carefully in a program like FontGoggles, which lets you preview the font and test all its OpenType features. If you find errors, revise your script and run it again.

8.2. Junicode and CSS/HTML

This section assumes a basic knowledge of HTML (Hypertext Markup Language, used to construct web pages) and CSS (Cascading Style Sheets, used to format them). If you want to learn about these subjects, the number of good books and online tutorials is so great that it makes no sense to try to list them. Just make sure that the instructional materials you choose are of recent vintage, because the relevant standards are always changing.

In the CSS for your web page, the `@font-face` at-rule for a variable font is a little different from the one for a static font in that the range of possible values for each axis can be declared:

```
@font-face {
    font-family: "Junicode VF";
    src: url("./webfiles/JunicodeVFsubset.woff2");
    font-weight: 300 700;
    font-stretch: 75% 125%;
    font-style: normal;
}
```

These ranges are not strictly necessary, but they will prevent your supplying invalid values for `font-weight` and `font-stretch` (that is, `width`) in other CSS rules.

Once you have declared the font, you can invoke it in setting up classes. For example:

```
body {
  font-family: "Junicode VF";
  font-size: 28px;
  font-weight: normal; /* that is, 400 */
```

```
  font-stretch: 112.5%; /* that is, semiexpanded */
}
h1 {
  font-family: "Junicode VF";
  font-size: 125%;
  font-weight: 600; /* that is, semibold */
  font-stretch: 112.5%; /* that is, semiexpanded */
}
.annotation {
  font-size: 90%;
  font-weight: 300; /* that is, light */
  font-stretch: 87.5%; /* that is, semicondensed */
}
```

These classes should be tested in all browsers. If any fail to display text properly, you can use `font-variation-settings` instead of the high-level `font-weight` and `font-stretch`:

```
body {
  font-family: "Junicode VF";
  font-size: 28px;
  font-variation-settings: "wght" 400, "wdth" 112.5;
}
h1 {
  font-family: "Junicode VF";
  font-size: 125%;
  font-variation-settings: "wght" 600, "wdth" 112.5;
}
.annotation {
  font-size: 90%;
  font-variation-settings: "wght" 300, "wdth" 87.5;
}
```

To accommodate older browsers, you should make a selection of Junicode static fonts, subset them, and include them in your CSS. For example, if you need normal and bold weights of Junicode roman, your `@font-face` at-rule may look like this:

```
@font-face {
    font-family: "Junicode VF";
    src: url("./webfiles/JunicodeVFsubset.woff2");
    font-weight: 300 700;
    font-stretch: 75% 125%;
    font-style: normal;
}
@font-face {
    font-family: "Junicode";
    src: url("./webfiles/Junicode-Regular.woff2");
    font-weight: 400;
    font-style: normal;
}
@font-face {
    font-family: "Junicode";
    src: url("./webfiles/Junicode-Bold.woff2");
    font-weight: 700;
    font-style: normal;
}
```

Now use `@supports` in your CSS rules to determine which font gets downloaded:

```
body {
  font-family: "Junicode", serif;
}
@supports (font-variation-settings: normal) {
  body {
    font-family: "Junicode VF", serif;
  }
}
b {
  font-weight: 700;
}
@supports (font-variation-settings: "wght" 700) {
  b {
    font-variation-settings: "wght" 700;
  }
```

```
}
```

The variable version of Junicode will be downloaded only if the browser supports it, and the static version will be downloaded only if needed.

9. Junicode and TEX

9.1. Loading the packages

There are packages for both Junicode (the static font) and Junicode VF (the variable font) in CTAN, the TEX repository, and also in the TEX Live distribution (run `tlmgr` to get them). Both static and variable versions have a convenient script for loading and managing the font: use `\usepackage{junicode}` for the static font and `\usepackage{junicodevf}` for the variable font (which requires LuaTEX). These commands accept several options commonly used in font packages:

light The weight of the type for the main text is light instead of regular.

medium The weight of the type for the main text is medium, somewhat heavier than regular.

semibold The weight of bold type is somewhat lighter than the usual bold. This may be a good choice if you have selected the `light` option.

condensed The width of the type is narrow. Note that in the static font, bold type cannot be condensed: when this option is selected, any bold type in the text will have normal width.

semicondensed The width of the type is wider than condensed but narrower than the default. In the static font, bold type cannot be semicondensed.

expanded The width of the type is about 125%. Note that in the static font, light type cannot be expanded: using both the `light` and the `expanded` options will produce an error.

semiexpanded The width of the type is wider than the default but narrower than expanded. In the static font, light type cannot be semiexpanded.

proportional Numbers in the document will be proportionally spaced. This is the default.

tabular Numbers will be tabular (or monospaced).

oldstyle Numbers will be old-style, harmonizing with lowercase letters.

lining Numbers will be lining, harmonizing with uppercase letters.

renderer Choose a renderer: this can be any of those accepted by fontspec (the default is OpenType). This should not generally be changed.

With the variable font, terms like "light" and "semibold" (and, for that matter, "regular") do not denote a fixed shape the way they do with the static font, but rather a range of weights and widths that vary with the point size. You can see these variations if we scale a line of footnote text and a line of header text to the same `\large` size:

> Here is some sample text for footnotes (about 8pt).
> Here is some sample text for headers (18pt or larger).

The glyphs for footnote text are heavier and wider than those for headers, recalling the way punchcutters in the era of metal type often designed small sizes to be relatively thicker and wider than main text or titles. This promoted legibility at small sizes and also evenness of color on pages with diverse text blocks.

Junicode VF provides vastly more flexibility than static Junicode, starting with two options that go with the weight and width options listed above:

weightadjustment Adjusts the weight of the type by adding this number. For example, if you choose `medium` for your document (weight averaging about 500) and `bold` (weight around 700), and also include the option `weightadjustment=-25`, then the weights of medium and bold text will be lightened by 25 (to 475 and 675).

widthadjustment Adjusts the width of the type by adding this number. For example, if you choose semicondensed for your document (width averaging 87.5), and you also include the option `widthadjustment=5`, then the average width will be 92.5, between semicondensed and regular.

9.2. Advanced Options

If you are using the variable font and the basic options listed above don't yield the results you want, the options listed in this section allow you to choose from an effectively infinite number of styles. Do this by supplying custom axis coordinates for one or more of the four basic styles of the main text (Regular, Italic, Bold, BoldItalic) via package options called SizeFeatures. For example, here are the SizeFeatures for this document:

```
\usepackage[
    MainRegularSizeFeatures={
        {size=8.6,wght=550,wdth=120},
        {size=10.99,wght=475,wdth=115},
        {size=21.59,wght=400,wdth=112.5},
        {size=21.59,wght=351,wdth=100}
    },
    MainItalicSizeFeatures={
        {size=8.6,wght=550,wdth=118},
        {size=10.99,wght=475,wdth=114},
        {size=21.59,wght=450,wdth=111},
        {size=21.59,wght=372,wdth=98}
    },
    MainBoldSizeFeatures={
        {size=8.6,wght=700,wdth=120},
        {size=10.99,wght=700,wdth=115},
        {size=21.59,wght=650,wdth=112.5},
        {size=21.59,wght=600,wdth=100}
    },
    MainBoldItalicSizeFeatures={
        {size=8.6,wght=700,wdth=118},
        {size=10.99,wght=700,wdth=114},
        {size=21.59,wght=650,wdth=111},
        {size=21.59,wght=600,wdth=98}
    }
```

```
]{junicodevf}
```

These options consist of lists of associative arrays, each prescribing axis coordinates for a range of sizes. In these arrays, the `size` key is mandatory: any array without one is ignored. The arrays should be in order of point size. The first array prescribes axis coordinates for all sizes up to `size`, the last array for all sizes greater than `size`, and any intermediate arrays a range from the previous to the current `size`.[1] So the ranges covered in each list above are `-8.6`, `8.6-10.99`, `10.99-21.59`, and `21.59-`.[2]

The keys other than `size` are the four-letter tags for the font's axes: `wght` (Weight), `wdth` (Width), and `ENLA` (Enlarge).[3] When a key is omitted, the default value for that axis is used. It is up to the user to make sure the values given for each axis are valid—the package does no checking (though fontspec will do a good bit of checking for you). When SizeFeatures are given in this way, they override any other options that set or change axis coordinates (e.g. weightadjustment).

The SizeFeatures options can only set axis coordinates; with the Features options you can set OpenType features for the main text or for the four main styles individually.

For example, if you want your document to use the conventions observed by early English typesetters for the distribution of **s** and **f**, load the package this way:

```
\usepackage[MainFeatures={
    Language=English,
    StylisticSet=8
}]{junicodevf}
```

If you want to use these conventions only for text in the regular style, use MainRegularFeatures instead of MainFeatures. For the other styles, use MainItalicFeatures, MainBoldFeatures, and MainBoldItalicFeatures. All of the features you pass via these

[1] If you want only one size array, make `size` improbably low (e.g. 5) and place a comma after the closing brace of the array.

[2] Any modification of the default text size (e.g. in the `\documentclass` command) can affect the size definitions in these arrays, with the result that (for example) `10` no longer means exactly "ten points." You may have to experiment to get these numbers right.

[3] By convention, tags for axes defined in the OpenType standard are lowercase; custom axes are uppercase. Junicode's `ENLA` is a custom axis.

options must be valid for fontspec: in fact, they are passed straight through to fontspec.

9.3. Selecting Alternate Styles

In addition to the document's main font, you can choose from up to fifty predefined styles—thirty-eight if you are using the static font (in the list below, styles available only to variable font users are red). The commands for shifting to these styles are as follows (of the italic styles, only the base "jItalic" is listed; append "Italic" to any of the others, except "jRegular"):

\jRegular	\jSmExpLight	\jSmCondSmbold
\jItalic	\jExpLight	\jSmExpSmbold
\jCond	\jMedium	\jExpSmbold
\jSmCond	\jCondMedium	\jBold
\jSmExp	\jSmCondMedium	\jCondBold
\jExp	\jSmExpMedium	\jSmCondBold
\jLight	\jExpMedium	\jSmExpBold
\jCondLight	\jSmbold	\jExpBold
\jSmCondLight	\jCondSmbold	

These commands will be self-explanatory if you bear in mind Junicode's abbreviations for style names: Cond=Condensed, Exp=Expanded, Sm=Semi.[4] Use them to shift temporarily to a style other than that of the main text. For example, to shift to the Condensed Light style for a short phrase, use this code:

```
{\jCondLight a short phrase}.
```

The result: a short phrase.

To add features to any of these styles (variable font only), use the style name (without the prefixed "j" and with Features appended) as a package option. To change the size features for the style, do the same, but with SizeFeatures instead of Features appended:

[4] The purpose of these abbreviations is to keep font names under the character-limit imposed by some systems.

```
\usepackage[
    CondLightFeatures={
        Language=English,
        StylisticSet=2
    },
    CondLightSizeFeatures={{size=5,wght=325,wdth=80},}
]{junicodevf}
```

This will shift text in the Condensed Light style from default to insular letter-shapes and slightly increase the weight and width of all glyphs in that style. Here the SizeFeatures section is very simple (as in the package file itself), but you can have as many size ranges as you want, just as you can for the main font.

9.4. The Enlarge Axis

The variable package defines four different styles for Junicode VF's Enlarge axis, in four sizes:

Not Enlarged	abc	*abc*
\EnlargedOne	abc	*abc*
\EnlargedTwo	abc	*abc*
\EnlargedThree	abc	*abc*
\EnlargedFour	abc	*abc*

You can produce an italic version of the enlarged minuscule by appending "Italic" to the style name. You can also customize these styles with SizeFeatures:

```
\usepackage[
    EnlargedThreeSizeFeatures={{size=5,ENLA=85},}
]{junicodevf}
```

This example will set all axes except for ENLA to their default coordinates. You can, of course, define other axes, and, as with Junicode's other SizeFeatures options, as many size arrays as you like. Features options are not available for the Enlarged styles.

9.5. Other Commands

The font packages' other commands (listed in the following table) are offered as conveniences, being shorter and more mnemonic than the fontspec commands they invoke (though of course all fontspec commands remain available). Each of these commands also has a corresponding "text" command that works like `\textit{}`—that is, it takes as its sole argument the text to which the command will be applied. Each "text" command consists of the main command with "text" prefixed—for example, `\textInsularLetterForms{}` corresponding to `\InsularLetterForms`. For a fuller account of the OpenType features applied by these commands, see Chapter 4, Feature Reference.

\AltThornEth	Applies sso1, Alternate thorn and eth.
\InsularLetterForms	Applies sso2, Insular letter-forms.
\IPAAlternates	Applies sso3, IPA alternates.
\HighOverline	Applies sso4, High Overline.
\MediumHighOverline	Applies sso5, Medium-high Overline.
\EnlargedMinuscules	Applies sso6, Enlarged minuscules.
\Underdotted	Applies sso7, Underdotted.
\ContextualLongS	Applies sso8, Contextual long s.
\AlternateFigures	Applies sso9, Alternate Figures.
\EntitiesAndTags	Applies ss10, Entities and Tags.
\EarlyEnglishFuthorc	Applies ss12, Early English Futhorc.
\ElderFuthark	Applies ss13, Elder Futhark.
\YoungerFuthark	Applies ss14, Younger Futhark.
\LongBranchToShortTwig	Applies ss15, Long Branch to Short Twig.
\ContextualRRotunda	Applies ss16, Contextual r rotunda.
\RareDigraphs	Applies ss17, Rare Digraphs.
\OldStylePunctuation	Applies ss18, Old-style Punctuation.

\LatinToGothic	Applies ss19, Latin to Gothic.
\LowDiacritics	Applies ss20, Low Diacritics.
\jcv, \textcv	Applies any Character Variant feature (see below).

The syntax of \jcv is `\jcv[num]{num}`, where the second (required) argument is the number of the Character Variant feature, and the first (optional) argument is an index into the variants provided by that feature (starting with zero, the default). \textcv takes an additional required argument (`\textcv[num]{num}{text}`—the text to which the feature should be applied.

Character Variant features can also be selected by means of commands consisting of the prefix `\jcv` plus any letter of the basic Latin alphabet (e.g. `\jcvA`, `\jcvz`), or any of the mnemonics below. For example, a feature for lowercase **a** can be expressed as `\textcv[2]{\jcva}{a}`, yielding **ɑ**.

\jcvaa
\jcvAE
\jcvae
\jcvAO
\jcvao
\jcvAogonek
\jcvaogonek
\jcvASCIItilde
\jcvasterisk
\jcvav
\jcvbrevebelow
\jcvcombiningdieresis
\jcvcombiningdoublemacron
\jcvcombininginsulard
\jcvcombiningopena
\jcvcombiningoverline
\jcvcombiningrrotunda
\jcvcombiningzigzag
\jcvcomma
\jcvcurrency
\jcvdbar
\jcvdcroat
\jcvEng
\jcvEogonek
\jcvetabbrev
\jcvexclam
\jcvflorin
\jcvGermanpenny
\jcvglottal
\jcvlb
\jcvlhighstroke
\jcvmacron
\jcvmiddot
\jcvoPolish
\jcvounce
\jcvperiod
\jcvpunctuselevatus
\jcvquestion
\jcvrum
\jcvsemicolon
\jcvslash
\jcvspacingusabbrev
\jcvspacingzigzag
\jcvsterling
\jcvthorncrossed
\jcvTironianEt
\jcvYogh

10. Encoded Glyphs in Junicode

The following table lists all the encoded glyphs in Junicode Roman. The font also contains more than 2,000 *unencoded* glyphs, accessible via OpenType features. For a comprehensive list of these features, see Chapter 4, Feature Reference.

Code points for which Junicode has no glyphs are represented in the table by blue bullets (the actual bullet at U+2022 is black). Many of Junicode's glyphs (e.g. spaces, formatting marks) are invisible: these are represented by blanks in the table. A few glyphs are too large for their table cells, and these spill out on one or more sides.

Table 10.1: Encoded Glyphs in Junicode

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Basic Latin																
U+0000 - 000F	•	•	•	•	•	•	•	•	•	•	•	•	•		•	•
U+0020 - 002F		!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
U+0030 - 003F	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
U+0040 - 004F	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
U+0050 - 005F	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
U+0060 - 006F	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
U+0070 - 007F	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	•

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Latin-1 Supplement																
U+00A0 - 00AF		¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
U+00B0 - 00BF	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
U+00C0 - 00CF	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
U+00D0 - 00DF	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
U+00E0 - 00EF	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
U+00F0 - 00FF	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ
Latin Extended-A																
U+0100 - 010F	Ā	ā	Ă	ă	Ą	ą	Ć	ć	Ĉ	ĉ	Ċ	ċ	Č	č	Ď	ď
U+0110 - 011F	Đ	đ	Ē	ē	Ĕ	ĕ	Ė	ė	Ę	ę	Ě	ě	Ĝ	ĝ	Ğ	ğ
U+0120 - 012F	Ġ	ġ	Ģ	ģ	Ĥ	ĥ	Ħ	ħ	Ĩ	ĩ	Ī	ī	Ĭ	ĭ	Į	į
U+0130 - 013F	İ	ı	IJ	ij	Ĵ	ĵ	Ķ	ķ	ĸ	Ĺ	ĺ	Ļ	ļ	Ľ	ľ	Ŀ
U+0140 - 014F	ŀ	Ł	ł	Ń	ń	Ņ	ņ	Ň	ň	ŉ	Ŋ	ŋ	Ō	ō	Ŏ	ŏ
U+0150 - 015F	Ő	ő	Œ	œ	Ŕ	ŕ	Ŗ	ŗ	Ř	ř	Ś	ś	Ŝ	ŝ	Ş	ş
U+0160 - 016F	Š	š	Ţ	ţ	Ť	ť	Ŧ	ŧ	Ũ	ũ	Ū	ū	Ŭ	ŭ	Ů	ů
U+0170 - 017F	Ű	ű	Ų	ų	Ŵ	ŵ	Ŷ	ŷ	Ÿ	Ź	ź	Ż	ż	Ž	ž	ſ
Latin Extended-B																
U+0180 - 018F	ƀ	Ɓ	Ƃ	ƃ	Ƅ	ƅ	Ɔ	Ƈ	ƈ	Ɖ	Ɗ	Ƌ	ƌ	ƍ	Ǝ	Ə
U+0190 - 019F	Ɛ	Ƒ	ƒ	Ɠ	Ɣ	ƕ	Ɩ	Ɨ	Ƙ	ƙ	ƚ	ƛ	Ɯ	Ɲ	ƞ	Ɵ
U+01A0 - 01AF	Ơ	ơ	Ƣ	ƣ	Ƥ	ƥ	Ʀ	Ƨ	ƨ	Σ	ƪ	ƫ	Ƭ	ƭ	Ʈ	Ư
U+01B0 - 01BF	ư	Ʊ	Ʋ	Ƴ	ƴ	Ƶ	ƶ	Ʒ	Ƹ	ƹ	ƺ	ƻ	Ƽ	ƽ	ƾ	ƿ
U+01C0 - 01CF	ǀ	ǁ	ǂ	!	DŽ	Dž	dž	LJ	Lj	lj	NJ	Nj	nj	Ǎ	ǎ	Ǐ
U+01D0 - 01DF	ǐ	Ǒ	ǒ	Ǔ	ǔ	Ǖ	ǖ	Ǘ	ǘ	Ǚ	ǚ	Ǜ	ǜ	ə	Ǟ	ǟ

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+01E0 - 01EF	Ǡ	ǡ	Ǣ	ǣ	Ǥ	ǥ	Ǧ	ǧ	Ǩ	ǩ	Ǫ	ǫ	Ǭ	ǭ	Ǯ	ǯ
U+01F0 - 01FF	ǰ	Ǳ	ǲ	ǳ	Ǵ	ǵ	Ƕ	Ƿ	Ǹ	ǹ	Ǻ	ǻ	Ǽ	ǽ	Ǿ	ǿ
U+0200 - 020F	Ȁ	ȁ	Ȃ	ȃ	Ȅ	ȅ	Ȇ	ȇ	Ȉ	ȉ	Ȋ	ȋ	Ȍ	ȍ	Ȏ	ȏ
U+0210 - 021F	Ȑ	ȑ	Ȓ	ȓ	Ȕ	ȕ	Ȗ	ȗ	Ș	ș	Ț	ț	Ȝ	ȝ	Ȟ	ȟ
U+0220 - 022F	Ƞ	ȡ	Ȣ	ȣ	Ȥ	ȥ	Ȧ	ȧ	Ȩ	ȩ	Ȫ	ȫ	Ȭ	ȭ	Ȯ	ȯ
U+0230 - 023F	Ȱ	ȱ	Ȳ	ȳ	·	·	·	ȷ	ȸ	ȹ	Ⱥ	·	·	·	·	·
U+0240 - 024F	·	Ɂ	ɂ	Ƀ	·	·	Ɇ	ɇ	Ɉ	ɉ	·	·	·	·	·	·

IPA Extensions

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+0250 - 025F	ɐ	ɑ	ɒ	ɓ	ɔ	ɕ	ɖ	ɗ	ɘ	ə	ɚ	ɛ	ɜ	ɝ	ɞ	ɟ
U+0260 - 026F	ɠ	ɡ	ɢ	ɣ	ɤ	ɥ	ɦ	ɧ	ɨ	ɩ	ɪ	ɫ	ɬ	ɭ	ɮ	ɯ
U+0270 - 027F	ɰ	ɱ	ɲ	ɳ	ɴ	ɵ	ɶ	ɷ	ɸ	ɹ	ɺ	ɻ	ɼ	ɽ	ɾ	ɿ
U+0280 - 028F	ʀ	ʁ	ʂ	ʃ	ʄ	ʅ	ʆ	ʇ	ʈ	ʉ	ʊ	ʋ	ʌ	ʍ	ʎ	ʏ
U+0290 - 029F	ʐ	ʑ	ʒ	ʓ	ʔ	ʕ	ʖ	ʗ	ʘ	ʙ	ʚ	ʛ	ʜ	ʝ	ʞ	ʟ
U+02A0 - 02AF	ʠ	ʡ	ʢ	ʣ	ʤ	ʥ	ʦ	ʧ	ʨ	ʩ	ʪ	ʫ	ʬ	ʭ	ʮ	ʯ

Spacing Modifier Letters

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+02B0 - 02BF	ʰ	ʱ	ʲ	ʳ	ʴ	ʵ	ʶ	ʷ	ʸ	ʹ	ʺ	ʻ	ʼ	ʽ	ʾ	ʿ
U+02C0 - 02CF	ˀ	ˁ	˂	˃	˄	˅	ˆ	ˇ	ˈ	ˉ	ˊ	ˋ	ˌ	ˍ	ˎ	ˏ
U+02D0 - 02DF	ː	ˑ	˒	˓	˔	˕	˖	˗	˘	˙	˚	˛	˜	˝	˞	˟
U+02E0 - 02EF	ˠ	ˡ	ˢ	ˣ	ˤ	˥	˦	˧	˨	˩	˪	˫	ˬ	˭	ˮ	˯
U+02F0 - 02FF	˰	˱	˲	˳	˴	˵	˶	˷	˸	˹	˺	˻	˼	˽	˾	˿

Combining Diacritical Marks

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+0300 - 030F	◌̀	◌́	◌̂	◌̃	◌̄	◌̅	◌̆	◌̇	◌̈	◌̉	◌̊	◌̋	◌̌	◌̍	◌̎	◌̏

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+0310 - 031F	◌̐	◌̑	◌̒	◌̓	◌̔	◌̕	◌̖	◌̗	◌̘	◌̙	◌̚	◌̛	◌̜	◌̝	◌̞	◌̟
U+0320 - 032F	◌̠	◌̡	◌̢	◌̣	◌̤	◌̥	◌̦	◌̧	◌̨	◌̩	◌̪	◌̫	◌̬	◌̭	◌̮	◌̯
U+0330 - 033F	◌̰	◌̱	◌̲	◌̳	◌̴	◌̵	◌̶	◌̷	◌̸	◌̹	◌̺	◌̻	◌̼	◌̽	◌̾	◌̿
U+0340 - 034F	◌̀	◌́	◌̂	◌̓	◌̄	◌̅	◌̆	◌̇	◌̈	◌̉	◌̊	◌̋	◌̌	◌̍	◌̎	◌̏
U+0350 - 035F	◌̐	◌̑	◌̒	◌̓	◌̔	◌̕	◌̖	◌̗	◌̘	◌̙	◌̚	◌̛	◌̜	◌̝	◌̞	◌̟
U+0360 - 036F	◌͠	◌͡	◌͢	◌ͣ	◌ͤ	◌ͥ	◌ͦ	◌ͧ	◌ͨ	◌ͩ	◌ͪ	◌ͫ	◌ͬ	◌ͭ	◌ͮ	◌ͯ

Greek and Coptic

U+0370 - 037F	◌̀	◌́	◌̂	◌̓	◌̔	◌̕	◌̖	◌̗	◌̘	◌̙	◌̚	◌̛	◌̜	◌̝	◌̞	◌̟
U+0380 - 038F	◌̠	◌̡	◌̢	◌̣	◌̤	◌̥	◌̦	◌̧	◌̨	◌̩	◌̪	◌̫	◌̬	◌̭	◌̮	◌̯
U+0390 - 039F	◌̰	◌̱	◌̲	◌̳	◌̴	◌̵	◌̶	◌̷	◌̸	◌̹	◌̺	◌̻	◌̼	◌̽	◌̾	◌̿
U+03A0 - 03AF	◌̀	◌́	◌̂	◌̓	◌̄	◌̅	◌̆	◌̇	◌̈	◌̉	◌̊	◌̋	◌̌	◌̍	◌̎	◌̏
U+03B0 - 03BF	◌̐	◌̑	◌̒	◌̓	◌̔	◌̕	◌̖	◌̗	◌̘	◌̙	◌̚	◌̛	◌̜	◌̝	◌̞	◌̟
U+03C0 - 03CF	◌̠	◌̡	◌̢	◌̣	◌̤	◌̥	◌̦	◌̧	◌̨	◌̩	◌̪	◌̫	◌̬	◌̭	◌̮	◌̯
U+03D0 - 03DF	◌̰	◌̱	◌̲	◌̳	◌̴	◌̵	◌̶	◌̷	◌̸	◌̹	◌̺	◌̻	◌̼	◌̽	◌̾	◌̿
U+03E0 - 03EF	◌̀	◌́	◌̂	◌̓	◌̄	◌̅	◌̆	◌̇	◌̈	◌̉	◌̊	◌̋	◌̌	◌̍	◌̎	◌̏
U+03F0 - 03FF	◌̐	◌̑	◌̒	◌̓	◌̔	◌̕	◌̖	◌̗	◌̘	◌̙	◌̚	◌̛	◌̜	◌̝	◌̞	◌̟

Cyrillic

U+0420 - 042F	◌̠	◌̡	◌̢	◌̣	◌̤	◌̥	◌̦	◌̧	◌̨	◌̩	◌̪	◌̫	◌̬	◌̭	◌̮	◌̯
U+0440 - 044F	◌̰	◌̱	◌̲	◌̳	◌̴	◌̵	◌̶	◌̷	◌̸	◌̹	◌̺	◌̻	◌̼	◌̽	◌̾	◌̿

Georgian

U+10F0 - 10FF	◌̀	◌́	◌̂	◌̓	◌̄	◌̅	◌̆	◌̇	◌̈	◌̉	◌̊	◌̋	◌̌	◌̍	◌̎	◌̏

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Runic																
U+16A0 - 16AF	ᚠ	ᚡ	ᚢ	ᚣ	ᚤ	ᚥ	ᚦ	ᚧ	ᚨ	ᚩ	ᚪ	ᚫ	ᚬ	ᚭ	ᚮ	ᚯ
U+16B0 - 16BF	ᚰ	ᚱ	ᚲ	ᚳ	ᚴ	ᚵ	ᚶ	ᚷ	ᚸ	ᚹ	ᚺ	ᚻ	ᚼ	ᚽ	ᚾ	ᚿ
U+16C0 - 16CF	ᛀ	ᛁ	ᛂ	ᛃ	ᛄ	ᛅ	ᛆ	ᛇ	ᛈ	ᛉ	ᛊ	ᛋ	ᛌ	ᛍ	ᛎ	ᛏ
U+16D0 - 16DF	ᛐ	ᛑ	ᛒ	ᛓ	ᛔ	ᛕ	ᛖ	ᛗ	ᛘ	ᛙ	ᛚ	ᛛ	ᛜ	ᛝ	ᛞ	ᛟ
U+16E0 - 16EF	ᛠ	ᛡ	ᛢ	ᛣ	ᛤ	ᛥ	ᛦ	ᛧ	ᛨ	ᛩ	ᛪ	᛫	᛬	᛭	ᛮ	ᛯ
U+16F0 - 16FF	ᛰ	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
Buginese																
U+1AC0 - 1ACF	·	·	·	·	·	·	·	·	·	·	·	᩻	ᩳ	᩵	᩶	·
Phonetic Extensions																
U+1D00 - 1D0F	A	Æ	ᴂ	ᴃ	C	D	Ð	E	Ɜ	Ị	J	K	Ł	M	И	O
U+1D10 - 1D1F	Ɔ	ᴏ	ᴐ	ᴓ	ᴔ	ᴕ	ᴖ	ᴗ	P	Я	ᴙ	T	U	ᴝ	ᴞ	ᴟ
U+1D20 - 1D2F	V	ᴡ	Z	ᴣ	ᴤ	ᴥ	Γ	Λ	Π	Ρ	Ψ	Л	A	Æ	B	ʙ
U+1D30 - 1D3F	D	E	Ǝ	G	H	I	J	K	L	M	N	И	O	ᴽ	P	R
U+1D40 - 1D4F	T	U	ᴡ	a	ɐ	ɑ	ᴂ	b	d	e	ə	ɛ	ɜ	g	ị	k
U+1D50 - 1D5F	m	ŋ	o	ɔ	ᴖ	ᴗ	p	t	u	ᴝ	ɯ	v	ᴥ	β	γ	δ
U+1D60 - 1D6F	φ	χ	i	r	u	v	β	γ	ρ	φ	χ	ᴭ	ᴆ	ᴅ	ᴇ	ᴈ
U+1D70 - 1D7F	ᴉ	·	·	·	·	·	·	·	·	ᴝ	·	ᴚ	·	·	·	ᴥ
Phonetic Extensions Supplement																
U+1D90 - 1D9F	·	·	·	·	·	·	·	·	·	·	·	·	ᶜ	·	·	·
U+1DA0 - 1DAF	ᶠ	·	·	·	·	ᶥ	·	·	·	·	·	·	·	·	·	·
U+1DB0 - 1DBF	·	·	·	·	·	·	·	·	·	·	·	ᶻ	·	·	·	ᶿ

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Combining Diacritical Marks Supplement																
U+1DC0 - 1DCF	◌᷀	◌᷁	◌᷂	◌᷃	◌᷄	◌᷅	◌᷆	◌᷇	◌᷈	◌᷉	◌᷊	◌᷋	◌᷌	◌᷍	◌᷎	◌᷏
U+1DD0 - 1DDF	◌᷐	◌᷑	◌᷒	◌ᷓ	◌ᷔ	◌ᷕ	◌ᷖ	◌ᷗ	◌ᷘ	◌ᷙ	◌ᷚ	◌ᷛ	◌ᷜ	◌ᷝ	◌ᷞ	◌ᷟ
U+1DE0 - 1DEF	◌ᷠ	◌ᷡ	◌ᷢ	◌ᷣ	◌ᷤ	◌ᷥ	◌ᷦ	◌ᷧ	◌ᷨ	◌ᷩ	◌ᷪ	◌ᷫ	◌ᷬ	◌ᷭ	◌ᷮ	◌ᷯ
U+1DF0 - 1DFF	◌ᷰ	◌ᷱ	◌ᷲ	◌ᷳ	◌ᷴ	◌᷵	◌᷶	◌᷷	◌᷸	◌᷹	◌᷺	◌᷻	◌᷼	◌᷽	◌᷾	◌᷿
Latin Extended Additional																
U+1E00 - 1E0F	Ḁ	ḁ	Ḃ	ḃ	Ḅ	ḅ	Ḇ	ḇ	Ḉ	ḉ	Ḋ	ḋ	Ḍ	ḍ	Ḏ	ḏ
U+1E10 - 1E1F	Ḑ	ḑ	Ḓ	ḓ	Ḕ	ḕ	Ḗ	ḗ	Ḙ	ḙ	Ḛ	ḛ	Ḝ	ḝ	Ḟ	ḟ
U+1E20 - 1E2F	Ḡ	ḡ	Ḣ	ḣ	Ḥ	ḥ	Ḧ	ḧ	Ḩ	ḩ	Ḫ	ḫ	Ḭ	ḭ	Ḯ	ḯ
U+1E30 - 1E3F	Ḱ	ḱ	Ḳ	ḳ	Ḵ	ḵ	Ḷ	ḷ	Ḹ	Ḻ	Ḽ	ḽ	Ḿ	ḿ		
U+1E40 - 1E4F	Ṁ	ṁ	Ṃ	ṃ	Ṅ	ṅ	Ṇ	ṇ	Ṉ	ṉ	Ṋ	ṋ	Ṍ	ṍ	Ṏ	ṏ
U+1E50 - 1E5F	Ṑ	ṑ	Ṓ	ṓ	Ṕ	ṕ	Ṗ	ṗ	Ṙ	ṙ	Ṛ	ṛ	Ṝ	ṝ	Ṟ	ṟ
U+1E60 - 1E6F	Ṡ	ṡ	Ṣ	ṣ	Ṥ	ṥ	Ṧ	ṧ	Ṩ	ṩ	Ṫ	ṫ	Ṭ	ṭ	Ṯ	ṯ
U+1E70 - 1E7F	Ṱ	ṱ	Ṳ	ṳ	Ṵ	ṵ	Ṷ	ṷ	Ṹ	ṹ	Ṻ	ṻ	Ṽ	ṽ	Ṿ	ṿ
U+1E80 - 1E8F	Ẁ	ẁ	Ẃ	ẃ	Ẅ	ẅ	Ẇ	ẇ	Ẉ	ẉ	Ẋ	ẋ	Ẍ	ẍ	Ẏ	ẏ
U+1E90 - 1E9F	Ẑ	ẑ	Ẓ	ẓ	Ẕ	ẕ	ẖ	ẗ	ẘ	ẙ	ẚ	ẛ	ẜ	ẝ	ẞ	ẟ
U+1EA0 - 1EAF	Ạ	ạ	Ả	ả	Ấ	ấ	Ầ	ầ	Ẩ	ẩ	Ẫ	ẫ	Ậ	ậ	Ắ	ắ
U+1EB0 - 1EBF	Ằ	ằ	Ẳ	ẳ	Ẵ	ẵ	Ặ	ặ	Ẹ	ẹ	Ẻ	ẻ	Ẽ	ẽ	Ế	ế
U+1EC0 - 1ECF	Ề	ề	Ể	ể	Ễ	ễ	Ệ	ệ	Ỉ	ỉ	Ị	ị	Ọ	ọ	Ỏ	ỏ
U+1ED0 - 1EDF	Ố	ố	Ồ	ồ	Ổ	ổ	Ỗ	ỗ	Ộ	ộ	Ớ	ớ	Ờ	ờ	Ở	ở
U+1EE0 - 1EEF	Ỡ	ỡ	Ợ	ợ	Ụ	ụ	Ủ	ủ	Ứ	ứ	Ừ	ừ	Ử	ử	Ữ	ữ
U+1EF0 - 1EFF	Ự	ự	Ỳ	ỳ	Ỵ	ỵ	Ỷ	ỷ	Ỹ	ỹ	Ỻ	ỻ	Ỽ	ỽ	Ỿ	ỿ

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Greek Extended																
U+1F00 - 1F0F	ἀ	ἁ	ἂ	ἃ	ἄ	ἅ	ἆ	ἇ	Ἀ	Ἁ	Ἂ	Ἃ	Ἄ	Ἅ	Ἆ	Ἇ
U+1F10 - 1F1F	ἐ	ἑ	ἒ	ἓ	ἔ	ἕ	.	.	Ἐ	Ἑ	Ἒ	Ἓ	Ἔ	Ἕ	.	.
U+1F20 - 1F2F	ἠ	ἡ	ἢ	ἣ	ἤ	ἥ	ἦ	ἧ	Ἠ	Ἡ	Ἢ	Ἣ	Ἤ	Ἥ	Ἦ	Ἧ
U+1F30 - 1F3F	ἰ	ἱ	ἲ	ἳ	ἴ	ἵ	ἶ	ἷ	Ἰ	Ἱ	Ἲ	Ἳ	Ἴ	Ἵ	Ἶ	Ἷ
U+1F40 - 1F4F	ὀ	ὁ	ὂ	ὃ	ὄ	ὅ	.	.	Ὀ	Ὁ	Ὂ	Ὃ	Ὄ	Ὅ	.	.
U+1F50 - 1F5F	ὐ	ὑ	ὒ	ὓ	ὔ	ὕ	ὖ	ὗ	.	Ὑ	.	Ὓ	.	Ὕ	.	Ὗ
U+1F60 - 1F6F	ὠ	ὡ	ὢ	ὣ	ὤ	ὥ	ὦ	ὧ	Ὠ	Ὡ	Ὢ	Ὣ	Ὤ	Ὥ	Ὦ	Ὧ
U+1F70 - 1F7F	ὰ	ά	ὲ	έ	ὴ	ή	ὶ	ί	ὸ	ό	ὺ	ύ	ὼ	ώ	.	.
U+1F80 - 1F8F	ᾀ	ᾁ	ᾂ	ᾃ	ᾄ	ᾅ	ᾆ	ᾇ	ᾈ	ᾉ	ᾊ	ᾋ	ᾌ	ᾍ	ᾎ	ᾏ
U+1F90 - 1F9F	ᾐ	ᾑ	ᾒ	ᾓ	ᾔ	ᾕ	ᾖ	ᾗ	ᾘ	ᾙ	ᾚ	ᾛ	ᾜ	ᾝ	ᾞ	ᾟ
U+1FA0 - 1FAF	ᾠ	ᾡ	ᾢ	ᾣ	ᾤ	ᾥ	ᾦ	ᾧ	ᾨ	ᾩ	ᾪ	ᾫ	ᾬ	ᾭ	ᾮ	ᾯ
U+1FB0 - 1FBF	ᾰ	ᾱ	ᾲ	ᾳ	ᾴ	.	ᾶ	ᾷ	Ᾰ	Ᾱ	Ὰ	Ά	ᾼ	᾽	ι	᾿
U+1FC0 - 1FCF	῀	῁	ῂ	ῃ	ῄ	.	ῆ	ῇ	Ὲ	Έ	Ὴ	Ή	ῌ	῍	῎	῏
U+1FD0 - 1FDF	ῐ	ῑ	ῒ	ΐ	.	.	ῖ	ῗ	Ῐ	Ῑ	Ὶ	Ί	.	῝	῞	῟
U+1FE0 - 1FEF	ῠ	ῡ	ῢ	ΰ	ῤ	ῥ	ῦ	ῧ	Ῠ	Ῡ	Ὺ	Ύ	Ῥ	῭	΅	`
U+1FF0 - 1FFF	.	.	ῲ	ῳ	ῴ	.	ῶ	ῷ	Ὸ	Ό	Ὼ	Ώ	ῼ	´	῾	.
General Punctuation																
U+2000 - 200F															.	.
U+2010 - 201F	‐	‑	‒	–	—	―	‖	‗	‘	’	‚	‛	“	”	„	‟
U+2020 - 202F	†	‡	•	‣	‧	..	…	‧	.	.	.	.	.	.	.	
U+2030 - 203F	‰	‱	′	″	‴	‵	‶	‷	‸	‹	›	※	‼	‽	‾	‿
U+2040 - 204F	⁀	⁁	⁂	⁃	⁄	⁅	⁆	⁇	⁈	⁉	⁊	⁋	⁌	⁍	⁎	⁏

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+2050 - 205F	⁀	⁑	⁒	~	·	·	⁖	⁗	⁘	·	·	·	⁜	·	·	·

Superscripts and Subscripts

U+2070 - 207F	o	i	·	·	4	5	6	7	8	9	+	−	=	(	)	n
U+2080 - 208F	o	ı	2	3	4	5	6	7	8	9	+	−	=	(	)	·
U+2090 - 209F	a	e	o	x	ə	h	k	l	m	n	p	s	t	·	·	·

Currency Symbols

U+20A0 - 20AF	·	·	·	·	·	·	·	·	·	·	·	·	€	·	·	·
U+20B0 - 20BF	₰	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·

Combining Diacritical Marks for Symbols

U+20D0 - 20DF	·	·	·	·	·	·	·	·	·	·	·	·	·	◯	·	·

Letterlike Symbols

U+2100 - 210F	·	·	·	·	·	·	·	·	Ⅎ	·	·	·	·	·	·	·
U+2110 - 211F	·	·	·	·	ℏ	·	·	·	·	·	·	·	·	·	℞	℟
U+2120 - 212F	·	·	™	℣	·	℥	Ω	·	·	·	·	·	·	·	·	·
U+2130 - 213F	·	·	Ⅎ	·	·	·	·	·	·	·	·	·	·	·	·	·
U+2140 - 214F	·	·	·	·	·	·	·	·	·	·	·	·	·	·	ⅎ	·

Number Forms

U+2150 - 215F	⅐	⅑	⅒	⅓	⅔	⅕	⅖	⅗	⅘	⅙	⅚	⅛	⅜	⅝	⅞	⅟
U+2160 - 216F	Ⅰ	Ⅱ	Ⅲ	Ⅳ	Ⅴ	Ⅵ	Ⅶ	Ⅷ	Ⅸ	Ⅹ	Ⅺ	Ⅻ	Ⅼ	Ⅽ	Ⅾ	Ⅿ
U+2170 - 217F	ⅰ	ⅱ	ⅲ	ⅳ	ⅴ	ⅵ	ⅶ	ⅷ	ⅸ	ⅹ	ⅺ	ⅻ	ⅼ	ⅽ	ⅾ	ⅿ
U+2180 - 218F	ↀ	ↁ	ↂ	Ↄ	ↄ	ↅ	↓	ↆ	ↇ	ↈ	·	·	·	·	·	·

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Arrows																
U+2190 - 219F	←	↑	→	↓	·	·	·	·	·	·	·	·	·	·	·	·
U+21A0 - 21AF	·	·	·	·	·	·	·	·	·	↩	·	·	·	·	·	·
Mathematical Operators																
U+2200 - 220F	·	·	∂	·	·	∅	Δ	·	·	·	·	·	·	·	·	∏
U+2210 - 221F	·	∑	−	·	·	∕	·	·	·	•	√	·	·	·	∞	·
U+2220 - 222F	·	·	·	·	·	·	·	∧	·	·	·	∫	·	·	·	·
U+2230 - 223F	·	·	·	·	∴	∵	·	∷	·	·	·	÷	·	·	·	·
U+2240 - 224F	·	·	·	·	·	·	·	·	≈	·	·	·	·	·	·	·
U+2260 - 226F	≠	·	·	·	≤	≥	·	·	·	·	·	·	·	·	·	·
U+22D0 - 22DF	·	·	·	·	·	·	·	⩾	·	·	·	·	·	·	·	·
Miscellaneous Technical																
U+2300 - 230F	·	·	·	·	·	·	·	·	⌈	⌉	·	·	·	·	·	·
U+2320 - 232F	·	·	·	·	·	·	·	·	·	⟨	⟩	·	·	·	·	·
U+23D0 - 23DF	·	⏡	⏢	⏤	⏥	·	·	·	·	·	·	·	·	·	·	·
Enclosed Alphanumerics																
U+2460 - 246F	①	②	③	④	⑤	⑥	⑦	⑧	⑨	⑩	⑪	⑫	⑬	⑭	⑮	⑯
U+2470 - 247F	⑰	⑱	⑲	⑳	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
U+2480 - 248F	(13)	(14)	(15)	(16)	(17)	(18)	(19)	(20)	1.	2.	3.	4.	5.	6.	7.	8.
U+2490 - 249F	9.	10.	11.	12.	13.	14.	15.	16.	17.	18.	19.	20.	(a)	(b)	(c)	(d)
U+24A0 - 24AF	(e)	(f)	(g)	(h)	(i)	(j)	(k)	(l)	(m)	(n)	(o)	(p)	(q)	(r)	(s)	(t)

Table 10.1: Encoded Glyphs in Junicode, *cont.*

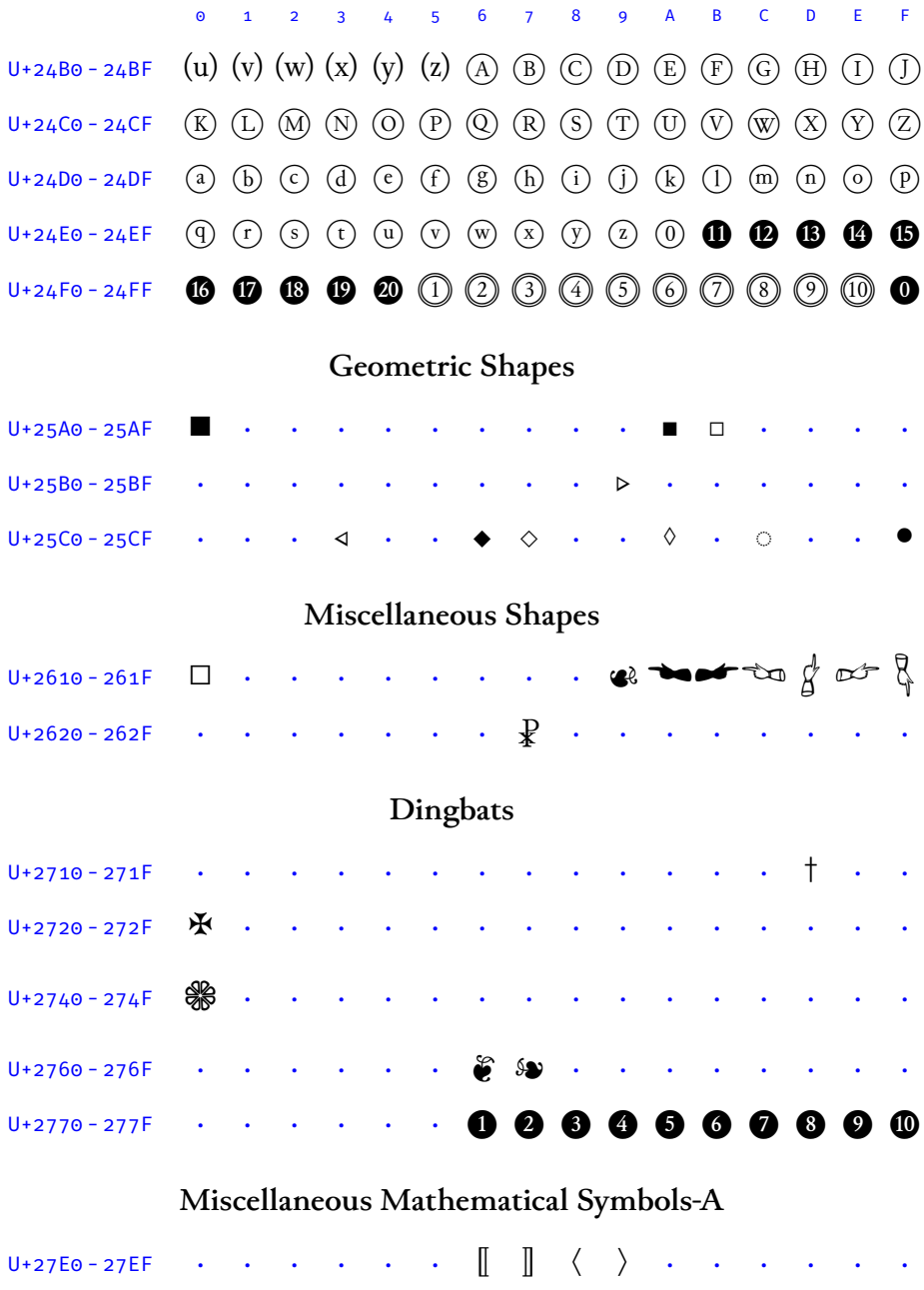


Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Supplemental Mathematical Operators																
U+2AF0 - 2AFF	·	·	·	·	·	·	·	·	·	·	·	·	·	//	·	·
Miscellaneous Symbols and Arrows																
U+2B20 - 2B2F	·	·	·	·	·	◆	◇	·	·	·	·	·	·	·	·	·
Latin Extended-C																
U+2C60 - 2C6F	·	·	·	·	·	ⱡ	·	·	·	·	·	·	·	·	·	·
U+2C70 - 2C7F	·	ⱱ	Ⱳ	ⱳ	·	Ⱨ	ⱨ	·	·	·	·	·	ⱼ	Ⱶ	·	·
Supplemental Punctuation																
U+2E00 - 2E0F	⸀	⸁	⸂	⸃	⸄	⸅	⸆	⸇	⸈	⸉	⸊	⸋	⸌	⸍	⸎	⸏
U+2E10 - 2E1F	⸐	⸑	⸒	⸓	⸔	⸕	⸖	⸗	⸘	⸙	⸚	⸛	⸜	⸝	⸞	⸟
U+2E20 - 2E2F	⸠	⸡	⸢	⸣	⸤	⸥	⸦	⸧	⸨	⸩	⸪	⸫	⸬	⸭	⸮	ⸯ
U+2E30 - 2E3F	⸰	⸱	⸲	⸳	⸴	⸵	⸶	⸷	⸸	⸹	⸺	⸻	⸼	⸽	⸾	⸿
U+2E40 - 2E4F	⹀	⹁	⹂	⹃	⹄	⹅	⹆	⹇	⹈	⹉	⹊	⹋	⹌	⹍	⹎	⹏
U+2E50 - 2E5F	⹐	⹑	⹒	⹓	⹔	⹕	⹖	⹗	⹘	⹙	⹚	⹛	⹜	⹝	⹞	⹟
CJK Symbols and Punctuation																
U+3000 - 300F		、	。	〃	〄	々	〆	〇	〈	〉	《	》	「	」	『	』
Modifier Tone Letters																
U+A710 - A71F	꜐	꜑	꜒	꜓	꜔	꜕	꜖	ꜗ	ꜘ	ꜙ	ꜚ	ꜛ	↑	↓	ꜟ	꜠
Latin Extended-D																
U+A720 - A72F	꜡	Ꜣ	ꜣ	Ꜥ	ꜥ	Ꜧ	ꜧ	Ꜩ	ꜩ	Ꜫ	ꜫ	Ꜭ	ꜭ	Ꜯ	ꜯ	ꜰ
U+A730 - A73F	ꜱ	Ꜳ	ꜳ	Ꜵ	ꜵ	Ꜷ	ꜷ	Ꜹ	ꜹ	Ꜻ	ꜻ	Ꜽ	ꜽ	Ꜿ	ꜿ	Ꝁ

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+A740 - A74F	Ꝁ	ꝁ	Ꝃ	ꝃ	Ꝅ	ꝅ	Ꝇ	ꝇ	Ꝉ	ꝉ	Ꝋ	ꝋ	Ꝍ	ꝍ	Ꝏ	ꝏ
U+A750 - A75F	Ꝑ	ꝑ	Ꝓ	ꝓ	Ꝕ	ꝕ	Ꝗ	ꝗ	Ꝙ	ꝙ	Ꝛ	ꝛ	Ꝝ	ꝝ	Ꝟ	ꝟ
U+A760 - A76F	Ꝡ	ꝡ	Ꝣ	ꝣ	Ꝥ	ꝥ	Ꝧ	ꝧ	Ꝩ	ꝩ	Ꝫ	ꝫ	Ꝭ	ꝭ	Ꝯ	ꝯ
U+A770 - A77F	ꝰ	ꝱ	ꝲ	ꝳ	ꝴ	ꝵ	ꝶ	ꝷ	ꝸ	Ꝺ	ꝺ	Ꝼ	ꝼ	Ᵹ	Ꝿ	ꝿ
U+A780 - A78F	Ꞁ	ꞁ	Ꞃ	ꞃ	Ꞅ	ꞅ	Ꞇ	ꞇ	·	·	·	·	·	·	·	·
U+A790 - A79F	·	·	Ꞓ	ꞓ	·	·	Ꞗ	ꞗ	·	·	·	·	·	·	·	·
U+A7A0 - A7AF	·	·	·	·	·	·	·	·	·	ꞩ	·	·	·	·	·	·
U+A7B0 - A7BF	·	·	·	·	·	·	·	·	Ꞹ	ꞹ	Ꞻ	ꞻ	Ꞽ	ꞽ	Ꞿ	ꞿ
U+A7C0 - A7CF	Ꟁ	ꟁ	Ꟃ	ꟃ	·	·	·	·	·	·	·	·	·	ꟍ	·	·
U+A7D0 - A7DF	Ꟑ	ꟑ	·	ꟓ	·	ꟕ	Ꟗ	ꟗ	Ꟙ	ꟙ	Ꟛ	ꟛ	Ƛ	·	·	·
U+A7F0 - A7FF	·	·	·	·	·	·	·	·	·	·	·	ꟻ	ꟼ	ꟽ	ꟾ	ꟿ

LatinExtended-E

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+AB30 - AB3F	·	·	·	·	·	ꬵ	ꬶ	·	ꬸ	·	ꬺ	ꬻ	ꬼ	·	·	·
U+AB40 - AB4F	·	·	·	·	·	·	ꭆ	·	ꭈ	ꭉ	ꭊ	ꭋ	ꭌ	ꭍ	·	·
U+AB50 - AB5F	·	·	ꭒ	ꭓ	ꭔ	·	ꭖ	ꭗ	ꭘ	·	·	·	·	·	·	·

Private Use Area

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+E000 - E00F	·	·	·	·	Ą́	·	·	·	·	·	Ā́	·	·	·	·	·
U+E010 - E01F	Ā̆	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+E020 - E02F	·	·	·	·	·	A̋	·	·	·	·	·	·	Aͤ	·	·	·
U+E030 - E03F	·	·	·	Ả	·	·	Æ̣	·	·	·	Ǣ́	·	·	Ǣ̆	·	Ǣ
U+E040 - E04F	Æ̨	Æ̋	Æ̈	Æ̇	B́	·	·	·	·	·	·	·	·	·	·	·
U+E060 - E06F	·	·	·	·	·	·	C̣	·	·	·	·	·	·	·	·	·

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+E070 - E07F	·	·	·	·	·	·	Ç	Ď	·	·	·	·	·	·	·	·
U+E080 - E08F	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	Ḍ
U+E090 - E09F	·	·	·	·	·	·	·	·	·	Ę́	·	·	·	·	·	·
U+E0B0 - E0BF	·	·	·	·	·	·	·	Ě	·	·	·	·	Ę̄	·	·	·
U+E0C0 - E0CF	·	·	·	·	·	·	·	·	Ě	·	·	·	·	·	·	·
U+E0D0 - E0DF	·	Ế	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+E0E0 - E0EF	·	Ḛ	·	·	·	·	·	·	Ę	Ė	Ę́	Ę̇	Ę̌	·	Ḟ	·
U+E0F0 - E0FF	Ḟ	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+E100 - E10F	·	Ģ	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+E110 - E11F	·	·	·	·	·	·	Ḣ	·	·	·	·	·	·	·	·	·
U+E120 - E12F	Ḥ	·	·	·	·	·	·	·	·	·	Ỉ	·	·	·	·	·
U+E130 - E13F	·	·	·	·	·	Ḯ	·	Ǐ	·	·	·	·	·	·	·	·
U+E140 - E14F	·	·	·	Ḯ	·	·	·	·	·	·	·	·	·	·	·	·
U+E150 - E15F	Ī	Į	J̄	J́	J̄	·	·	·	·	·	·	·	J̇	·	·	·
U+E160 - E16F	·	·	J̋	J̓	·	·	·	·	K̇	·	·	·	·	·	·	·
U+E190 - E19F	·	·	·	·	·	·	·	·	·	·	·	·	·	·	L̇	·
U+E1B0 - E1BF	·	·	·	·	·	·	·	·	M̄	·	·	·	·	·	·	·
U+E1D0 - E1DF	·	·	M̅	·	·	·	·	·	·	·	·	·	N̄	·	·	·
U+E200 - E20F	·	·	·	·	·	·	·	·	Ǫ	·	·	·	Ǫ́	·	·	·
U+E210 - E21F	·	·	·	·	·	·	·	·	·	·	·	Ŏ	·	·	·	·
U+E220 - E22F	·	·	·	·	·	·	·	·	·	·	·	·	·	Ô	·	·
U+E240 - E24F	·	·	·	·	Ó	·	Ő	·	·	·	·	·	·	·	·	Ǫ̇

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+E250 - E25F	·	·	Ø̄	Ø̄̆	·	Ǫ̸	·	Ǫ̸́	·	Œ́	·	·	·	Œ̄	·	·
U+E260 - E26F	Œ̄̆	·	Œ̨	·	·	·	·	·	P̋	·	·	·	·	P̣	·	·
U+E270 - E27F		·	·	·	·	·	·	·	·	·	·	·	·		·	·
U+E280 - E28F	·	·	Q̇	·	·	·	·	·	Q̣	·	·	·	·	·	·	·
U+E2E0 - E2EF	·	·	T́	·	·	·	·	·	·	·	·	·	·	·	T̨	·
U+E300 - E30F	·	·	·	·	·	·	·	·	·	Ū́	·	Ū̆	·	·	·	·
U+E310 - E31F	·	·	·	·	·	U̇	·	Ü̂	·	·	·	·	·	·	·	·
U+E320 - E32F	·	·	·	·	U̍	·	·	·	·	·	·	Uͤ	·	Uͦ	·	·
U+E330 - E33F	·	Ủ	·	·	·	·	·	Ꝥ	·	·	V́	V̂	·	·	·	·
U+E340 - E34F	·	·	V̈	·	·	·	·	·	·	·	·	V̋	V̇	V̄	V̍	·
U+E350 - E35F	W̋	·	·	Ẇͤ	·	·	·	W̄	·	·	·	·	·	·	·	·
U+E370 - E37F	·	·	·	Ȳ́	·	Ȳ̆	Y̆	·	·	·	·	·	Y̋	·	·	·
U+E380 - E38F	·	·	·	·	Ẏ́	Ỷ	·	·	·	·	·	·	·	·	·	·
U+E390 - E39F	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	Þ̣
U+E3D0 - E3DF	·	·	·	Ỏ	Ø̉	·	·	·	·	·	·	·	·	·	·	·
U+E3E0 - E3EF	·	·	·	·	·	ꝼ̣	ƿ̣	ƿ̇	·	·	·	·	·	·	·	·
U+E400 - E40F	·	·	·	·	ą́	·	·	·	·	·	ā́	·	·	·	·	·
U+E410 - E41F	ā̆	·	·	·	·	·	·	·	·	·	ā̂	·	·	ạ̈	·	å̂
U+E420 - E42F	·	·	·	·	·	a̋	·	·	·	·	·	·	aͤ	aͦ	aͮ	·
U+E430 - E43F	·	·	·	ả	·	aͧ	æ̣	·	·	·	ǣ́	·	·	ǣ̆	·	æ̆
U+E440 - E44F	æ̨	æ̋	æ̈	æ̇	b́	·	·	·	·	·	·	·	·	ƀ	·	·
U+E460 - E46F	·	·	·	·	·	·	ç̣	·	·	·	·	·	·	·	·	·

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+E470 - E47F	·	·	·	·	·	·	ç	d́	·	·	·	·	·	·	·	·
U+E480 - E48F	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	o̍
U+E490 - E49F	·	đ	·	·	·	·	·	·	e̍	ę́	·	·	·	·	·	ê
U+E4B0 - E4BF	·	·	·	·	·	·	·	ĕ	·	·	·	·	ę̄	·	·	·
U+E4C0 - E4CF	·	·	·	·	·	·	·	·	ě	·	·	·	·	ë	·	e̊
U+E4D0 - E4DF	·	e̋	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+E4E0 - E4EF	·	e̊	i̇	v̇	·	·	·	·	ę	ė	ę̋	ę̇	ę̈	·	f̣	·
U+E4F0 - E4FF	f́	e̊	e̊	·	·	·	·	·	·	·	·	·	·	·	·	·
U+E500 - E50F	·	g̨	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+E510 - E51F	·	·	·	·	·	·	ḥ	ħ	·	·	·	·	·	·	·	·
U+E520 - E52F	·	·	·	·	·	·	·	·	·	·	ɪ̀	·	·	·	·	·
U+E530 - E53F	·	·	·	·	·	ī́	·	ĭ	·	·	·	·	·	·	·	·
U+E540 - E54F	·	·	·	ı̋	·	·	·	·	i̯	·	i̊	v̇	·	·	·	·
U+E550 - E55F	ī	j̣	j̄	j́	j̄	·	·	·	·	·	·	·	·	·	·	·
U+E560 - E56F	·	·	j̋	ȷ́	·	·	·	·	k̇	·	·	·	·	·	·	·
U+E580 - E58F	·	·	·	·	·	·	·	·	·	·	·	·	Ī	·	·	·
U+E590 - E59F	·	·	·	·	·	·	Ī	·	·	·	·	·	·	·	i̇	·
U+E5A0 - E5AF	·	·	·	·	l̥	·	·	·	·	·	·	·	·	·	·	·
U+E5B0 - E5BF	·	t̄	·	·	·	·	·	·	m̄	·	·	·	·	·	·	·
U+E5C0 - E5CF	·	·	·	·	·	m̥	·	·	·	·	·	·	·	·	·	·
U+E5D0 - E5DF	·	·	m̄	·	·	·	·	n̂	·	·	·	·	n̄	·	·	·
U+E5E0 - E5EF	·	·	·	·	·	·	·	·	·	·	·	·	·	·	n̥	·
U+E600 - E60F	·	·	·	·	·	·	·	·	ǫ	·	·	·	ǫ́	·	ô	·

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+E610 - E61F	•	•	•	•	•	•	•	•	•	•	•	ō̆	•	•	•	•
U+E620 - E62F	•	•	•	•	•	•	•	•	•	•	•	•	ȯ́	ȯ̂	•	•
U+E630 - E63F	•	•	•	•	•	•	•	o̊	•	•	•	•	•	•	•	•
U+E640 - E64F	•	•	•	oͣ	oͤ	oͥ	oͧ	oͮ	•	•	•	•	•	•	•	ǫ̇
U+E650 - E65F	•	•	ø̄	ø̄̆	•	ø̨	•	ǿ̨	•	œ́	•	•	•	œ̄	•	•
U+E660 - E66F	œ̄̆	•	œ̨	•	•	p̄	•	•	p̋	•	•	•	•	p̣	•	•
U+E670 - E67F		•	•	•	•	•	•	•	•	•	•	•	•		•	•
U+E680 - E68F	•	q̄	q̇	•	•	•	•	•	q̣	•		q̃	•	•	•	•
U+E6A0 - E6AF	•	•	•	r̥	•	•	•	•	•	•	•	•	•	•	•	•
U+E6E0 - E6EF	•	•	t́	•	•	•	•	•	•	•	•	•	•	•	t̨	•
U+E700 - E70F	•	•	•	•	•	•	•	•	•	ū́	•	ū̆	•	•	•	•
U+E710 - E71F	•	•	•	•	•	u̇	•	ū̂	•	•	•	•	•	•	•	•
U+E720 - E72F	•	•	•	•	u̍	•	•	u̯	•	•	•	uͤ	uͥ	uͦ	•	•
U+E730 - E73F	•	ủ	iͣ	•	þ	ꝥ		þ́	•	•	v́	v̂	•	•	•	•
U+E740 - E74F	•	•	v̈	v̊	•	•	•	•	•	•	•	v̋	v̇	v̄	ꝟ	v̍
U+E750 - E75F	w̋	•	•	wͤ	ẘ	•	•	w̄	•	•	•	•	•	•	•	•
U+E770 - E77F	•	wͥ	•	ý	•	ȳ̆	y̆	•	•	•	•	ɏ	y̋	•	•	•
U+E780 - E78F	•	yͤ	•	•	ẏ̋	ỷ	•	•	•	•	•	•	•	•	•	•
U+E790 - E79F	•	•	•	•	•	•	•	•	•	•	•	•	•	•	ſ	þ̣
U+E7A0 - E7AF	•	•	ꝥ	•	•	•	•	•	•	•	•	•	•	•	•	•
U+E7B0 - E7BF	•	•	ꞥ	•	•	•	•	•	•	•	•	•	•	•	•	•
U+E7C0 - E7CF	•	ʒ̣	ſ̣	ꝁ	•	•	•	ꜧ	ƙ	•	•	•	ɔ̄	•	•	•
U+E7D0 - E7DF	•	•	•	ỏ	ø̉	•	•	•	•	•	•	•	•	•	•	•

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+E7E0 - E7EF	·	·	·	·	ɼ̣	ꝼ̣	ꝛ̣	ṗ	·	·	·	·	·	·	·	·
U+E8A0 - E8AF	·	ɨ̶	ɉ	hꝛ	·	·	·	·	·	·	·	·	·	ꞁ	ꝏ	ﬅ
U+E8B0 - E8BF	[illegible]	[illegible]	·	qꝛ	ꝗ	·	·	ẝ	ẜ	·	ꝟ	v̄	v̶	ẍ	x̧	qꝫ
U+E8C0 - E8CF	·	ꝥ	ħ	ħ	·	ꝁ	Ɯ	ɯ	UE	ᵫ	·	·	·	·	x̧	·
U+E8D0 - E8DF	·	æ̊	·	ǽ̨	·	ȧ̈	·	ȯ̇	·	·	Ꜹ	Y̸	Ꝑ	Pſ	Pi	Pſi
U+E8E0 - E8EF	aͥ	aͧ	eͤ	eͦ	iͣ	iͦ	iͧ	jͤ	mͤ	oͦ	rͤ	uͣ	uͮ	uᷱ	·	·
U+E8F0 - E8FF	wͣ	wͥ	wͧ	wͮ	·	·	·	·	·	·	·	·	·	·	·	·
U+EAD0 - EADF	ꝿ	ꝓ	ꝑ	·	·	·	·	·	·	·	ſt	·	ſɦ	fɦ	ſiɦ	·
U+EAF0 - EAFF	á	æ	ꜵ	ę	·	·	·	·	·	·	·	·	·	·	·	·
U+EBA0 - EBAF	fä	fh	fi	fl	fö	fp	ff	ffi	ffl	fti	ftr	fü	ſp	ſı	ſk	ſ́
U+EBB0 - EBBF	Ꜹ́	ꜹ́	ꝺ́	Ꝼ́	ꝼ́	Ɱ́	ḿ	Ỏ	ỏ	ꝛ́	Ƿ́	ƿ́	·	êa	êu	ů
U+EBC0 - EBCF	Ꜵ̋	ꜵ̋	Ꜹ̋	ꜹ̋	Ǫ̋	ǫ̋	Ø̋	ø̋	Œ̋	œ̋	Ꝡ̋	ꝡ̋	·	Ø̇	ø̇	ᴘ̇
U+EBD0 - EBDF	ʙ̇	ꝺ̇	ᴅ̇	Ꝼ̇	ꝼ̇	ꝑ̇	ꝥ̇	ꜰ̇	·	·	ʜ̇	ᴋ̇	ʟ̇	ᴍ̇	Ǫ̇	ǫ̇
U+EBE0 - EBEF	Ø̨	ø̨	J̈	j̈	Ꝏ̈	ꝏ̈	PP̈	pp̈	Ꝡ̈	ꝡ̈	Æ̉	æ̉	Ǿ	ǿ	Ø̆	ø̆
U+EBF0 - EBFF	Ꜽ	ꜽ	Ę̆	ę̆	Ă	ă	Ĭ	ĭ	Ŏ	ŏ	Ǫ̆	ǫ̆	Ø̆	ø̆	Ŭ	ŭ
U+EEC0 - EECF	·	·	bb	ꞗ	ck	ct	ꝺꝺ	ey	fä	fj	fr	ft	fü	fy	fft	ffy
U+EED0 - EEDF	fty	gg	ꝺ	ꝺ̀	ꝺ̆	ɴꞃ	pp	ꝑp	tr	tt	ꜩ	ty	tʒ	PP	gp	ʃ
U+EEE0 - EEEF	a	b	c	d	ꝺ	ð	e	f	g	h	i	j	k	l	m	n
U+EEF0 - EEFF	o	p	q	r	s	t	þ	u	v	w	x	y	z	ı	ȷ	ꝼ
U+EF00 - EF0F	·	·	·	·	·	·	·	·	·	·	·	·	ꞯ	·	·	·
U+EF10 - EF1F	·	x	·	·	·	Þ	·	·	·	·	·	·	·	·	·	·
U+EF20 - EF2F	Ġ	Ṅ	Ṙ	Ṡ	Ṫ	Ḅ	Ḍ	G̣	Ḷ	Ṃ	Ṇ	Ṛ	Ṣ	Ṭ	·	·

Table 10.1: Encoded Glyphs in Junicode, *cont.*

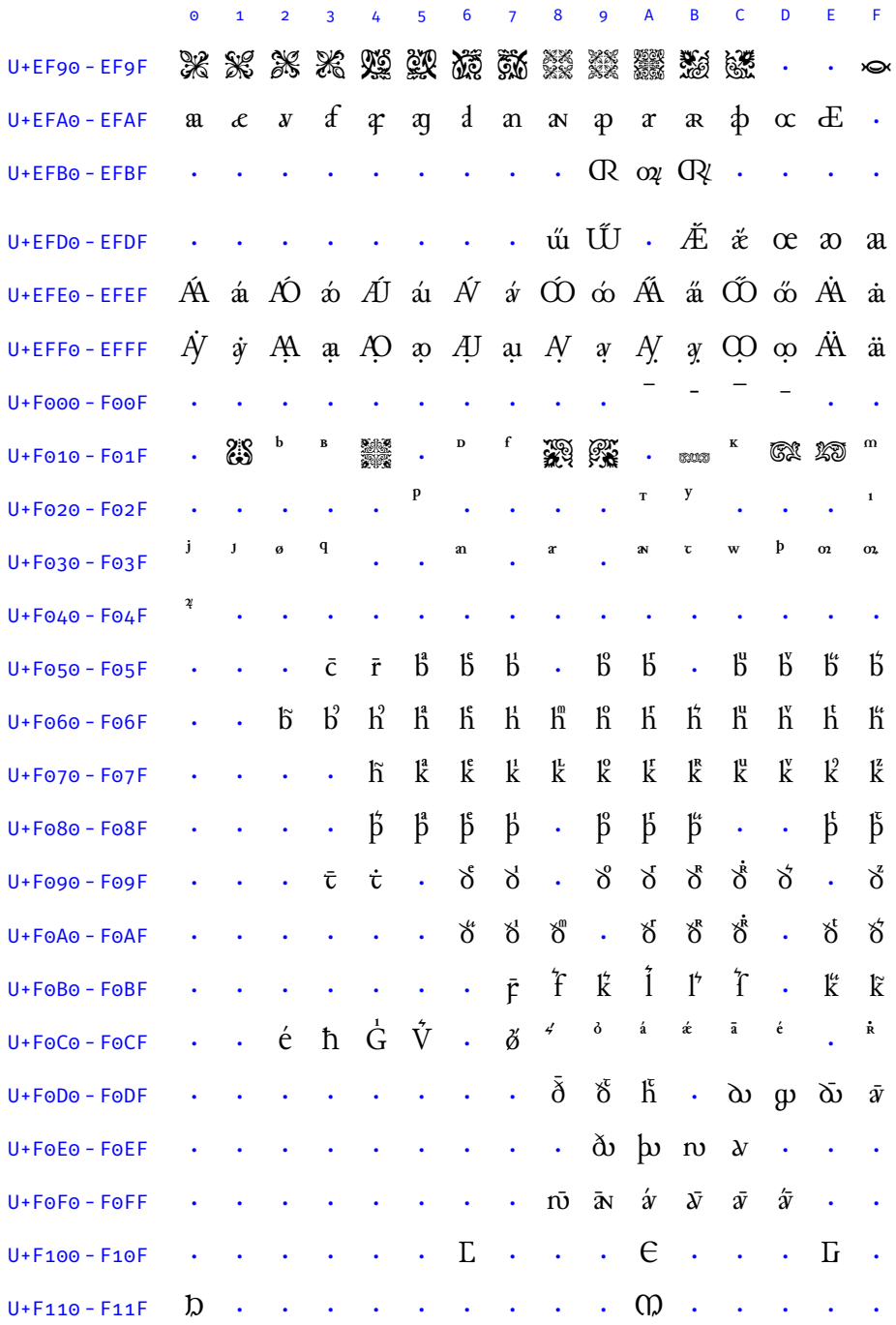


Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+F120 - F12F	·	·	·	·	·	·				·	·	·	·	·	·	·
U+F130 - F13F		·	·	·	·			·	·	·		·	·	·		
U+F140 - F14F	·	·	·	·	·	·	·	·	·		·	·	·	·	·	·
U+F150 - F15F	·	·	·	·	·	·	·	·			·	·	·	·	·	·
U+F160 - F16F				·	·	·	·	·	·	·	·	·	·	·	·	·
U+F170 - F17F	·		·	·	·		·		·		·		·	·	·	·
U+F180 - F18F	·		·		·		·		·		·		·	·	·	·
U+F190 - F19F	·		·					·					·	·	·	
U+F1A0 - F1AF	·	·	·	·	·				·	·	·	·		·	·	·
U+F1B0 - F1BF	·	·	·	·	·	·	·	·	·	·	·				·	
U+F1C0 - F1CF				·	·		·			·		·		·	·	·
U+F1D0 - F1DF	·	·		·	·	·	·	·	·	·		·	·	·	·	·
U+F1E0 - F1EF										·		·		·	·	·
U+F1F0 - F1FF				·			·							·	·	·
U+F200 - F20F											·	·	·	·	·	·
U+F210 - F21F	·	·	·	·			·									
U+F220 - F22F								·						·	·	·
U+F230 - F23F	·	·	·		·	·	·	·				·				
U+F2E0 - F2EF		·				·										
U+F2F0 - F2FF													·			
U+F4F0 - F4FF	·	·	·	·	·	·	·									
U+F700 - F70F														·	·	·
U+F710 - F71F	·	·	·	·										·	·	·

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+F750 - F75F	·	·	·	·	·	·	·	·	·	·	·	;	·	·	·	·
U+F7B0 - F7BF	·	·	V̄	X̄	L̄	C̄	D̄	·	·	·	·	·	·	·	·	·

Alphabetic Presentation Forms

U+FB00 - FB0F	ﬀ	ﬁ	ﬂ	ﬃ	ﬄ	ﬅ	ﬆ	·	·	·	·	·	·	·	·	·

Small Form Variants

U+FE50 - FE5F	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·	#
U+FE60 - FE6F	·	·	·	·	·	·	·	·	·	·	%	·	·	·	·	·

Specials ...

U+FFF0 - FFFF	·	·	·	·	·	·	·	·	·	·	·	·	·		·	·

Ancient Symbols

U+10190 - 1019F	=	–	Ⅼ	Ↄ	ↄ	»	X̶	V̶	HS̶	H̶	/	·	·	·	·	·

Gothic

U+10330 - 1033F	𐌰	𐌱	𐌲	𐌳	𐌴	𐌵	𐌶	𐌷	𐌸	𐌹	𐌺	𐌻	𐌼	𐌽	𐌾	𐌿
U+10340 - 1034F	𐍀	𐍁	𐍂	𐍃	𐍄	𐍅	𐍆	𐍇	𐍈	𐍉	·	·	·	·	·	·

Enclosed Alphanumeric Supplement

U+1F100 - 1F10F	0.	·	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+1F110 - 1F11F	(A)	(B)	(C)	(D)	(E)	(F)	(G)	(H)	(I)	(J)	(K)	(L)	(M)	(N)	(O)	(P)
U+1F120 - 1F12F	(Q)	(R)	(S)	(T)	(U)	(V)	(W)	(X)	(Y)	(Z)	·	·	·	·	·	·

Variation Selectors Supplement

U+E0030 - E003F											·	·	·	·	·	·

Table 10.1: Encoded Glyphs in Junicode, *cont.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+E0060 - E006F	·															
U+E0070 - E007F												·	·	·	·	

Supplementary Private Use Area-A

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
U+F0000 - F000F	A̸	ą	A̧	c̄	c̄	ꝺ	ꝺ̉	d̉	d̄	Ę̸	ę	f	ī	j	j̄	ľ
U+F0010 - F001F	m̄	ꝋ	ø	ꝋ	ꝋ	ꝗ	ꝣ	v̄	v̄	x̄	x̄	ˀ	ꝣ	ꝣ	A̸	ą
U+F0020 - F002F	ɸ	φ	·	·	·	·	·	·	·	·	·	·	·	·	·	·
U+F0030 - F003F	ʌ	ʙ	ᴄ	ᴅ	ᴇ	ᴇ́	ɢ	ʜ	ɪ	ᴋ	ʟ	ᴍ	ɴ	ᴏ	ᴘ	ǫ
U+F0040 - F004F	ʀ	ꜱ	ᴛ	ᴜ	x	ʏ	ᴢ	A	B	C	D	E	F	G	H	I
U+F0050 - F005F	K	L	M	N	O	P	Q	R	S	T	U	X	Y	Z	ẜ	·

Total number of glyphs shown from JunicodeVF-Roman.ttf: 3355

Because the package that produced the table above cannot handle Unicodes greater than U+FFFFF, these are listed separately below.

10A03D: S	10A022: F	10A037: N	10A01D: ꝺ
10A047: U	10A045: T	10A04F: ꝛ	10A049: y
10A026: H	10A023: G	10A007: æ	10A025: h
10A036: H	10A029: ħ	10A032: ꝑ	10A028: h
10A03F: Σ	10A044: T	10A050: ꝛ̇	10A01B: ꝺ̉
10A048: U	10A03E: r	10A033: ſ	10A051: ꝥ
10A04E: p	10A040: ſ	10A054: ꝛ	10A052: ꝺ
10A04A: V	10A031: ſ	10A003: a	10A02D: ft
10A04C: W	10A027: h	10A004: ɑ	10A02E: ft
10A008: Æ	10A053: ẜ	10A005: ɑ	10A02F: fþ
10A02B: ȷ	10A006: œ	10A039: ŋ	10A00A: bo
10A04B: V	10A021: ꝼ	10A024: ꝣ	10A00B: ꝺe
10A04D: W	10A02C: k	10A00D: ꝺ	10A00E: ꝺa

10A00F: 𐨏
10A010: 𐨐
10A011: 𐨑
10A012: 𐨒
10A013: 𐨓
10A014: 𐨔
10A015: 𐨕
10A016: 𐨖
10A017: 𐨗
10A018: 𐨘
10A034: 𐨴
10A035: 𐨵
10A03A: 𐨺
10A030: 𐨰
10A019: 𐨙
10A046: 𐩆
10A00C: 𐨌
10A03B: 𐨻
10A01F: 𐨟
10A041: 𐩁
10A03C: 𐨼
10A01E: 𐨞
10A042: 𐩂
10A043: 𐩃
10A001: 𐨁
10A002: 𐨂
10A055: 𐩕
10A056: 𐩖
10A057: 𐩗
10A058: 𐩘
10A059: 𐩙
10A05A: 𐩚
10A05B: 𐩛
10A05C: 𐩜
10A05D: 𐩝
10A05E: 𐩞
10A05F: 𐩟
10A060: 𐩠
10A061: 𐩡
10A062: 𐩢
10A063: 𐩣
10A064: 𐩤
10A065: 𐩥
10A066: 𐩦
10A067: 𐩧
10A068: 𐩨
10A069: 𐩩
10A06A: 𐩪
10A06B: 𐩫
10A06D: 𐩭
10A06E: 𐩮
10A06F: 𐩯
10A070: 𐩰
10A071: 𐩱
10A073: 𐩳
10A074: 𐩴
10A075: 𐩵
10A076: 𐩶
10A077: 𐩷
10A078: 𐩸
10A079: 𐩹
10A07A: 𐩺
10A07B: 𐩻
10A07C: 𐩼
10A07D: 𐩽
10A07E: 𐩾
10A07F: 𐩿

Index of Features and Formatting Chars

This document was set in 12pt Junicode VF
using the LuaLaTeX typesetting system with fontspec for font management.
The font for code is Fira Mono.
The sans serif font is Fira Sans.
The source for the document, JunicodeManual.tex, is available at
https://github.com/psb1558/Junicode-font.